NATIONAL AND KAPODISTRIAN UNIVERSITY OF ATHENS

SCHOOL OF SCIENCES
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS

Data Science and Information Technologies

MSc THESIS

A Comparative Analysis of Classical Machine Learning and Variational Quantum Classifiers for Breast Tumor Classification Using Fine Needle Aspiration Data

Nikolaos Z. Charisis

Supervisors: Mandilara Aikaterini, Researcher NKUA

ATHENS

FEBRUARY 2026

ΕΘΝΙΚΟ ΚΑΙ ΚΑΠΟΔΙΣΤΡΙΑΚΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΑΘΗΝΩΝ

ΣΧΟΛΗ ΘΕΤΙΚΩΝ ΕΠΙΣΤΗΜΩΝ
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ

ΕΠΙΣΤΗΜΗ ΔΕΔΟΜΕΝΩΝ & ΤΕΧΝΟΛΟΓΙΕΣ ΠΛΗΡΟΦΟΡΙΑΣ

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

Συγκριτική Ανάλυση Κλασικών Μεθόδων Μηχανικής Μάθησης και Μεταβλητών Κβαντικών Ταξινομητών για την Ταξινόμηση Όγκων Μαστού με Χρήση Δεδομένων Παρακέντησης με λεπτή βελόνα

Νικόλαος Ζ. Χαρίσης

Επιβλέποντες: Μανδηλαρά Αικατερίνη, Ερευνητής ΕΚΠΑ

ΑΘΗΝΑ

ΦΕΒΡΟΥΑΡΙΟΣ 2026

MSc THESIS

A Comparative Analysis of Classical Machine Learning and Variational Quantum Classifiers for Breast Tumor Classification Using Fine Needle Aspiration Data

Nikolaos Z. Charisis
S.N.: 7115152400008

SUPERVISORS: **Mandilara Aikaterini**, Researcher NKUA

EXAMINATION COMMITTEE: **Mandilara Aikaterini**, Researcher NKUA
Gunopulos Dimitrios, Professor NKUA
Syvridis Dimitrios, Professor NKUA

Examination Date: 1 February, 2026

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

Συγκριτική Ανάλυση Κλασικών Μεθόδων Μηχανικής Μάθησης και Μεταβλητών Κβαντικών Ταξινομητών για την Ταξινόμηση Όγκων Μαστού με Χρήση Δεδομένων Παρακέντησης με λεπτή βελόνα

Νικόλαος Ζ. Χαρίσης
Α.Μ.: 7115152400008

ΕΠΙΒΛΕΠΟΝΤΕΣ: Μανδηλαρά Αικατερίνη, Ερευνητής ΕΚΠΑ

ΕΞΕΤΑΣΤΙΚΗ ΕΠΙΤΡΟΠΗ: Μανδηλαρά Αικατερίνη, Ερευνητής ΕΚΠΑ
Γουνόπουλος Δημήτριος, Καθηγητής ΕΚΠΑ
Συβρίδης Δημήτριος, Καθηγητής ΕΚΠΑ

Ημερομηνία Εξέτασης: 1 Φεβρουαρίου 2026

ABSTRACT

The abstract details the scope and purpose of this thesis, methodology, any experiments and the results thereof. You also need to declare the subject area and up to 5 keywords to be included after the abstract. The abstract and keywords should span no more than 2 pages.

SUBJECT AREA: Quantum Machine Learning

KEYWORDS: quantum, machine, learning, classification, tumor

ΠΕΡΙΛΗΨΗ

Η περίληψη περιλαμβάνει το σκοπό-αντικείμενο της εργασίας, τη μεθοδολογία, τα κύρια βήματα που ακολουθήθηκαν και τέλος τα κύρια αποτελέσματα. Μετά το τέλος της περίληψης δηλώνεται η επιστημονική περιοχή της εργασίας και 5 λέξεις κλειδιά. Η συνολική έκταση της περίληψης και των λέξεων δήλωσης επιστημονικής περιοχής και λέξεων-κλειδιών θα είναι 1-2 σελίδες.

ΘΕΜΑΤΙΚΗ ΠΕΡΙΟΧΗ: Κβαντική Μηχανική Μάθηση

ΛΕΞΕΙΣ ΚΛΕΙΔΙΑ: κβαντική, μηχανική, μάθηση, κατηγοριοποίηση, όγκος

Optional inscription section.
Included when passing inscr *in the \documentclass.*

ACKNOWLEDGMENTS

Acknowledgment page. This page is optional. Included when passing the *ack* option in the \documentclass.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

PREFACE

This is where you provide more non-scientific details not written in the abstract or the main matter.

This page is included by adding the *preface* option in the \documentclass.

1. INTRODUCTION

Machine learning is the art and science of enabling computers to learn from data to solve problems, rather than relying on explicit programming. Conversely, quantum computing involves processing information using devices governed by the laws of quantum theory. As both fields are expected to play a pivotal role in the future of information technology, exploring their integration is a natural progression. This synergy is the core focus of the emerging discipline known as quantum machine learning.

Computers are physical devices that process information using electronic circuits. Algorithms, or software, act as instructions for manipulating information represented by electrical currents within these circuits. While these processes involve microscopic particles such as electrons, they can generally be described using classical macroscopic theories of electronics. However, when microscopic systems like photons or atoms are used directly to process information, a different mathematical framework is required to account for the counterintuitive behavior of nature at small scales. This framework, known as quantum theory, has been recognized since the early twentieth century as the most comprehensive description of microscopic physics. A computer that operates according to these principles is defined as a quantum computer.

Since the 1990s, quantum physicists and computer scientists have explored the construction and potential applications of quantum computers. During this time, they have developed various frameworks to describe computations in quantum systems, allowing for the theoretical investigation of these devices. This has led to the proposal of a vast array of quantum algorithms, currently awaiting implementation on physical hardware. The most prominent model for formulating these algorithms is the circuit model. Its foundational concepts include the qubit, which replaces the classical bit, and quantum gates, which are used to perform computations on those qubits.

Building a quantum computer in a laboratory is a formidable challenge, as it requires the precise control of infinitesimal systems. Maintaining the fragile state of quantum coherence is essential; any disturbance can destroy the quantum effects necessary for computation. To preserve this coherence across thousands of operations, error correction is vital. However, quantum error correction is significantly more complex than its classical counterpart, representing one of the primary engineering hurdles in the development of full-scale quantum computers. Consequently, the implementation of most existing quantum algorithms must await further hardware advancements.

The endeavor to bring quantum technologies to market has ushered in the era of ”near-term” quantum computing, fundamentally reshaping research in the field over recent years. Quantum computing has transitioned from a purely academic pursuit into a strategic priority for numerous startups and major IT corporations. This shift has attracted a growing number of computer scientists and engineers who contribute diverse expertise to the community. Furthermore, software toolboxes and quantum programming languages, built upon major classical frameworks, are increasingly available, with new developments emerging annually. Consequently, the realization of quantum technology has evolved into

an international, interdisciplinary, and primarily commercial effort.

Significant progress has been made in developing Noisy Intermediate-Scale Quantum (NISQ) devices, which serve as the initial prototypes for future full-scale quantum computers. These devices typically feature between 10 and 100 qubits that may not be fully interconnected. Because they lack error correction, NISQ devices produce only approximate computational results.

Quantum devices in the NISQ era theoretically possess the power to demonstrate the advantages of quantum computing; however, the requirement to restrict algorithms to a limited number of qubits and gates profoundly impacts algorithmic design. The primary objective is to identify practical computational problems that small-scale quantum devices can solve while achieving significant, ideally exponential and provable, speedups over the best classical algorithms. Consequently, there is an intense search for a ”killer app”: a compact yet powerful algorithm specifically tailored for early quantum hardware. Machine learning and its foundational component, optimization, are frequently cited as leading candidates, which accounts for the massive momentum in quantum machine learning research over recent years.

This brings us to the other parent discipline: machine learning. Situated at the intersection of statistics, mathematics, and computer science, machine learning analyzes how computers can learn from prior examples, typically large datasets characterized by complex, nonlinear relationships, to make predictions or solve new problem instances. Originally developed as the data-driven branch of artificial intelligence, it seeks to provide machines with human-like capabilities such as image recognition, language processing, and decision-making. While these tasks are intuitive for humans, codifying how a machine might acquire similar skills is difficult. For instance, when viewing a mountain panorama, it is not immediately clear how the data from a single dark blue pixel relates to the broader concept of a mountain. Machine learning addresses this by enabling computers to extract patterns from data that inherently represent concepts like a ”mountain range”.

Machine learning transitioned into an industry-driven research discipline earlier and on a much larger scale than quantum computing. This shift was primarily fueled by the success of deep learning, a paradigm that involves stacking modular algorithms known as neural networks to create expansive architectures. These systems are fed massive datasets and trained using high-performance computing. Consequently, machine learning is often characterized as a ”dark art” of empirical engineering, frequently associated with substantial professional compensation.

Machine learning also possesses a rigorous academic and theoretical dimension. The rise of deep learning presents one of the most compelling challenges in modern research: determining exactly why it works. Traditional learning theory, rooted in statistics, originally posited a fundamental trade-off between a model’s expressivity and its ability to learn effectively. However, modern neural networks are exceptionally expressive, featuring millions or even billions of trainable parameters; empirical evidence suggests that as these models grow larger, their performance actually improves. This leads to a central theoretical question: why do these heavily overparameterized models avoid overfitting the data

they encounter?

Recent years have seen impressive progress in theoretical development, and while the broader mystery remains unsolved, it is increasingly evident that effective machine learning relies on a complex interplay between data, models, and optimization algorithms. These continuous advancements place quantum machine learning on a rapidly shifting foundation, creating a landscape that is as exhilarating as it is difficult to navigate.

1.1 Classical Machine Learning Methods

Generally speaking, machine learning is categorized into two primary types: unsupervised and supervised learning. The objective of unsupervised learning is to identify inherent structures and patterns within unlabeled data. The term ”unsupervised” signifies that these algorithms discover hidden patterns independently, without requiring human intervention or pre-existing labels. These methods are typically utilized for three core tasks: clustering, anomaly detection, and dimensionality reduction.

Supervised learning is primarily characterized by the use of labeled datasets, where a model learns by processing inputs paired with their corresponding correct outputs. Using a mechanism known as a loss function, the algorithm evaluates its accuracy and iteratively adjusts its parameters until the error is sufficiently minimized. The two primary subcategories of supervised learning are regression and classification; this thesis specifically focuses on the latter.

Classification algorithms are designed to accurately categorize test data into distinct groups, such as identifying whether an image depicts a dog or a cat, or determining if an email is spam. These methods are generally divided into two categories: binary and multi-class classification. As the names imply, binary classification involves choosing between two categories, while multi-class classification involves more than two. To address the tumor binary classification task at hand, we will briefly outline six of the most prominent and effective classification methods developed over the years.

1.1.1 Logistic Regression

Logistic Regression is a widely used linear classification algorithm that models the probability of a binary outcome as a function of the input features. Despite its name, Logistic Regression is a discriminative classifier rather than a regression model. It operates by computing a linear combination of the input variables and mapping this value to the interval [0,1] using the logistic (sigmoid) function. The output is interpreted as the probability that a sample belongs to the positive class, and a decision threshold, typically 0.5, is used to assign class labels.

The model is trained by maximizing the likelihood of the observed data, which is equivalent to minimizing the log-loss (cross-entropy) function. To prevent overfitting, especially in

high-dimensional settings, regularization terms such as L1 (lasso), L2 (ridge), or elastic net penalties are often incorporated. These penalties constrain the magnitude of the model coefficients and can also induce sparsity, improving generalization and interpretability. Logistic Regression is particularly valued in biomedical applications due to its transparency, as the learned coefficients directly reflect the direction and strength of each feature's contribution to the class prediction.

1.1.2 Naive Bayes Classifier

The Naive Bayes classifier is a probabilistic generative model based on Bayes' theorem. It estimates the posterior probability of a class given a set of features by combining prior class probabilities with class-conditional feature likelihoods. The defining assumption of Naive Bayes is conditional independence: all features are assumed to be independent of each other given the class label. While this assumption is often violated in real-world data, the classifier frequently performs well in practice due to its robustness and low variance.

Different variants of Naive Bayes exist depending on the assumed distribution of the features, such as Gaussian Naive Bayes for continuous data and Multinomial Naive Bayes for count-based features. During training, the model estimates class priors and distribution parameters directly from the data, making it computationally efficient and well-suited for small datasets. In classification, the model assigns a sample to the class with the highest posterior probability. Naive Bayes is particularly attractive when interpretability, speed, and probabilistic outputs are desired.

1.1.3 Support Vector Machines

Support Vector Machines (SVM) are powerful margin-based classifiers that aim to find an optimal decision boundary separating classes with the largest possible margin. In the case of linearly separable data, SVM identifies a hyperplane that maximizes the distance between the closest data points of each class, known as support vectors. By focusing on these critical samples, SVM achieves strong generalization performance.

For non-linearly separable data, SVM employs kernel functions, such as linear, polynomial, or radial basis function (RBF) kernels, to implicitly project the data into a higher-dimensional feature space where a linear separation becomes possible. The trade-off between margin maximization and classification error is controlled by a regularization parameter, which balances model complexity and misclassification tolerance. SVMs are particularly effective in high-dimensional spaces and are widely used in biomedical classification tasks, although their lack of probabilistic output and sensitivity to hyperparameter selection can be considered limitations.

1.1.4 Linear Discriminant Analysis

Linear Discriminant Analysis (LDA) is a generative classification method that models each class as a multivariate Gaussian distribution with a shared covariance matrix. The objective of LDA is to find a linear combination of features that maximizes the separation between class means while minimizing within-class variance. This results in a linear decision boundary between classes.

LDA relies on relatively strong assumptions, including normality of features and homoscedasticity (equal covariance across classes). When these assumptions are approximately satisfied, LDA can be highly effective and statistically efficient, particularly for small sample sizes. In addition to classification, LDA also performs dimensionality reduction, projecting data onto a lower-dimensional space that preserves class separability. Due to its interpretability and computational simplicity, LDA remains a popular baseline method in biomedical and statistical pattern recognition tasks.

1.1.5 Random Forests

Random Forests are ensemble learning methods based on the aggregation of multiple decision trees. Each tree is trained on a bootstrap sample of the training data, while at each split only a random subset of features is considered. This dual randomness, both in data sampling and feature selection, reduces correlation among individual trees and improves overall model robustness.

Classification is performed by majority voting across all trees in the ensemble. Random Forests are capable of modeling complex, non-linear relationships and interactions between features without requiring explicit feature engineering. They are relatively resistant to overfitting, particularly compared to single decision trees, and can handle noisy and high-dimensional data effectively. Additionally, Random Forests provide measures of feature importance, offering insights into which variables contribute most strongly to classification decisions.

1.1.6 Light Gradient Boosting Machine

LightGBM is a gradient boosting framework based on decision trees that is designed for high efficiency and scalability. Gradient boosting builds models sequentially, where each new tree is trained to correct the errors made by the previous ensemble. LightGBM introduces several algorithmic optimizations, including histogram-based feature binning and leaf-wise tree growth, which significantly reduce training time while maintaining high predictive accuracy.

Unlike traditional level-wise tree growth, LightGBM expands the leaf with the greatest loss reduction, allowing it to capture complex patterns with fewer trees. Regularization techniques such as learning rate control, tree depth constraints, and feature subsam-

pling are employed to mitigate overfitting. LightGBM is particularly well-suited for large, high-dimensional datasets and often achieves state-of-the-art performance in classification tasks, albeit at the cost of reduced interpretability compared to linear models.

1.2 Quantum Computing and Quantum Machine Learning

Quantum theory, often used interchangeably with the term quantum mechanics, acts primarily as a mathematical framework for calculating the statistical likelihood of specific results when measuring what are known as quantum systems. These systems generally consist of microscopic entities that exhibit behaviors that cannot be explained by the traditional laws of Newtonian mechanics. Practical instances where these classical rules fail, and where quantum calculus must be applied, include the study of the hydrogen atom, the analysis of light at very low intensities, or the behavior of a few electrons within a magnetic field.

Although quantum theory is often hailed as the most precise physical framework ever devised and technically includes classical mechanics as a specialized subset, the latter remains the standard for education and general science due to its practicality. Applying quantum mechanics to large-scale systems is often unfeasible because the required calculations rapidly become overwhelmingly complex. Additionally, quantum phenomena in the macroscopic world are typically so subtle that they can be omitted without causing noticeable inaccuracies, allowing simpler classical models to serve as effective substitutes. Much like the field of machine learning, where many exact problems are impossible to compute, researchers have created specific solvable models and advanced approximation tools that enable the practical application of quantum principles across various domains, including engineering, biology, medicine, and chemistry.

A quantum computer is a computational device that directly exploits uniquely quantum mechanical effects, such as superposition and entanglement, in order to carry out operations on information. In a classical computer, data are quantified in terms of bits, whereas in a quantum computer they are quantified in terms of qubits. The fundamental concept behind quantum computation is that the quantum characteristics of particles can be utilized to encode and organize information, and that specifically designed quantum processes can manipulate this information.

Experimental demonstrations have already been performed in which quantum computational procedures were implemented using a very small number of qubits. At the same time, research in both theoretical and experimental domains is progressing at a very rapid rate. Numerous national governmental and military funding bodies support research in quantum computing, aiming to develop quantum machines for civilian applications as well as for national security objectives, including cryptanalysis.

It is broadly believed that, if large-scale quantum computers can be realized, they will be capable of solving certain problems more efficiently than any classical computer.

1.2.1 The Qubit

One of the fundamental building blocks of quantum computers is the qubit (short for quantum bit), which serves as the quantum mechanical analogue of a classical bit. In classical computing, information is represented by bits, each restricted to a value of either zero or one. In contrast, quantum computing utilizes qubits to store and encode information, allowing for a more complex representation of data.

Generally, qubits are created by manipulating and measuring systems that exhibit quantum mechanical behavior, such as superconducting circuits, photons, electrons, trapped ions, and atoms. Today, qubits can be synthesized through various physical implementations, each offering distinct advantages for specific computational tasks. Some of the most common types of qubits include:

- **Superconducting qubits**: Fabricated from superconducting materials and operated at cryogenic temperatures, these qubits are preferred for their rapid gate speeds and precise control.
- **Trapped ion qubits**: These consist of individual ions confined by electromagnetic fields. They are recognized for their long coherence times and high-fidelity measurements, though they typically operate at slower speeds than superconducting variants.
- **Quantum dots**: These tiny semiconductors capture a single electron to serve as a qubit. They offer significant potential for scalability due to their compatibility with existing semiconductor manufacturing processes.
- Photons: As individual particles of light, photons can encode quantum information and transmit it over long distances via optical fibers. Consequently, they are essential for applications in quantum communication and cryptography.

More specifically in quantum computing, a qubit is a two-state quantum system defined by the basis vectors $|0\rangle$ and $|1\rangle$. Beyond the binary limitations of classical bits, a qubit possesses the capacity to exist in a superposition, a mathematical linear combination of both basis states.

In general, a qubit, which is a quantum state of two dimensions, can be represented in the following manner:

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle \tag{1.1}$$

where α and β are probability amplitudes with complex values. The primary basis states are expressed via Dirac (bra-ket) notation, corresponding to the column vectors $\binom{1}{0}$ and $\binom{0}{1}$. Within this framework, the "$|\cdot\rangle$" notation denotes a ket, while its adjoint, the "$\langle\cdot|$", is known as a bra. This bra serves as the Hermitian conjugate of the ket and is represented as a row vector, such as:

$$\langle\psi| = \begin{pmatrix} \alpha^* & \beta^* \end{pmatrix} \tag{1.2}$$

Upon measurement, a qubit is restricted to its basis states, which in this instance are $|0\rangle$ and $|1\rangle$. The specific state to which $|\psi\rangle$ "collapses" is determined by the Born rule; specifically, the likelihood of observing $|0\rangle$ is $|\alpha|^2$, while the likelihood of observing $|1\rangle$ is $|\beta|^2$. Because these squared amplitudes dictate the probabilities of the only two available measurement results, they are required to satisfy the following constraint:

$$|\alpha|^2 + |\beta|^2 = 1 \tag{1.3}$$

Should this requirement remain unfulfilled, the state is considered unnormalized. An alternative method for verifying normalization involves calculating the vector's norm, defined as the square root of the state's inner product with itself, and checking if it equals unity. A result of exactly one confirms that the state is properly normalized. Accordingly, this criterion may be expressed as follows:

$$\|\psi\| = \sqrt{\langle\psi|\psi\rangle} = 1 \tag{1.4}$$

If a state fails to satisfy the aforementioned criteria, we can derive its normalized form by dividing the vector by its own norm, as shown below:

$$|\tilde{\psi}\rangle = \frac{|\psi\rangle}{\|\psi\|} \tag{1.5}$$

Finally, a vector space equipped with an inner product is known as a Hilbert space. In this context, a qubit is defined as a vector residing in a two-dimensional complex Hilbert space, denoted as $\mathbb{C}^2$. Excluding the global phase, a state vector may also be represented in the following manner:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle \tag{1.6}$$

This representation allows for the visualization of a qubit via the Bloch sphere. Every two-level quantum system can be mapped to a point on the surface of this unit-radius sphere ($r = 1$), uniquely determined by the angular coordinates θ and ϕ. Provided the vector remains normalized, these two parameters are sufficient to define the state. Within this geometric model, the primary basis states are positioned at antipodal points: $|\psi\rangle = |0\rangle$ occurs at $\theta = 0$, while $|\psi\rangle = |1\rangle$ occurs at $\theta = \pi$.

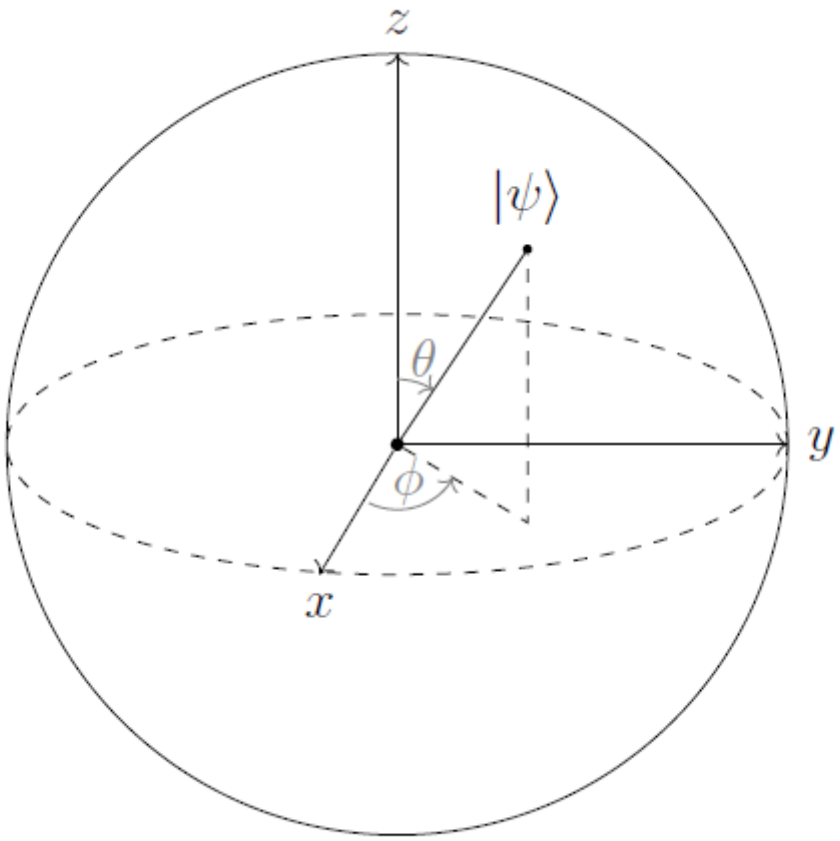

Figure 1.1: Bloch sphere representation of a qubit

1.2.2 Qubit Manipulation

Analogous to classical bits, qubits can be transitioned between different states. The mechanisms used to perform these transformations are known as operators. If an operator $\hat{A}$ is applied to a state $|\psi\rangle$, it produces a new state:

$$\hat{A}\,|\psi\rangle = |\phi\rangle \tag{1.7}$$

This principle applies not only to ”kets” but also to ”bras” in a similar fashion:

$$\hat{A}\,\langle\psi| = \langle\phi| \tag{1.8}$$

Among the most fundamental and straightforward operators are the identity operator ($\hat{I}$), which preserves the state’s original form, and the zero operator ($\hat{0}$), which maps the state onto the zero vector:

$$\hat{I}\left|\psi\right\rangle = \left|\psi\right\rangle \tag{1.9}$$

$$\hat{0}\left|\psi\right\rangle = 0 \tag{1.10}$$

A particularly vital set of operators in the field of quantum computing are the Pauli operators. This group of four includes the identity operator previously mentioned. Among the three others, the first is commonly known as the NOT operator, as it swaps the basis states. It is represented by the symbols σ_1, σ_x, or $\hat{X}$:

$$\hat{X}\left|0\right\rangle = \left|1\right\rangle, \hat{X}\left|1\right\rangle = \left|0\right\rangle \tag{1.11}$$

Following this, we encounter the σ_2, σ_y, or $\hat{Y}$ operator, which transforms the basis states according to the following rules:

$$\hat{Y}\left|0\right\rangle = -i\left|1\right\rangle, \hat{X}\left|1\right\rangle = i\left|0\right\rangle \tag{1.12}$$

Lastly, we define σ_3, σ_z, or $\hat{Z}$, which performs the following operations:

$$\hat{Z}\left|0\right\rangle = \left|0\right\rangle, \hat{Z}\left|1\right\rangle = -\left|1\right\rangle \tag{1.13}$$

In addition to its operation on basis states, an operator can be expressed in various other formats. One such method is the outer product. An outer product involving a ket and a bra is denoted as $\left|\psi\right\rangle\left\langle\phi\right|$. Applying this resulting product to a general state yields:

$$\left(\left|\psi\right\rangle\left\langle\phi\right|\right)\left|\chi\right\rangle = \left|\psi\right\rangle\left\langle\phi|\chi\right\rangle = \left\langle\phi|\chi\right\rangle\left|\psi\right\rangle \tag{1.14}$$

We can observe that our initial state $\left|\chi\right\rangle$ has been mapped to a new state proportional to $\left|\psi\right\rangle$, given that the inner product $\left\langle\phi|\chi\right\rangle$ simply evaluates to a complex scalar.

Another method for representing an operator is through matrix notation. Within an n-dimensional vector space, these operators take the form of $n \times n$ matrices. For the scope of this work, we focus on operators acting upon qubits within the two-dimensional Hilbert space $\mathbb{C}^2$; consequently, the operators under consideration are 2×2 matrices.

To determine the individual components of the matrix representation for an operator $\hat{A}$ within the standard basis, we employ the following expression:

$$\hat{A} = \begin{pmatrix} \left\langle 0|\hat{A}|0\right\rangle & \left\langle 0|\hat{A}|1\right\rangle \\ \left\langle 1|\hat{A}|0\right\rangle & \left\langle 1|\hat{A}|1\right\rangle \end{pmatrix} \tag{1.15}$$

By performing the necessary calculations for the Pauli operators, we arrive at the following matrix representations:

$$\hat{I} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \tag{1.16}$$

$$\hat{X} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \tag{1.17}$$

$$\hat{Y} = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \tag{1.18}$$

$$\hat{Z} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \tag{1.19}$$

Within the framework of quantum mechanics and quantum computing, two specific categories of operators are of primary importance: Hermitian and unitary operators. A given operator $\hat{A}$ is classified as Hermitian if it satisfies the following condition:

$$\hat{A} = \hat{A}^{\dagger} \tag{1.20}$$

The term $\hat{A}^{\dagger}$ denotes the Hermitian adjoint, which is obtained by transposing the matrix and taking the complex conjugate of every entry. An operator is classified as Hermitian if its diagonal entries consist of real numbers. A further defining trait of Hermitian operators is that they possess real eigenvalues. Due to these attributes, Hermitian operators are utilized in quantum mechanics to represent physical observables.

For an operator to be classified as unitary, its inverse must be equivalent to its Hermitian adjoint. Unitary operators are typically denoted by the symbol $\hat{U}$. Thus, by definition:

$$\hat{U}\hat{U}^{\dagger} = \hat{U}^{\dagger}\hat{U} = \hat{I} \tag{1.21}$$

Unitary operators are of great importance as they characterize the temporal evolution of a quantum state. It is worth noting that the Pauli operators belong to both the Hermitian and unitary classes.

We shall now examine the spectral decomposition theorem, which asserts that any normal operator defined on a vector space admits a diagonal matrix representation relative to a specific orthonormal basis of that space. An operator $\hat{A}$ is categorized as normal provided it fulfills the following relationship:

$$\hat{A}\hat{A}^{\dagger} = \hat{A}^{\dagger}\hat{A} \tag{1.22}$$

Following from this theorem, the operator can be represented in the subsequent form:

$$\hat{A}\hat{A}^{\dagger} = \hat{A}^{\dagger}\hat{A} \tag{1.23}$$

$$\hat{A} = \sum_i \lambda_i |u_i\rangle \langle u_i| \tag{1.24}$$

where λ_i are the eigenvalues of $\hat{A}$ and $|u_i\rangle$ is a basis.

Operators are also capable of serving as inputs for functions. By applying a Taylor series expansion, we obtain:

$$f(\hat{A}) = \sum_{n=0}^{\infty} \alpha_i A^n \tag{1.25}$$

The spectral decomposition theorem allows for the simplification of expressions involving functions of operators. If an operator $\hat{A}$ is normal, its spectral decomposition enables the preceding formula to be rewritten as:

$$f(\hat{A}) = \sum_i f(\lambda_i) |u_i\rangle \langle u_i| \tag{1.26}$$

Finally, due to the probabilistic nature of quantum mechanics, when measuring an observable represented by an operator, we do not rely on a single measurement. Instead, we prepare the quantum state $|\psi\rangle$ multiple times and average the results of these measurements. This average value, known as the expectation value of the operator, is denoted as:

$$\langle \hat{A} \rangle = \langle \psi | \hat{A} | \psi \rangle \tag{1.27}$$

1.2.3 Manipulation of Multiple Qubits

Up to this point, our focus has been limited to individual qubits, their characteristics, and their role in basic computations. To explore more sophisticated and advantageous quantum algorithms, it is vital to understand the principles of quantum mechanics as they apply to systems of multiple interacting qubits. Achieving this requires the construction of a composite Hilbert space derived from the individual Hilbert spaces of each qubit. The mathematical operation used to perform this construction is the Kronecker or tensor product, denoted by the symbol $\otimes$.

Let H_1 and H_2 denote two Hilbert spaces with dimensions N_1 and N_2, respectively. By applying the tensor product to these spaces, we can form a composite Hilbert space H with dimension $N_1 N_2$, defined such that:

$$H = H_1 \otimes H_2 \tag{1.28}$$

In a similar fashion, the tensor product can be employed to construct a state residing within the composite space H. If $|\phi\rangle \in H_1$ and $|\chi\rangle \in H_2$ are state vectors from the respective Hilbert spaces used to form H, then the following relationship holds:

$$|\psi\rangle = |\phi\rangle \otimes |\chi\rangle \tag{1.29}$$

Given that spaces composed of two-dimensional subspaces are our primary focus, the following procedure is employed to calculate the product:

$$|\psi\rangle = |\phi\rangle \otimes |\chi\rangle = \begin{pmatrix} a \\ b \end{pmatrix} \otimes \begin{pmatrix} c \\ d \end{pmatrix} = \begin{pmatrix} ac \\ ad \\ bc \\ bd \end{pmatrix} \tag{1.30}$$

Similarly, by utilizing the tensor product, we can construct a basis for our composite Hilbert space H. If $|u_i\rangle$ and $|v_i\rangle$ represent the basis vectors of H_1 and H_2, respectively, then the resulting basis for the joint space is defined as:

$$|w_i\rangle = |u_i\rangle \otimes |v_i\rangle \tag{1.31}$$

In many instances, the $\otimes$ symbol is omitted; for example, the tensor product $|\phi\rangle \otimes |\chi\rangle$ may be written as $|\phi\rangle\,|\chi\rangle$, or even more compactly as $|\phi\chi\rangle$.

Furthermore, we can define operators that act upon our composite system. Let $|\phi\rangle \in H_1$ and $|\chi\rangle \in H_2$, and consider an operator A acting on $|\phi\rangle$ and an operator B acting on $|\chi\rangle$. We can construct a new operator as the tensor product of these two, which acts on a state $|\psi\rangle \in H$ in the following manner:

$$(A \otimes B)\,|\psi\rangle = (A \otimes B)(|\phi\rangle \otimes |\chi\rangle) = (A\,|\phi\rangle) \otimes (B\,|\chi\rangle) \tag{1.32}$$

Assuming that the operators A and B act on a two-dimensional Hilbert space, the matrix representation of their tensor product is computed as follows:

$$A \otimes B = \begin{pmatrix} \alpha_{11}B & \alpha_{12}B \\ \alpha_{21}B & \alpha_{22}B \end{pmatrix} = \begin{pmatrix} \alpha_{11}\beta_{11} & \alpha_{11}\beta_{12} & \alpha_{12}\beta_{11} & \alpha_{12}\beta_{12} \\ \alpha_{11}\beta_{21} & \alpha_{11}\beta_{22} & \alpha_{12}\beta_{21} & \alpha_{12}\beta_{22} \\ \alpha_{21}\beta_{11} & \alpha_{21}\beta_{12} & \alpha_{22}\beta_{11} & \alpha_{22}\beta_{12} \\ \alpha_{21}\beta_{21} & \alpha_{21}\beta_{22} & \alpha_{22}\beta_{21} & \alpha_{22}\beta_{22} \end{pmatrix} \tag{1.33}$$

1.2.4 Quantum Entanglement

It is essential to recognize that the state of a composite system, such as a two-qubit system, cannot always be expressed as a tensor product of individual states. A system can only be written in product form if the qubits are prepared separately and kept in isolation,

effectively acting as independent closed systems. However, if the qubits are permitted to interact, they form a single integrated closed system, making a product state representation unfeasible. In such cases, the qubits are described as entangled, a fundamental phenomenon and a vital resource for quantum computation.

One of the most profound phenomena in quantum theory is quantum entanglement, the concept that particles sharing a common origin remain fundamentally linked regardless of the distance between them. Even when separated by vast stretches of time and space, these particles transcend a simple connection; they relinquish their individual quantum identities to form a single, unified quantum state. Consequently, any measurement or interaction involving one particle instantaneously influences the state of its entangled partners.

In 1935, Albert Einstein and his colleagues first identified the ”spooky” action of quantum entanglement. This phenomenon appeared to conflict with Einstein’s theory of special relativity, which postulates that nothing can travel faster than the speed of light, a principle mathematically represented by the famous equation $E = mc^2$.

The ability to instantaneously determine the quantum state of one particle by measuring its entangled partner elsewhere in the universe suggests that information must be transmitted faster than the speed of light. This apparent instantaneous influence contradicts the principles of Einstein’s theory of special relativity. Furthermore, it remains a profound mystery how these particles interact across such vast distances to maintain this shared connection.

Three decades passed before another scientist, John Stewart Bell, developed a method to test this phenomenon. His work ultimately provided the theoretical framework that enabled subsequent researchers to experimentally confirm the existence of quantum entanglement.

1.2.5 Quantum Gates

In classical computing, the fundamental operations performed on a bit of information involve moving it between memory locations or transforming it through logic gates. These gates serve as the building blocks for digital circuits, which facilitate complex calculations. Analogously, in the realm of quantum computing, transformations applied to qubit states are referred to as quantum gates, and a sequence of such operations forms a quantum circuit.

Quantum gates represent unitary operations performed on qubits; consequently, the terms ”gate” and ”unitary operator” are used interchangeably. Since operators can be described by matrices, each quantum gate corresponds to a specific unitary matrix. A gate processing n inputs and outputs is represented by a $2^n \times 2^n$ matrix. For instance, a single-qubit gate is depicted as a 2×2 matrix, whereas a quantum gate acting on two qubits requires a 4×4 matrix representation.

The most fundamental single-qubit operation is the quantum NOT gate, which is equiv-

alent to the Pauli X operator. This gate effectively swaps the basis states. Represented as a matrix in the computational basis, we have:

$$\hat{U}_{NOT} = \hat{X} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \tag{1.34}$$

The remaining three Pauli operators, $\hat{I}$, $\hat{Y}$, and $\hat{Z}$, likewise function as quantum gates, given their unitary nature. The specific operations these gates perform on a qubit have been previously detailed in Section 1.2.2 and Section **1.2.3**.

A critical component in quantum computation is the Hadamard gate, which transforms the standard basis states into a balanced superposition. The operation of the Hadamard gate on these basis states is defined as follows:

$$\hat{H}\,|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle \tag{1.35}$$

$$\hat{H}\,|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle \tag{1.36}$$

Relative to the standard basis, the Hadamard gate can be represented in the following matrix form:

$$\hat{H} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \tag{1.37}$$

We shall now investigate the method of using exponentiation to derive additional single-qubit gates. For any matrix $\hat{U}$ that is both unitary and Hermitian, it can be demonstrated via Taylor series expansion that:

$$e^{-i\theta\hat{U}} = \cos(\theta)\hat{U} - i\sin(\theta)\hat{U} \tag{1.38}$$

This implies that by exponentiating a matrix, we can construct a new quantum gate. For instance, exponentiating the Pauli matrices generates gates that perform rotations of a qubit along the x, y, and z axes of the Bloch sphere. These rotation matrices are defined as follows:

$$\hat{R}_x(\theta) = e^{-i\theta\frac{X}{2}} = \begin{pmatrix} \cos(\frac{\theta}{2}) & -i\sin(\frac{\theta}{2}) \\ -i\sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{pmatrix} \tag{1.39}$$

$$\hat{R}_y(\theta) = e^{-i\theta\frac{Y}{2}} = \begin{pmatrix} \cos(\frac{\theta}{2}) & -\sin(\frac{\theta}{2}) \\ \sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{pmatrix} \tag{1.40}$$

$$\hat{R}_z(\theta) = e^{-i\theta\frac{Z}{2}} = \begin{pmatrix} e^{-i\frac{\theta}{2}} & 0 \\ 0 & e^{-i\frac{\theta}{2}} \end{pmatrix} \tag{1.41}$$

By utilizing the rotation matrices defined in Equations **1.39**, **1.40**, and **1.41**, any arbitrary single-qubit gate can be represented as a specific sequence of rotations around the z and y axes. This approach is referred to as the Z-Y decomposition. Although these axes are frequently selected, they are not unique; any pair of non-parallel axes on the Bloch sphere could be utilized for such a decomposition. For any unitary operator U, there exist real-valued parameters a, b, c, and d such that:

$$\hat{U} = e^{ia}\hat{R}_z(b)\hat{R}_y(c)\hat{R}_z(d) \tag{1.42}$$

Quantum gate operations can be visualized through circuit diagrams, where the inputs and outputs of each unitary operator are represented by horizontal lines known as ”wires,” and the gates themselves are depicted as rectangular blocks. A representative example of such a graphical representation is shown in Figure **1.2**. The process is read from left to right: the left side indicates the initial state of the qubits prior to any operations, and gates are applied sequentially until the final output is reached on the right. In the illustrated example, a Hadamard gate and a Pauli-X gate are initially applied to the first and third qubits, respectively. Subsequently, a two-qubit U gate acts upon the first and second qubits, followed by a Z-axis rotation applied to the first qubit. Finally, a measurement is performed on the first and third qubits, represented by a meter symbol within a rectangle.

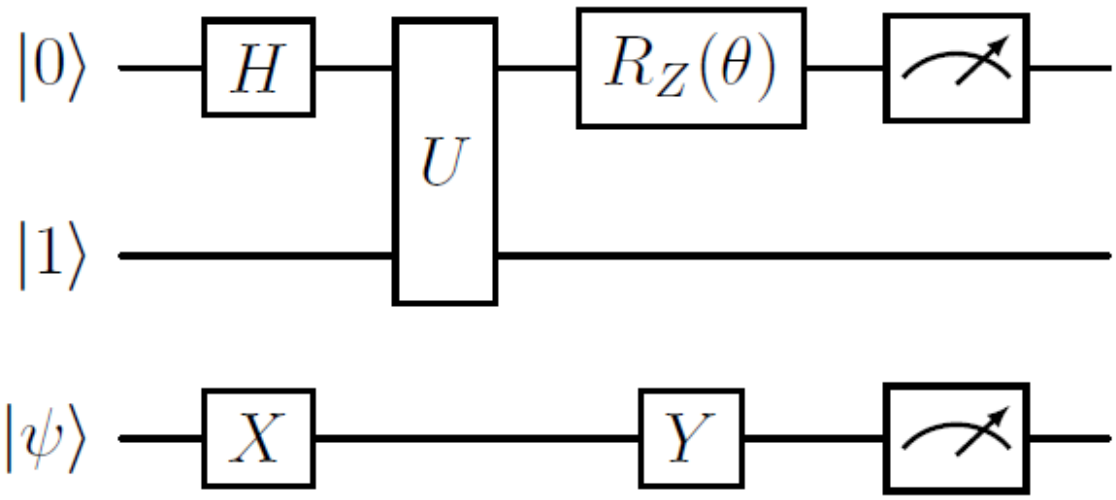

Figure 1.2: An example of a quantum circuit diagram

We shall now examine two-qubit gates, similar to the one illustrated in Figure **1.2**. Gates involving multiple qubits are frequently referred to as controlled gates, as they consist of control and target qubits. In the specific case of a two-qubit gate, one qubit acts as the control while the other serves as the target. The state of the target qubit remains unchanged if the control qubit is in the state $|0\rangle$; conversely, if the control qubit is in the

state $|1\rangle$, a specified unitary operation is applied to the target. The matrix representation of such a two-qubit controlled gate can be expressed as the following 4×4 matrix:

$$\hat{U} = \begin{pmatrix} \langle 00|\hat{U}|00\rangle & \langle 00|\hat{U}|01\rangle & \langle 00|\hat{U}|10\rangle & \langle 00|\hat{U}|11\rangle \\ \langle 01|\hat{U}|00\rangle & \langle 01|\hat{U}|01\rangle & \langle 01|\hat{U}|10\rangle & \langle 01|\hat{U}|11\rangle \\ \langle 10|\hat{U}|00\rangle & \langle 10|\hat{U}|01\rangle & \langle 10|\hat{U}|10\rangle & \langle 10|\hat{U}|11\rangle \\ \langle 11|\hat{U}|00\rangle & \langle 11|\hat{U}|01\rangle & \langle 11|\hat{U}|10\rangle & \langle 11|\hat{U}|11\rangle \end{pmatrix} \tag{1.43}$$

The specific two-qubit gate essential for the objectives of this thesis is the $controlled$-NOT or $CNOT$ gate. Under this operation, the target qubit remains unchanged if the control qubit is in the state $|0\rangle$; however, if the control qubit is in the state $|1\rangle$, the target qubit undergoes a bit-flip, as if a NOT gate had been applied. The transformations performed by the $CNOT$ gate on the complete set of basis input states are defined as follows:

$$CNOT\,|00\rangle = |00\rangle$$

$$CNOT\,|01\rangle = |01\rangle$$

$$CNOT\,|10\rangle = |11\rangle$$

$$CNOT\,|11\rangle = |10\rangle$$

Given the general matrix representation of a two-qubit gate and the specific transformations of the $CNOT$ gate on the qubit states, it follows that the $CNOT$ matrix is defined as:

$$CNOT = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \tag{1.44}$$

1.2.6 Input Encoding Techniques

There are different ways to encode or embed information into an n-qubit system described by a quantum state, and we will introduce some relevant strategies in this section. While for machine learning the question of information encoding becomes central, this is not true for many other topics in quantum computing, which is presumably why quantum computing has not had much specific terminology for such strategies. Here, we will refer to them as **basis** encoding, **amplitude** encoding, **time-evolution** encoding and **Hamiltonian** encoding.

Basis encoding associates a computational basis state of an n-qubit system (such as $|3\rangle = |0011\rangle$) with a classical n-bit string (0011). In a way, this is the most straightforward way of encoding, since each bit gets literally replaced by a qubit, and a computation acts in parallel on all bit sequences in a superposition.

If the output of the algorithm is likewise encoded into a computational basis state, the value of the amplitudes α_i of each basis state has the role to mark the result of the computation with a high enough probability of being measured. For example, if the basis state $|0011\rangle$ with amplitude α_3 has a probability $|\alpha_3|^2 > 0.5$ of being measured, repeated execution of the algorithm and measurement of the final state in the computational basis will reveal it as the most likely measurement result, and hence the overall result of the algorithm. The goal of a quantum algorithm is therefore to increase the probability or absolute square of the amplitude that corresponds to the basis state encoding the solution.

As in classical computers, basis encoding uses a binary representation of numbers. A quantum state $|\chi\rangle$ with $\chi \in \mathbb{R}$ will therefore refer to a binary representation of χ with the number n of bits that the qubit register encoding $|\chi\rangle$ provides. There are different ways to represent a real number in binary form, for example, by fixed or floating point representations. In the following, it is always assumed that such a strategy is given, and we will elaborate more on this point where the need arises.

Time-evolution encoding prescribes to associate a scalar value $\chi \in \mathbb{R}$ with the time t in the unitary evolution by a Hamiltonian H, resulting in Equation **1.45**

$$U(\chi) = e^{-i\chi H} \tag{1.45}$$

Thusly,the state after the evolution, $|\psi(\chi)\rangle = U(\chi)\,|\psi_0\rangle$, depends on χ in a way that is defined by the Hamiltonian H. In quantum machine learning, this kind of encoding is particularly popular when encoding classical trainable parameters into a quantum circuit. The most common choice are the Pauli rotation gates in which $H = \frac{1}{2}\sigma_\alpha$ and $\alpha \in x, y, z$. Successive gates or evolutions of the form $U(\chi)$ can be used to encode a real-valued vector $\chi \in \mathbb{R}^N$.

For some applications, it can be useful to encode matrices into the Hamiltonian of a time evolution, as used in the famous HHL algorithm for matrix inversion. The basic idea is to associate a Hamiltonian H with a square matrix **A**. In case **A** is not Hermitian, one can sometimes use the trick of encoding in Equation **1.46**.

$$U(\chi) = \begin{pmatrix} 0 & \mathbf{A} \\ \mathbf{A}^\dagger & 0 \end{pmatrix} \tag{1.46}$$

This allows us to perform the computations in two subspaces of the Hilbert space. Hamiltonian encoding allows us to extract and process the eigenvalues of **A**, for example, to multiply **A** or $\mathbf{A}^{-1}$ with an amplitude-encoded vector. Strategies of applying operations with a custom Hamiltonian are called **Hamiltonian simulation**.

1.2.7 Variational Quantum Classifier

Significant research and development have been dedicated to the creation of early-stage quantum processors, collectively referred to as NISQ devices. These systems currently operate with approximately 50 to 100 qubits and lack robust error correction mechanisms, meaning they can only provide approximate computational results. Furthermore, connectivity is limited, as not all qubits in these devices can interact directly with one another. While NISQ-era hardware serves as a theoretical platform for exploring the advantages of quantum computing, the necessity of limiting algorithms to a restricted number of qubits and gates poses a major challenge to algorithmic design. To circumvent these constraints, a specialized class of hybrid protocols known as variational quantum algorithms (VQAs) has been developed.[CITE]

These algorithms are characterized not by a fixed sequence of operations, but by an ”ansatz”—a structural template that defines the arrangement of gates across specific qubits. This modular framework is scalable, as it can be extended to an arbitrary number of qubits by repeating the pattern in successive layers. The term ”variational” stems from the fact that specific gates within the ansatz, typically Pauli rotations, are governed by continuous, adjustable parameters. These parameters are iteratively refined through a classical optimization process designed to minimize or maximize a cost function derived from the specific problem being addressed.

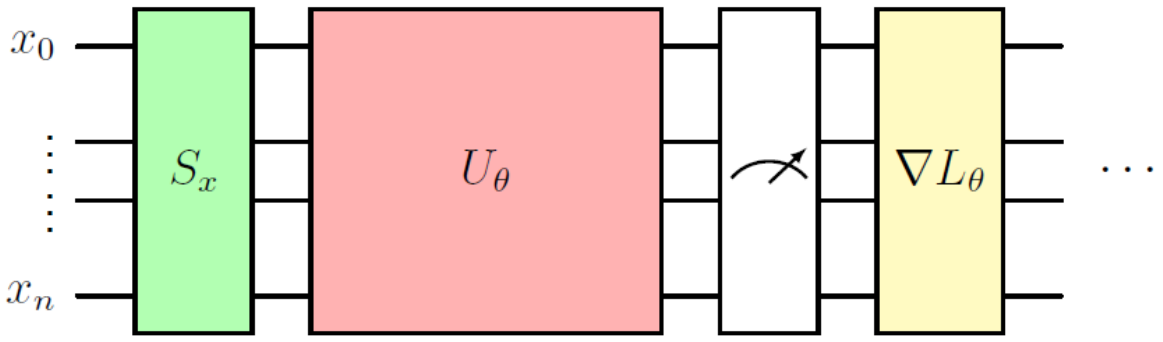

Figure 1.3: General structure of a variational quantum circuit

A graphical representation of a variational quantum circuit is provided in Figure **1.3**. The process begins with an input vector x consisting of n features. Initially, the classical data must be transformed into a quantum-compatible format through a procedure known as state preparation, denoted by the S_x block. Following this, the state undergoes unitary transformations within the U_θ block, which utilizes a combination of parameterized and non-parameterized gates. Once the transformations are complete, the system is measured. The resulting output is then processed by a classical optimizer to determine updated parameters. These new values are used to adjust the U_θ block, and the entire cycle is iterated until an optimal solution is achieved.

Variational quantum circuits can be conceptualized as a class of machine learning models [CITE] [CITE]. By applying a quantum circuit $U(x, \theta)$, which is dependent on both the input features x and the trainable parameters θ, to an initial state, the circuit functions as a model whose output is derived from the expectation value of an observable O. In the context of a classification task, this expectation value $\langle O \rangle \in \mathbb{R}$ is typically interpreted as the probability of the input belonging to a specific category. The discrepancy between the predicted expectation value and the actual label is subsequently utilized to define the cost function for optimization [CITE] [CITE].

2. RESULTS FROM CLASSICAL MACHINE LEARNING METHODS

Breast cancer is the most common malignancy among women worldwide, accounting for approximately 36% of all oncological cases [1] with numerous machine learning algorithms have been applied to support the prognosis of breast cancer tissues [2]. Breast cancer is also the most prevalent malignant tumor worldwide, with an estimated 2.089 million women diagnosed in 2018, and its incidence continues to rise across all regions of the globe [1]. The highest incidence rates are observed in industrialized countries, a trend commonly associated with factors such as unhealthy dietary habits, nicotine use, chronic stress, and low levels of physical activity.

In addition to these lifestyle-related factors, several other risk factors have been identified, including but not limited to: advanced age (with risk increasing from approximately 35 years of age), a family history of breast cancer, infection with oncogenic viruses, early onset of menstruation, late menopause, first pregnancy at an advanced age or the absence of pregnancy, and the use of hormone replacement therapy.

Fine Needle Aspiration (FNA) is a minimally invasive technique used to collect liquid or tissue samples from patients presenting with newly identified growths or masses that require diagnostic evaluation. When the lesion is superficial and palpable, the procedure can be performed by directly inserting the needle and gently aspirating the sample. In cases where the lesion is non-palpable or not externally visible, ultrasound guidance is required. This typically involves the use of a specialized endoscopic device equipped with an ultrasound probe and a needle-deployment mechanism at its tip. FNA is primarily employed for diagnostic purposes, but it also serves prognostic roles, such as identifying genetic or molecular markers that indicate susceptibility to specific chemotherapeutic or biological treatments. Additionally, it can be used therapeutically to drain abscess contents in conjunction with antibiotic therapy [3].

In recent years, various classical machine learning approaches have been proposed for predicting malignancy based on different sets of features, depending on the experimental design and methodology of each study [2], [4], [5]. In this work, we utilize a dataset derived from measurements obtained from fine needle aspirates to evaluate the predictive power of its features. A range of machine learning algorithms is examined, including Logistic Regression, Support Vector Machines, Linear Discriminant Analysis, Gaussian Naïve Bayes, LightGBM, and Random Forest classifiers, as implemented in the scikit-learn library, to distinguish between benign and malignant breast tissue samples [6].

Our analysis consists of three main steps. First, we perform an exploratory data analysis to examine the provided data, determining the number of features, their numerical ranges, and their distributions. We also check for missing values and potential class imbalances; this process allows us to understand the underlying patterns within the data and assess the complexity of the classification problem. The second step involves identifying which of the six proposed models classifies the breast cancer tumors most accurately. To achieve this, we employ repeated nested cross-validation, splitting the dataset into training and validation subsets. All necessary data cleaning and preprocessing are performed within

this stage to prevent data leakage between the subsets. Finally, once the best-performing model is identified, we fine-tune it using bootstrapping to gain a better understanding of its generalization potential. The resulting model is then trained on the entire dataset and saved. For future inference, the saved model can be loaded and applied to unseen data.

2.1 Exploratory Data Analysis

Loading the dataset using the pandas Python library, we observe that it contains 569 entries across 32 columns. One column contains the tumor prognosis, indicating whether it is malignant or benign, while another represents a unique identification number for each sample. The remaining 30 columns constitute the features, which consist of measurements taken from masses drawn via fine-needle aspirates. Specifically, these features describe the cell nuclei and are based on properties such as the radius, defined as the mean distance from the center to perimeter points; texture, representing the standard deviation of grayscale values; perimeter; area; smoothness, or the local variation in radius lengths; compactness, calculated as perimeter squared divided by area minus one; concavity, the severity of concave portions of the contour; concave points, the number of concave portions of the contour; symmetry; and fractal dimension, a measure of complexity using coastline approximation. Each of these properties is characterized in three ways: the mean value, the standard error, and the worst value, which is the mean of the three largest recorded values.

Upon loading the dataset, we find that it contains no missing or duplicate values. This is highly advantageous, as the presence of missing data would otherwise necessitate an imputation strategy to fill gaps or the removal of duplicate entries. While several reliable imputation methods exist for datasets of this nature, none can guarantee that synthesized values reflect the true underlying data; consequently, such adjustments could inadvertently skew the evaluation results. Furthermore, the absence of duplicates ensures that the sample size remains intact, as any further reduction of entries would only hinder the evaluation by making the available data even more scarce.

Examining the distribution of the two classes across the 569 samples, we notice a class imbalance between the malignant and benign categories. There are 357 instances of benign tumors and 212 instances of malignant tumors, meaning that benign cases account for 62.74% of the samples while malignant cases cover the remaining 37.25%. This imbalance must be carefully considered when splitting the dataset and creating subsets. To address this, we ensure that every split of the data is stratified, maintaining the original class proportions across all subsets.

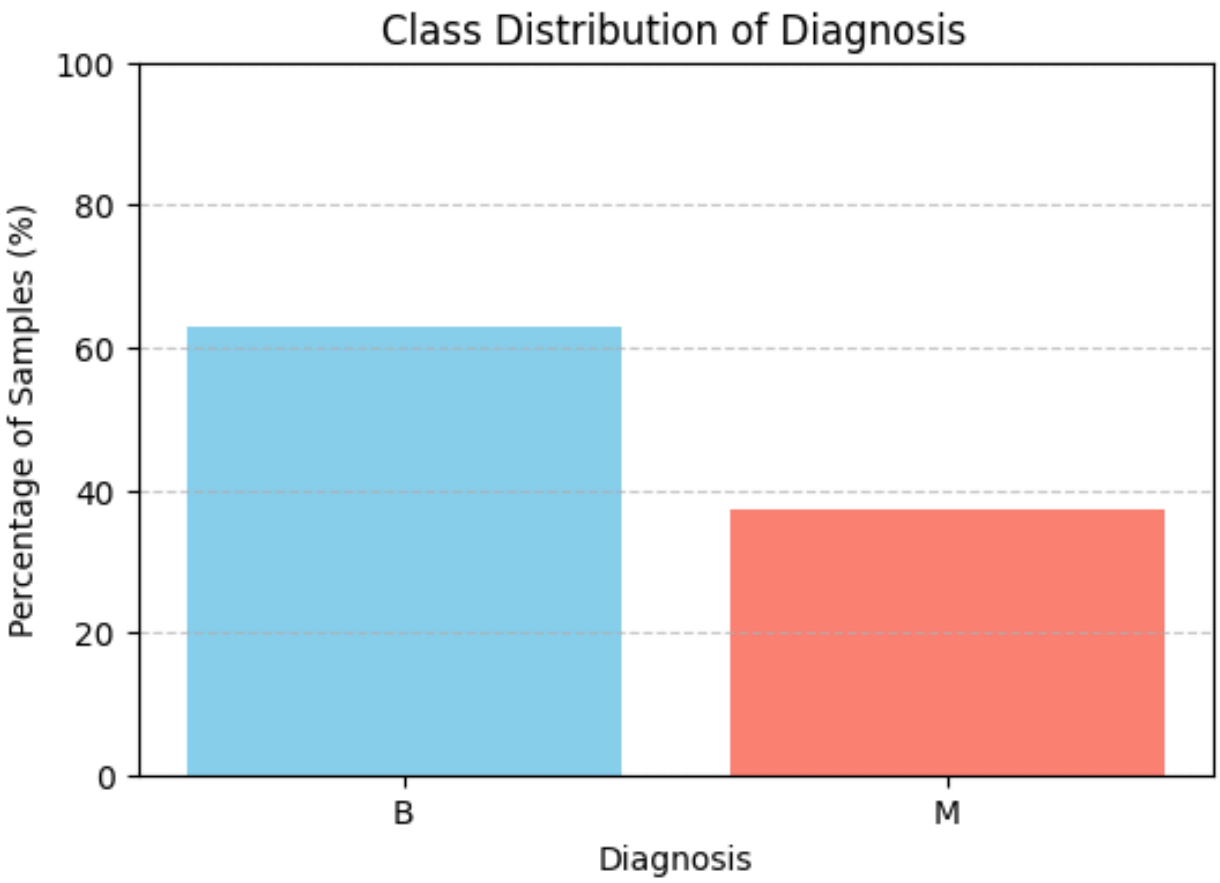

Figure 2.1: Distribution of the two tumor classes with ”B” standing for benign and ”M” standing for malignant

Observing the distribution of each feature, we notice that the majority of them exist on different scales. To account for this, we will scale them using appropriate methods during the application of the repeated nested cross-validation technique.

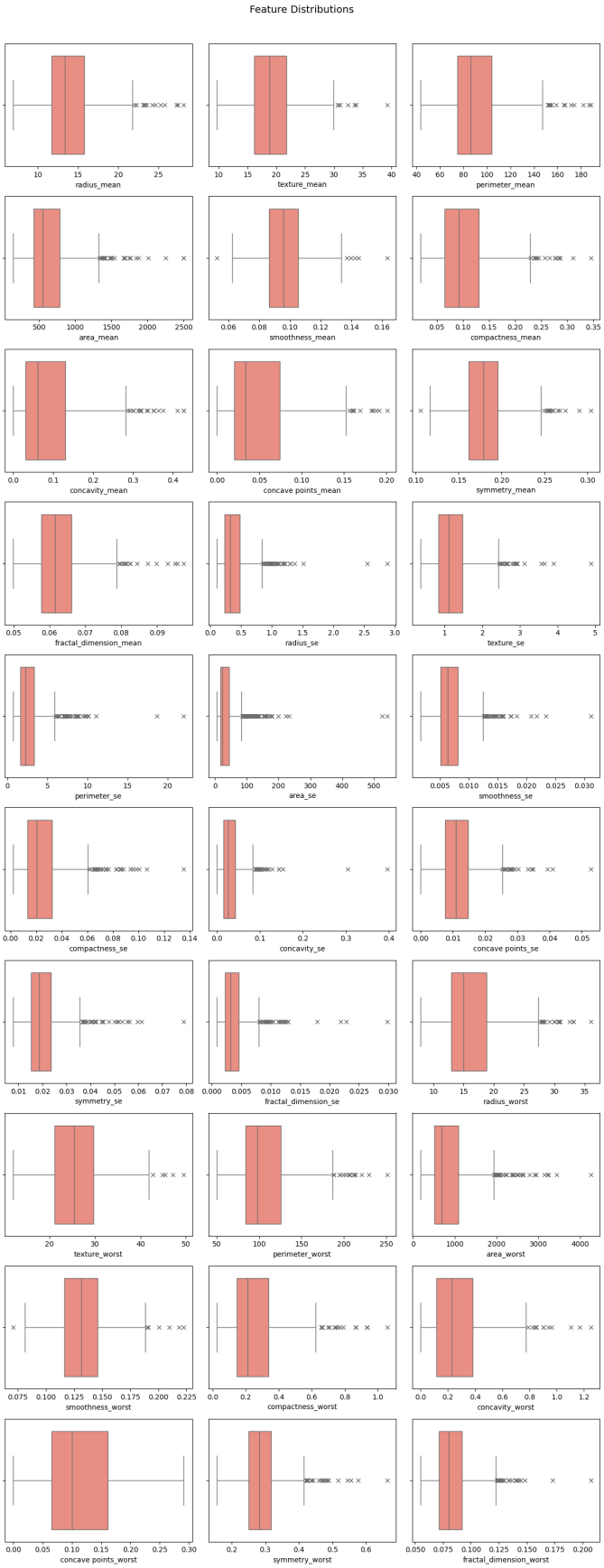

Figure 2.2: Distribution of each of the 30 features

Following this, we proceed to inspect the correlation between the features and the diagnosis label. Based on Pearson's correlation coefficient, we find that 15 out of the 30 features are heavily correlated, either positively or negatively, with the class label. These features include $radius_mean$, $perimeter_mean$, $area_mean$, $compactness_mean$, $concavity_mean$, $concavepoints_mean$, $radius_se$, $perimeter_se$, $area_se$, $radius_worst$, $perimeter_worst$, $area_worst$, $compactness_worst$, $concavity_worst$, and $concavepoints_worst$. While we could elect to retain only these features and significantly reduce the dimensionality of the problem, we believe that in a critical task like breast cancer detection, every feature provides valuable information.

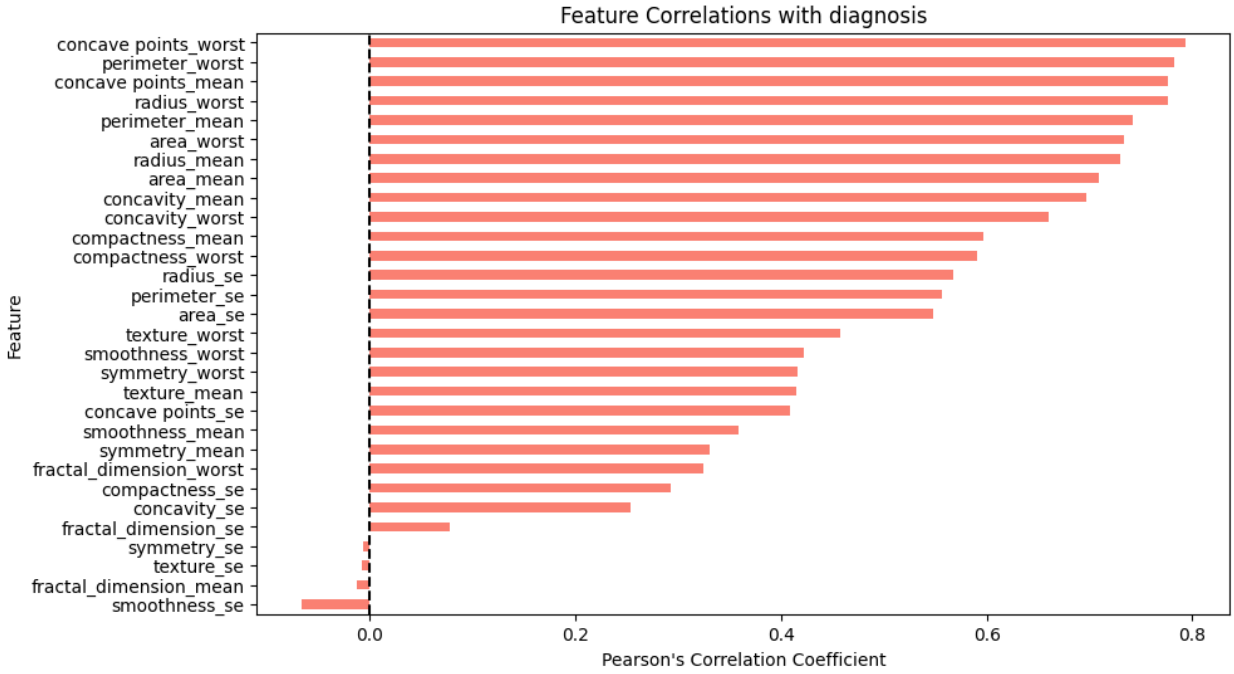

Figure 2.3: Heatmap of the correlation of each feature with the diagnosis label

Regarding the feature-wise correlation, we detect no significant linear relationships between features that are not naturally expected to be correlated. For instance, $radius_mean$ is heavily correlated with $perimeter_mean$, and $radius_worst$ shows a strong correlation with $perimeter_worst$. Such relationships are expected, as the radius is fundamentally used to calculate the perimeter in most circular or near-circular shapes.

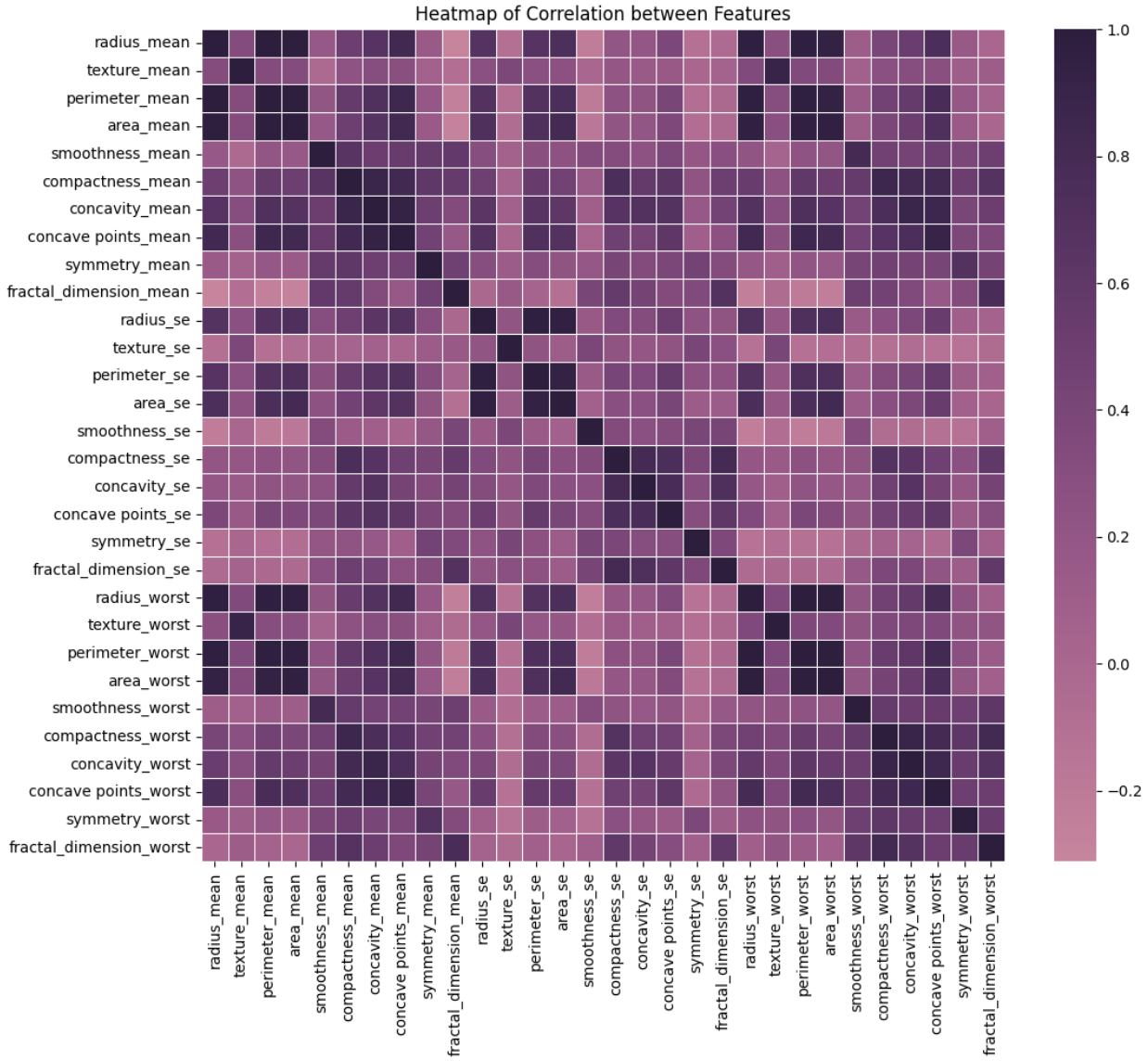

Figure 2.4: Heatmap of the pairwise feature correlation

As a final step of this analysis, we also perform principal component analysis on the entire dataset. Our purpose here is twofold. First and foremost, for our quantum implementation, we must perform some form of dimensionality reduction, as the number of features we can encode into the quantum circuit is limited. Therefore, we need to gain a rough understanding of how much information from our problem is preserved when we utilize 2, 4, 8, and 16 principal components.

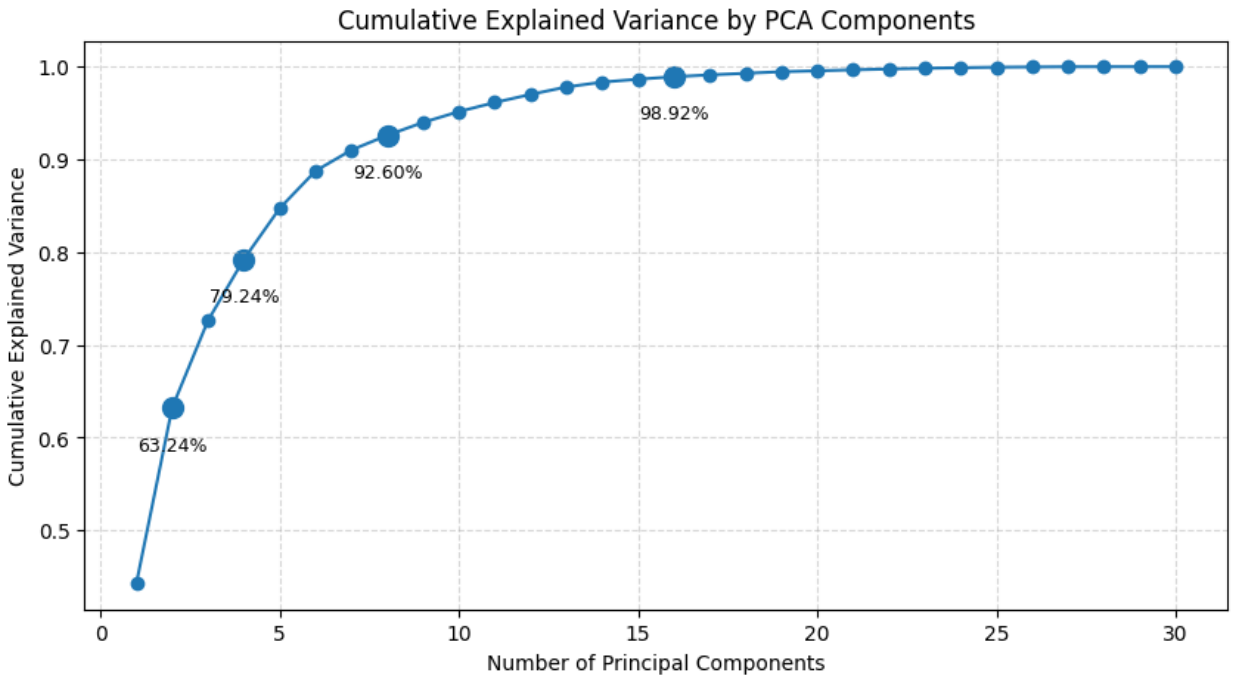

Figure 2.5: Cumulative Variance our tumor data explained by Principal Components

Secondly, PCA allows us to gain a general understanding of class separability. If only two principal components are utilized and no significant linear class separability is observed, it is likely that the classes are not linearly separable.

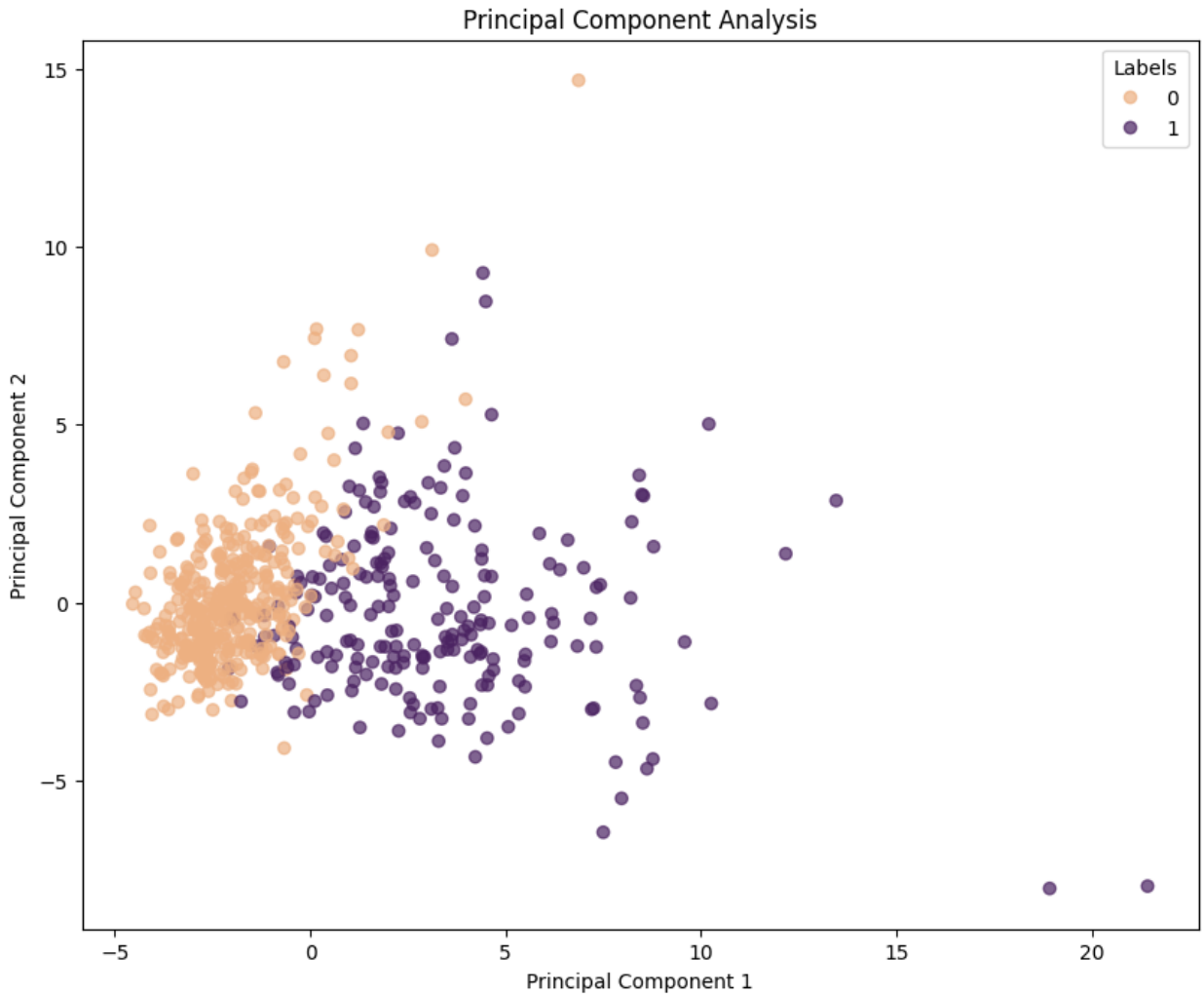

Figure 2.6: Scatterplot of the 2 principal components used to describe the entirety of our data

As a final step of the exploratory data analysis and to prepare for the evaluation of the proposed methods, we perform an 80-20 split of our dataset while ensuring that the class imbalance is preserved across both subsets. Additionally, since the models require numerical rather than categorical values, we encode our classes accordingly: the benign class, originally denoted by the letter B, is assigned the number 0, while the malignant class, originally denoted by the letter M, is assigned the number 1. Any further data processing will be conducted within the folds of the repeated nested cross-validation.

2.2 Evaluation of Classical Machine Learning Methods

To determine which classical machine learning method is most suitable for our tumor classification task, we evaluate the generalization capabilities of Logistic Regression, Support Vector Machines, Linear Discriminant Analysis, Gaussian Naive Bayes, LightGBM, and

Random Forest classifiers across nine key metrics. Since each model is characterized by its own unique set of hyperparameters, we utilize the Optuna [CITE] library in conjunction with repeated nested cross-validation, employing ten repetitions, five outer folds, and three inner folds, to obtain the scores for these metrics.

2.2.1 Classical Models and their Respective Hyperparameters

To determine the most effective classifier for the tumor classification task, six well-established supervised learning algorithms were evaluated. These models were deliberately chosen to span a broad spectrum of learning paradigms, including linear, probabilistic, discriminant, kernel-based, ensemble, and gradient boosting approaches, thereby enabling a comprehensive comparison of modeling strategies. For each classifier, hyperparameter optimization was carried out using Optuna, a Bayesian optimization framework that efficiently explores high-dimensional search spaces. This tuning process was embedded within the inner loop of a repeated nested cross-validation (rnCV) pipeline, ensuring that hyperparameter selection was strictly separated from performance estimation and thereby preventing information leakage. The first five models were implemented using the scikit-learn [CITE] library, while the gradient boosting model was implemented using the dedicated LightGBM [CITE] library.

Logistic Regression (LR) was employed as a representative linear classifier. This model estimates the log-odds of the binary target variable as a linear combination of the input features and is particularly effective when the classes are approximately linearly separable. Beyond its predictive capability, Logistic Regression offers strong interpretability, as model coefficients can be directly linked to feature influence. Hyperparameter tuning focused on the type of regularization applied to mitigate overfitting, including (l_1), (l_2), and elastic net penalties. The inverse regularization strength was also optimized, allowing control over the bias–variance trade-off, while the elastic net mixing parameter was tuned when applicable. The saga solver was selected due to its compatibility with elastic net regularization and its efficiency on larger datasets.

Gaussian Naive Bayes (GNB) was included as a probabilistic baseline model. This classifier assumes that features are conditionally independent given the class label and that each feature follows a Gaussian distribution within each class. Despite the simplicity and often unrealistic nature of these assumptions, GNB is computationally efficient and can perform surprisingly well in practice, particularly on smaller datasets. Hyperparameter tuning was limited to the variance smoothing parameter, which adds a small constant to feature variances in order to enhance numerical stability and prevent division-by-zero errors during likelihood estimation.

Linear Discriminant Analysis (LDA) was used as a classical generative model that assumes normally distributed classes with equal covariance matrices. LDA seeks a linear projection of the data that maximizes class separability by optimizing the ratio of between-class to within-class variance. This projection can also be interpreted as a dimensionality reduction step that preserves discriminative information. During tuning, different solvers

for parameter estimation were explored, along with the convergence tolerance, which governs numerical precision and computational stability.

Support Vector Machines (SVMs) were applied as a powerful kernel-based classification method capable of modeling both linear and highly non-linear decision boundaries. SVMs operate by identifying the hyperplane that maximizes the margin between classes in a transformed feature space. The flexibility of the model arises from the choice of kernel function, which was treated as a tunable parameter alongside the regularization parameter controlling the trade-off between margin maximization and classification error. Probability estimation was enabled to allow the computation of threshold-independent performance metrics such as the area under the ROC curve.

Random Forests (RF) were included as an ensemble learning method based on aggregating the predictions of multiple decision trees trained on bootstrapped subsets of the data. By combining many weak learners, Random Forests are able to capture complex non-linear relationships while reducing variance and mitigating overfitting. Hyperparameter optimization targeted the number of trees in the ensemble, the maximum depth of each tree, and the minimum number of samples required to split an internal node, all of which directly influence model complexity and generalization performance.

Finally, LightGBM was employed as a state-of-the-art gradient boosting framework based on decision trees. Unlike traditional level-wise tree growth, LightGBM adopts a leaf-wise growth strategy, which often leads to improved accuracy and faster convergence. This model is particularly well suited for tabular data and large-scale learning problems. Hyperparameter tuning focused on the number of boosting iterations, the maximum tree depth, and the learning rate, which controls the contribution of each individual tree to the ensemble. Logging verbosity was disabled to ensure clean execution during large-scale cross-validation.

2.2.2 Evaluation Metrics

Balanced Accuracy is a performance metric designed to address the limitations of standard accuracy in the presence of class imbalance. Instead of weighting all samples equally, it computes the average of the classification accuracy obtained on each class separately. In binary classification, this corresponds to the mean of sensitivity (recall) and specificity. By treating positive and negative classes symmetrically, Balanced Accuracy ensures that correct classification of the minority class contributes equally to the final score. This makes it particularly suitable for biomedical, anomaly detection, and rare-event classification tasks, where the majority class can otherwise dominate the evaluation. For reasons of brevity, it will therefore be referred to as simply Accuracy.

$$\text{Balanced Accuracy} = \frac{1}{2}\left(\frac{TP}{TP+FN} + \frac{TN}{TN+FP}\right) \tag{2.1}$$

Precision, also known as the positive predictive value, measures the proportion of pre-

dicted positive instances that are truly positive. It reflects the reliability of positive predictions made by the classifier. High precision indicates that the model produces few false positives, which is crucial in applications where incorrectly labeling a negative instance as positive is costly, such as medical diagnosis, fraud detection, or spam filtering. Precision alone, however, does not capture how many true positive instances are missed.

$$\text{Precision} = \frac{TP}{TP + FP} \tag{2.2}$$

Recall, also referred to as sensitivity or true positive rate, measures the proportion of actual positive instances that are correctly identified by the classifier. It quantifies the model's ability to detect positive cases and is particularly important when false negatives are undesirable. High recall indicates that the classifier successfully captures most of the positive samples, even if this comes at the cost of an increased number of false positives. As a result, recall is often prioritized in safety-critical and medical screening applications.

$$\text{Recall} = \frac{TP}{TP + FN} \tag{2.3}$$

The F1 score is a single metric that combines precision and recall through their harmonic mean. Unlike the arithmetic mean, the harmonic mean penalizes extreme values, meaning that the F1 score will be low if either precision or recall is low. This property makes the F1 score especially useful when a balance between false positives and false negatives is required. It is widely used in scenarios where class distributions are skewed and where both types of classification errors carry significant importance.

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN} \tag{2.4}$$

The Matthews Correlation Coefficient (MCC) is a comprehensive metric that measures the correlation between observed and predicted class labels. Unlike accuracy or F1 score, MCC incorporates all four elements of the confusion matrix, providing a balanced evaluation even when class sizes are highly unequal. Its values range from −1, indicating complete disagreement, through 0, corresponding to random prediction, to +1, representing perfect classification. Due to its robustness and interpretability, MCC is often considered one of the most reliable single-number metrics for binary classification

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \tag{2.5}$$

Area Under the Receiver Operating Characteristic Curve, or ROC AUC, evaluates the ability of a classifier to distinguish between classes across all possible decision thresholds. The Receiver Operating Characteristic (ROC) curve is constructed by plotting the true positive rate against the false positive rate as the classification threshold varies. The area under this curve represents the probability that the classifier assigns a higher score to a

randomly chosen positive instance than to a randomly chosen negative one. ROC AUC is threshold-independent and provides a global view of discriminative performance, although it can be overly optimistic in heavily imbalanced datasets.

$$\text{ROC AUC} = \int_0^1 \text{TPR}(\text{FPR})\, d(\text{FPR}), \quad \text{where TPR} = \frac{TP}{TP+FN},\ \text{FPR} = \frac{FP}{FP+TN} \tag{2.6}$$

Area Under the Precision–Recall Curve, or PR AUC, summarizes the relationship between precision and recall across different classification thresholds. The Precision–Recall curve focuses explicitly on the performance of the positive class, making it more informative than ROC AUC when dealing with strong class imbalance. In such settings, PR AUC provides a clearer picture of how well the model identifies positive samples without generating excessive false positives. Higher PR AUC values indicate better overall performance in detecting the minority class.

$$\text{PR AUC} = \int_0^1 \text{Precision}(\text{Recall})\, d(\text{Recall}) \tag{2.7}$$

Specificity, also known as the true negative rate, measures the proportion of actual negative instances that are correctly classified as negative. It reflects the model's ability to avoid false positive predictions. High specificity is critical in applications where falsely labeling a negative instance as positive can lead to unnecessary interventions or costs. Specificity complements recall, which focuses on the positive class, and together they provide a complete view of class-wise performance.

$$\text{Specificity} = \frac{TN}{TN+FP} \tag{2.8}$$

Negative Predictive Value (NPV) measures the proportion of predicted negative instances that are truly negative. It reflects the reliability of negative predictions produced by the classifier. NPV is particularly important in decision-making contexts where ruling out a condition or event with confidence is essential. Unlike specificity, which conditions on actual negatives, NPV conditions on predicted negatives, making it sensitive to class prevalence in the dataset.

$$\text{NPV} = \frac{TN}{TN+FN} \tag{2.9}$$

2.2.3 Repeated Nested Cross Validation

A central challenge in supervised machine learning is the reliable estimation of a model's generalization performance, particularly in settings where model hyperparameters must

be tuned using the available data. Hyperparameter optimization is an inherently data-driven process, and if performance is evaluated on data that has influenced this optimization, the resulting estimates are optimistically biased. This issue is especially pronounced in small or moderate-sized datasets, high-dimensional feature spaces, or when flexible models with many tunable parameters are employed. Repeated nested cross-validation (rnCV) has been developed to address these challenges by providing an evaluation framework that simultaneously minimizes bias and reduces variance in performance estimation.

In conventional k-fold cross-validation, the dataset is partitioned into k subsets (folds), with each fold used once as a validation set while the remaining folds serve as training data. The performance metrics obtained across the k folds are then averaged to yield an overall estimate. While this approach is appropriate when model hyperparameters are fixed a priori, it becomes problematic when the same cross-validation procedure is used both to select hyperparameters and to estimate predictive performance. In such cases, the model effectively "double dips" into the data: hyperparameters are chosen because they perform well on the validation folds, and the same folds are then used to report performance. This reuse of information leads to systematically inflated performance estimates and undermines the validity of model comparisons.

Nested cross-validation was introduced as a principled solution to this problem by explicitly separating the processes of model selection and model evaluation. In nested cross-validation, two cross-validation loops are employed. The outer loop is responsible for estimating generalization performance, while the inner loop is dedicated exclusively to hyperparameter tuning. Specifically, the dataset is first split into k folds for the outer cross-validation. For each outer fold, one fold is held out as a test set, and the remaining $k-1$ folds are used as training data. Within this outer training set, an inner cross-validation procedure with L folds is performed to evaluate different hyperparameter configurations. The hyperparameters that achieve the best performance in the inner loop are then selected, and a model with these hyperparameters is trained on the entire outer training set. Finally, this model is evaluated on the outer test fold. Repeating this procedure for all outer folds yields K independent estimates of test performance, which can be averaged to obtain an unbiased estimate of generalization ability.

Although nested cross-validation effectively removes the optimistic bias associated with hyperparameter tuning, it still suffers from relatively high variance. The specific partitioning of the data into folds can substantially influence both the selected hyperparameters and the resulting performance estimates. This variability is particularly evident in small datasets or in problems characterized by noisy measurements or class imbalance. As a result, conclusions drawn from a single run of nested cross-validation may be sensitive to the random choice of data splits, potentially leading to unstable model rankings or irreproducible findings.

Repeated nested cross-validation extends the nested cross-validation framework by incorporating an additional repetition layer, thereby addressing the variance issue. In rnCV, the entire nested cross-validation procedure is repeated R times, each time with a different random reshuffling of the dataset prior to fold construction. For each repetition, K-fold outer cross-validation is performed, and within each outer fold, L-fold inner cross-

validation is used for hyperparameter optimization. Across all repetitions, this results in R×K independent performance estimates obtained from strictly held-out test sets. The final performance is summarized by aggregating these estimates, typically by computing their mean and standard deviation, and optionally confidence intervals.

An important conceptual aspect of repeated nested cross-validation is the interpretation of what is being estimated. rnCV does not evaluate the performance of a single trained model instance; rather, it estimates the expected generalization performance of the entire model selection procedure. That is, it answers the question of how well one should expect the modeling pipeline, including data splitting, hyperparameter tuning, and training, to perform when applied to new, unseen data drawn from the same underlying distribution. This distinction is critical in scientific and applied settings, where the goal is often to assess the robustness and reliability of a methodological approach rather than to optimize a single model realization.

The primary motivation for using repeated nested cross-validation lies in its ability to provide both unbiased and statistically stable performance estimates. By strictly separating training, validation, and testing phases, rnCV eliminates information leakage from the hyperparameter tuning process into the performance evaluation. At the same time, repetition across multiple random data partitions substantially reduces the variance associated with any single split. This combination makes rnCV particularly well suited for fair model comparison, as it mitigates the risk of falsely declaring one algorithm superior due to favorable data partitioning or chance effects.

Repeated nested cross-validation is especially valuable in research domains such as bioinformatics, medical machine learning, and neuroscience, where datasets are often limited in size, class distributions may be imbalanced, and model interpretability and reliability are of paramount importance. In such contexts, performance estimates are frequently used to support scientific claims, guide experimental decisions, or inform clinical applications. The rigorous evaluation provided by rnCV therefore enhances the credibility and reproducibility of reported results.

Despite its methodological advantages, rnCV is computationally demanding. The total number of model trainings scales with the number of repetitions, outer folds, inner folds, and the size of the hyperparameter search space. Consequently, rnCV may be impractical for very large datasets or computationally expensive models unless sufficient computational resources are available. For this reason, rnCV is typically employed during the model assessment and comparison phase rather than for training the final model intended for deployment. Once a model class and hyperparameter ranges have been selected based on rnCV results, a final model is usually trained on the full dataset.

In summary, repeated nested cross-validation represents a rigorous and statistically sound framework for model evaluation in supervised learning. By combining the unbiased estimation properties of nested cross-validation with the variance-reducing benefits of repetition, rnCV provides a reliable estimate of the generalization performance of a complete model selection pipeline. For studies in which methodological validity, fairness of comparison, and reproducibility are critical, rnCV constitutes a gold-standard evaluation strategy.

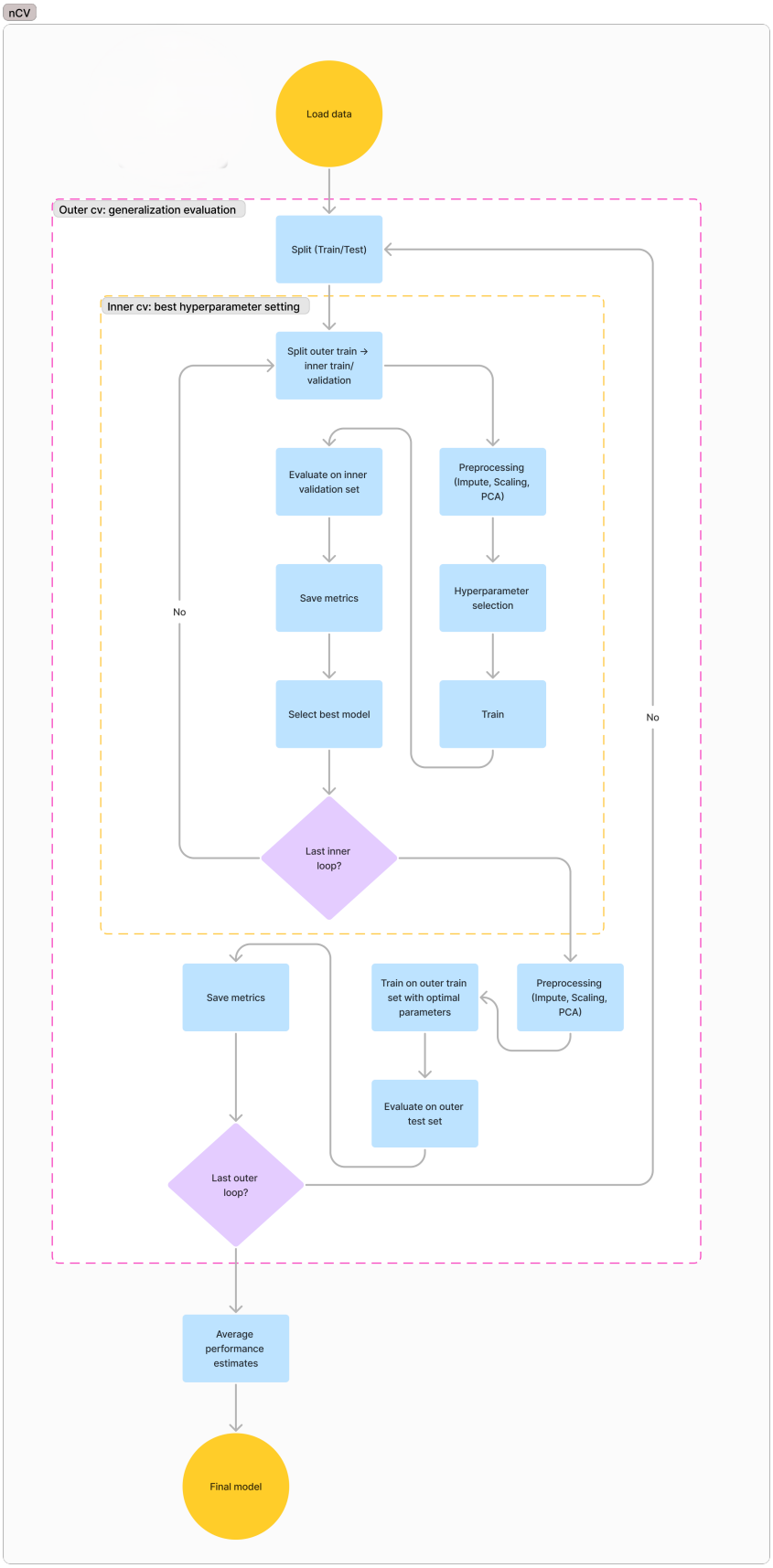

Figure 2.7: Flowchart of the nested cross validation technique

2.3 Finding the Best Classical Model

Upon completing the rnCV technique in approximately 25 minutes, we were able to identify the optimal model by observing the number of metrics for which each model was the top performer. As the results indicate, Logistic Regression was the superior model in eight of the nine metrics, lagging only in ROC AUC, where it was marginally surpassed by SVM. This is a particularly advantageous outcome, as Logistic Regression allows us not only to achieve high classification performance but also to gain meaningful insights into which features contribute to the classification process and to what degree.

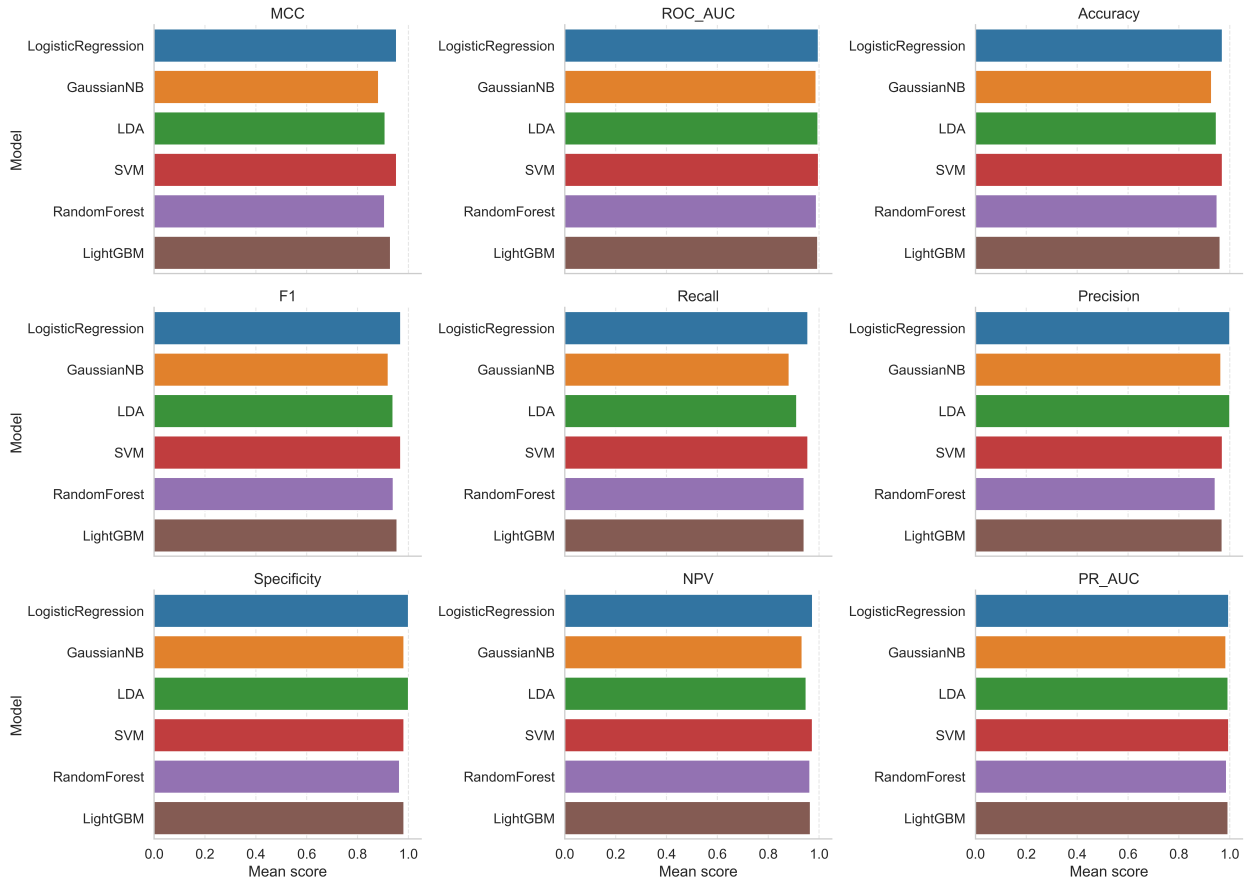

Figure 2.8: Qualitative performance of the 6 models across the 9 metrics

Taking a more quantitative look into our rnCV results, we report that Logistic Regression scored 97.06% in accuracy, 95.59% in recall, 100% in precision, 96.97% in F1-score, 100% in specificity, 97.40% in NPV, 99.56% in PR AUC, 99.69% in ROC AUC, and 95.33% in MCC. GNB achieved 92.81% in accuracy, 88.24% in recall, 96.49% in precision, 92.06% in F1-score, 98.25% in specificity, 93.33% in NPV, 98.45% in PR AUC, 98.79% in ROC AUC, and 88.28% in MCC. LDA scored 94.71% in accuracy, 91.18% in recall, 100% in precision, 93.94% in F1-score, 100% in specificity, 94.92% in NPV, 99.33% in PR AUC, 99.56% in ROC AUC, and 90.80% in MCC. SVM reached 97.06% in accuracy, 95.59%

in recall, 97.06% in precision, 96.97% in F1-score, 98.25% in specificity, 97.36% in NPV, 99.53% in PR AUC, 99.72% in ROC AUC, and 95.30% in MCC. Random Forest recorded 95.01% in accuracy, 94.12% in recall, 94.29% in precision, 94.03% in F1-score, 96.49% in specificity, 96.40% in NPV, 98.70% in PR AUC, 98.94% in ROC AUC, and 90.61% in MCC. Finally, LightGBM scored 96.18% in accuracy, 94.12% in recall, 96.97% in precision, 95.52% in F1-score, 98.25% in specificity, 96.55% in NPV, 99.28% in PR AUC, 99.46% in ROC AUC, and 92.94% in MCC.

We can observe that while Logistic Regression is the top performer across most metrics, SVM remains highly competitive. In fact, SVM slightly surpasses Logistic Regression in the ROC AUC metric and achieves identical scores for accuracy, recall, and F1-score. However, Logistic Regression maintains a slight edge in the other key metrics. Two particularly noteworthy points are the perfect scores it achieves in precision and specificity, as well as its superior performance in MCC, which is a critical metric for evaluating the quality of binary classification tasks.

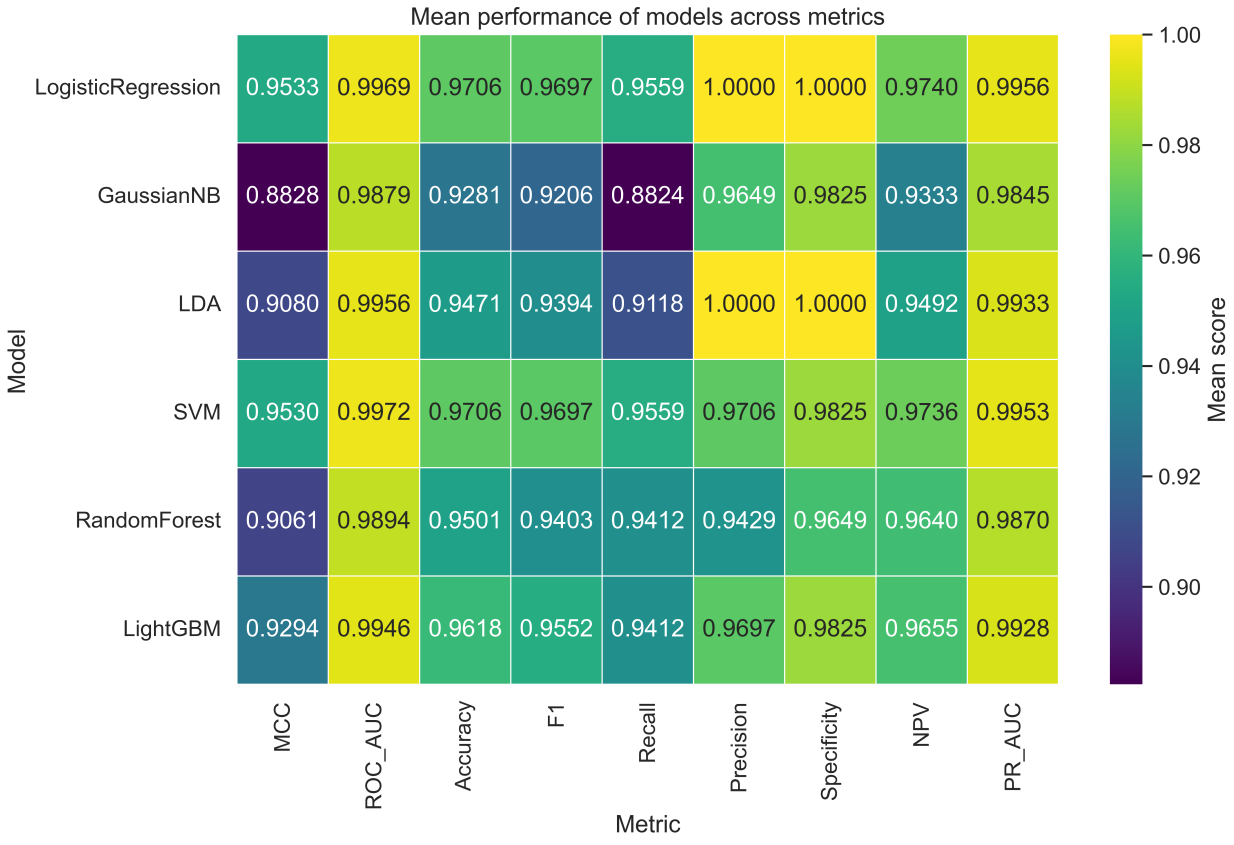

Figure 2.9: Quantitative performance of the 6 models across the 9 metrics

2.4 Finetuning the Best Model

Since we have determined that Logistic Regression is the most suitable model for our tumor classification task, we utilize bootstrapping to further assess its performance. For this purpose, we use the validation subset to generate five thousand bootstrap samples and produce 95% confidence intervals for each metric. Finally, this top-performing model is trained and fine-tuned on the entirety of the original dataset before being saved for future inference.

2.4.1 Bootstrapping

Bootstrapping is a resampling-based statistical technique that is widely used to quantify the uncertainty of an estimated statistic when only a single dataset is available. In the context of machine learning model evaluation, bootstrapping is primarily employed to approximate the sampling distribution of a performance metric, such as accuracy, area under the ROC curve (AUC), F1-score, or mean squared error, without relying on parametric assumptions about the underlying data distribution. The fundamental principle of bootstrapping is that the empirical dataset can be treated as a proxy for the unknown population, allowing repeated resampling with replacement to simulate the process of drawing new datasets from the same population.

The main motivation for using bootstrapping lies in its ability to provide insight into the variability and reliability of a model's performance. A single evaluation on a validation set yields only a point estimate, which may be highly dependent on the particular samples included in that set. Bootstrapping addresses this limitation by generating a distribution of performance estimates, thereby allowing the researcher to assess how sensitive the reported performance is to variations in the validation data. This is especially important in machine learning applications where datasets are often limited in size and where performance metrics may exhibit substantial variability.

Unlike classical statistical approaches, bootstrapping does not assume that performance estimates follow a normal distribution, nor does it rely on large-sample asymptotic properties. This makes it particularly suitable for complex machine learning models and non-standard evaluation metrics, whose sampling distributions are often unknown or skewed. As a result, bootstrapping is frequently preferred in applied settings such as biomedical data analysis, where strict parametric assumptions are difficult to justify and where robust uncertainty estimation is essential.

In practical model evaluation, bootstrapping is typically applied after the best-performing model has already been selected. Model selection, including hyperparameter tuning and architecture choice, should be performed using training and validation data, often through cross-validation. Once the final model configuration has been fixed, bootstrapping is used solely for performance assessment and uncertainty quantification. Importantly, during the bootstrapping procedure the model remains unchanged and is not retrained; only the validation data are resampled.

The bootstrapping procedure begins by drawing multiple bootstrap samples from the validation set. Each bootstrap sample is created by sampling with replacement and has the same size as the original validation set. Due to the nature of sampling with replacement, some observations appear multiple times in a given bootstrap sample, while others may be omitted entirely. On average, approximately 36.8% of the original validation samples are excluded from any single bootstrap sample. For each resampled dataset, the fixed model is evaluated and the chosen performance metric is computed.

Repeating this process a large number of times, typically between 1,000 and 10,000 iterations, yields an empirical distribution of the performance metric. This distribution provides a direct estimate of the variability of the model's performance and enables the computation of summary statistics such as the mean, standard deviation, and confidence intervals. A commonly reported measure is the 95% confidence interval, which is usually obtained using the percentile method by taking the 2.5th and 97.5th percentiles of the bootstrap distribution. The resulting interval represents a range within which the true performance metric is expected to lie with 95% confidence.

Reporting a 95% confidence interval alongside the mean performance provides a much more informative assessment than a single point estimate alone. For example, stating that a model achieves an AUC of 0.87 with a 95% confidence interval of [0.84, 0.90] conveys both the expected performance and the associated uncertainty. Narrow confidence intervals indicate a stable and robust model, whereas wide intervals may suggest sensitivity to data variability or insufficient sample size.

It is important to emphasize that bootstrapping is not intended to replace cross-validation or improve model performance. Instead, it complements model selection procedures by offering a principled way to assess the reliability of the final model's evaluation. Furthermore, bootstrapping should be applied to the validation set rather than the test set, as the latter is typically reserved for a single, unbiased estimate of generalization performance. Using bootstrapping appropriately ensures that performance claims are statistically grounded and scientifically credible.

In summary, bootstrapping provides a powerful and flexible framework for estimating the uncertainty of machine learning model performance. By generating an empirical distribution of evaluation metrics through repeated resampling of the validation data, it enables the computation of meaningful statistics such as 95% confidence intervals. This approach is particularly well suited for small or complex datasets and is strongly recommended in rigorous academic work, where transparent reporting of uncertainty is as important as reporting performance itself.

2.4.2 95% Confidence Intervals based on bootstrapping

After bootstrapping five thousand sets on our Logistic Regression model using the validation subset, we obtained the 95% confidence intervals for the nine key metrics. Accuracy lies within the [0.9595, 1] interval, while precision and specificity both maintain a perfect [1, 1] interval. Recall is situated between [0.9189, 1.000], and the F1-score falls within

[0.9577, 1]. Additionally, the NPV is in the [0.9552, 1] range, and the MCC, a crucial metric for binary classification, lies within [0.9395, 1]. Both the ROC AUC and PR AUC remain exceptionally high, with intervals of [0.9919, 1] and [0.9877, 1], respectively. Notably, every metric reaches a perfect upper bound of 1.0, while the lower bounds remain consistently above 91%. Furthermore, those metrics that demonstrated perfect mean scores during the rnCV process continue to exhibit high performance throughout the bootstrapping analysis.

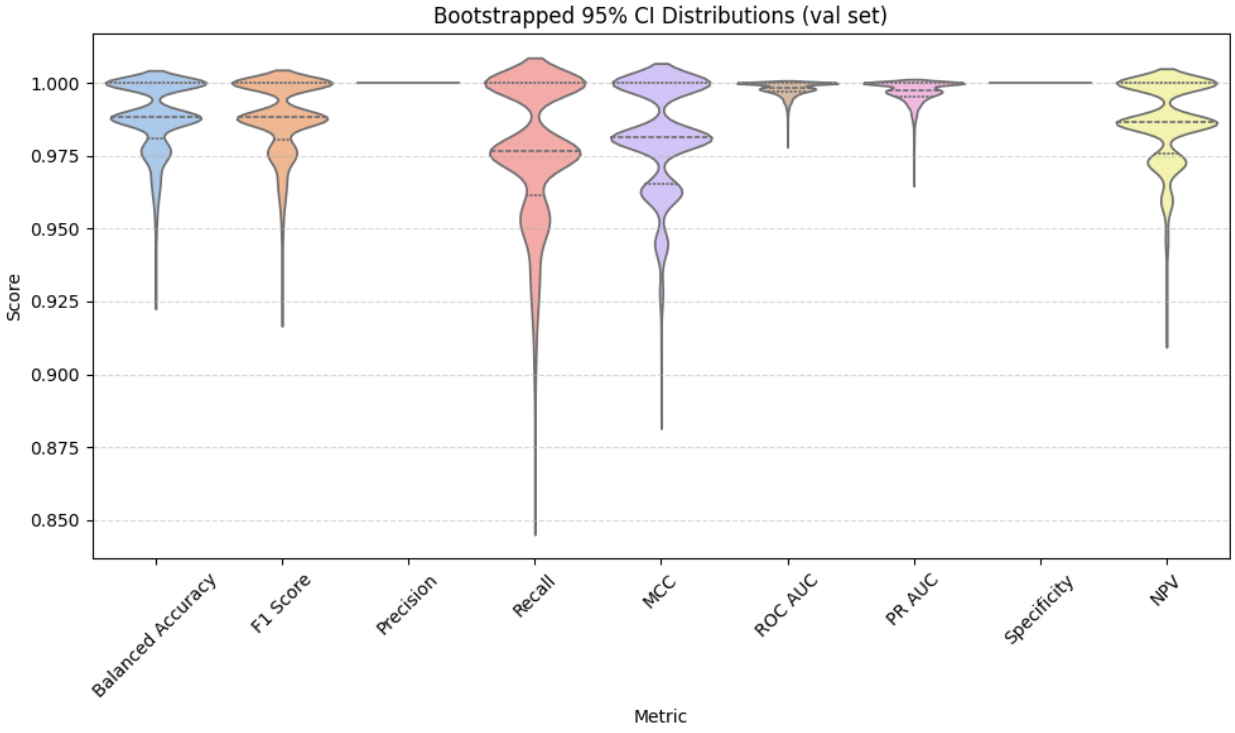

Figure 2.10: Violin plot of the confidence intervals

As a final step in our classical approach, we utilize the entire dataset to train and fine-tune the Logistic Regression model, from which we extract the 16 most influential features based on the descending absolute values of their coefficients. These features, in order of impact, are $texture_worst$, $area_se$, $texture_mean$, $concavepoints_mean$, $concavity_worst$, $symmetry_worst$, $concavity_mean$, $compactness_se$, $area_worst$, $smoothness_worst$, $radius_se$, $symmetry_se$, $concavepoints_worst$, $radius_worst$, $perimeter_worst$, and $fractal_dimension_se$. Upon examining the coefficients, we observe that $compactness_se$, $symmetry_se$, and $fractal_dimension_se$ are negative, indicating that higher values in these features are associated with the majority class—the benign tumors. Conversely, the remaining 13 features possess positive coefficients, suggesting that higher measurements in these areas correlate with an increased likelihood of the minority class, which corresponds to malignant tumors.

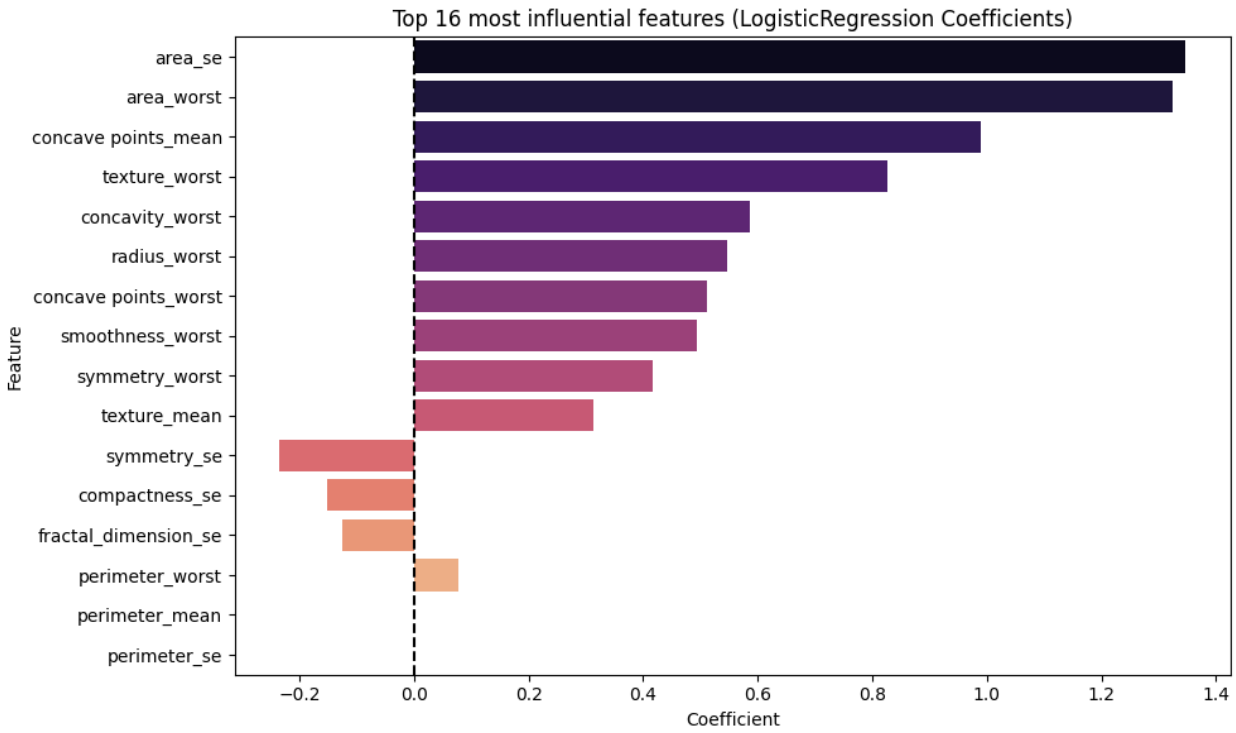

Figure 2.11: Top 16 influential features according to Logistic Regression

By saving our final fine-tuned model for future inference on unseen data, we conclude our investigation into classical machine learning methods. This marks the transition from establishing a robust performance benchmark with traditional algorithms to exploring quantum approaches for the same classification task.

3. RESULTS FROM QUANTUM MACHINE LEARNING METHODS

Following the determination that Logistic Regression represents the most efficacious classical machine learning approach for the current tumor classification problem, the research transitions toward the implementation of Quantum Machine Learning (QML) paradigms. Specifically, this study investigates the Variational Quantum Classifier (VQC) to identify optimal sets of trainable parameters that provide a robust and satisfactory solution to the diagnostic task. This exploration involves a systematic evaluation of the VQC's classification capabilities across architectures consisting of 1, 2, 3, 4, and 5 qubits, respectively. To facilitate this, Principal Component Analysis (PCA) is employed to perform the necessary dimensionality reduction, ensuring the input features are structurally compatible with the constraints of the quantum circuits. The computational implementation of this framework is supported by the PennyLane **[CITE]** Python library for quantum circuit simulation and differentiable programming, while NumPy facilitates essential numerical manipulations. Furthermore, pandas and scikit-learn are utilized for data orchestration and dimensionality reduction, while Seaborn **[CITE]** and Matplotlib **[CITE]** are employed for high-fidelity data visualization and statistical plotting.

A critical component of this transition involves a refined approach to data management and integrity, starting with a tripartite division of the dataset. Unlike the previous classical approach, we utilize a 60-20-20 split, wherein 60% of the original data is allocated for model training, 20% is reserved for validation to guide the optimization of variational parameters, and the remaining 20% serves as a dedicated test set for final, unbiased model evaluation. To maintain the statistical integrity of the dataset, class imbalance is meticulously accounted for through stratified sampling across all three subsets, ensuring that the proportional representation of benign and malignant cases remains consistent. Furthermore, to mitigate the significant risks associated with data leakage, we ensure that no information from the validation or test sets influences the training phase. In practical terms, this necessitates that any preprocessing transformers or feature scalers, including the PCA model, are fitted exclusively on the training subset and subsequently applied to the validation and test sets only after the initial stratified split has been performed.

Another significant departure from the classical methodology is the modification of the target label encoding. Rather than the standard binary assignment of 0 for benign and 1 for malignant, we adopt a bipolar encoding scheme where the benign class is represented by -1 and the malignant class by 1. This preprocessing step is necessitated by the physical nature of the quantum classifier, which essentially utilizes the sign of the output measurement to assign a predicted label to each input vector. This encoding aligns with the measurement of a quantum bit in the computational basis, where expectation values naturally range within the interval [-1, 1]. Consequently, the Mean Square Error (MSE) is selected as the objective loss function for its simplicity and its compatibility with these continuous expectation values. Notably, during the optimization process, we utilize the raw measurement values rather than their discrete signs to preserve the granular gradient information required for effective training. The rationale behind this specific choice and its impact on the convergence of the model will be expanded upon in subsequent sections of

this study.

As a concluding note on the experimental design, the evaluation of the VQC's performance is streamlined to focus on four primary metrics: Accuracy, Recall, Precision, and the F1-score. By restricting the evaluation to these fundamental indicators, we aim to provide a clear and focused assessment of the VQC's potential within a medical diagnostic context without the confounding variables introduced by a larger collection of comparative methods. This simplified evaluation framework is intended to illuminate the relative strengths of the quantum approach in comparison to established classical baselines. Having established these methodological differences and the underlying logic of our quantum framework, we now proceed to an in-depth explanation of the specific training, validation, and testing protocols designed for the Variational Quantum Classifier.

The methodological framework adopted for this research remains consistent across all experimental trials involving Variational Quantum Classifiers (VQCs) of various qubit capacities. This framework is partitioned into two primary segments: the data preparation stage and the training and testing phase. The data preparation process is fundamentally guided by the specific number of qubits utilized in each iteration, primarily involving the implementation of dimensionality reduction techniques and the systematic scaling of the training, validation, and test datasets. Additionally, a preliminary exploratory data visualization of the processed data is conducted to evaluate the spatial separability of the two classes, thereby gauging the inherent complexity of the classification task.

Following the completion of the data preprocessing phase for all three subsets, the training and evaluation stage focuses on optimizing the Variational Quantum Classifier using the training and validation sets, followed by a final assessment of its performance on the test set. The fundamental circuit architecture remains invariant regardless of the qubit count and is categorized into three specific components. The initial segment involves data encoding, wherein classical information is mapped into the quantum state. To achieve this, we employ the Amplitude Efficient State Preparation method proposed by Petruccione and Schuld **[CITE]**, which facilitates the encoding of 2^n features using n qubits by first normalizing the input vectors.

In cases where the number of features is less than 2^n, the input vector is padded with a constant value of 0.1 to satisfy the dimensionality requirements of the encoding scheme. This padding ensures that the input dimensions remain consistent with the Hilbert space of the chosen qubit configuration. This segment is crucial as it defines the initial state of the quantum system before any transformations occur in the variational layers.

The second component comprises the variational segment, which incorporates the system's trainable parameters. Drawing inspiration from the work of Schuld et al.**[CITE]** and contemporary methodologies in classical neural networks, this architectural section consists of layers that are repeated for a predetermined number of iterations. Each layer is composed of a sequence of single-qubit rotation gates followed by Controlled-NOT ($CNOT$) gates to facilitate multi-qubit entanglement. Adopting the notation utilized by the PennyLane library, the single-qubit rotation gates are implemented as arbitrary rotations, defined mathematically as $Rot(\phi, \theta, \omega) = R_Z(\omega)R_Y(\theta)R_Z(\phi)$. This sequence represents a

rotation around the z-axis by an angle ω, followed by a rotation around the y-axis by θ, and a final rotation around the z-axis by ϕ.

This specific configuration of gates allows the model to navigate the Hilbert space effectively, with these rotation angles serving as the primary trainable quantum parameters. Consequently, the number of trainable parameters per layer scales linearly with the number of qubits: three for a single qubit, six for two qubits, nine for three qubits, twelve for four qubits, and fifteen for a five-qubit configuration. Crucially, these parameter values are unique to each layer; for instance, a single-qubit classifier utilizing two variational layers would comprise six distinct trainable parameters. As such, the total parameter space expands rapidly in relation to both circuit width and depth

The third and final component of the Variational Quantum Classifier architecture is the measurement stage, during which the quantum state is collapsed to extract classical information. To maintain a streamlined and interpretable framework, the measurement is restricted to the first qubit of the circuit. The integration of Controlled-NOT gates in the preceding layers is critical at this stage, as the entanglement generated by these multi-qubit operations ensures that the state of the primary qubit is inextricably linked to the states of all other qubits in the register. Consequently, the expectation value derived from the first qubit, typically obtained through measurement in the Z basis, reflects a global state influenced by the entire circuit. This approach exploits the intrinsic properties of quantum entanglement, where a measurement on a single subsystem provides representative insights into the collective quantum state of the system. An illustrative representation of a VQC configuration comprising two qubits and six layers is provided in Figure **3.1** to clarify this architectural design.

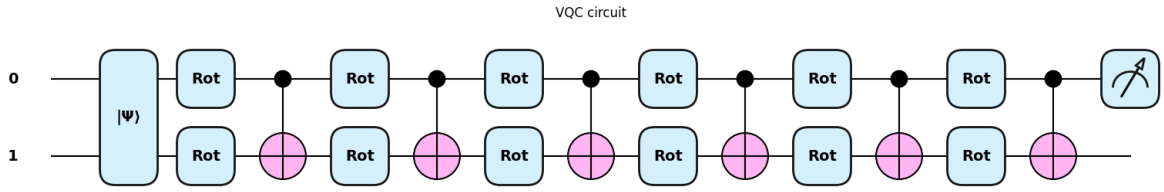

Figure 3.1: A VQC configuration comprising of two qubits and six layers

To derive the final predictive output of the circuit, we extract the expectation value from the measurement of the first qubit. To this result, we introduce a classical scalar known as a bias term, which is initially set to zero. Consequently, the decision function of the classifier is defined as the summation of the quantum circuit's measurement and this classical value. We distinguish between two primary configurations: the unbiased Variational Quantum Classifier, where the bias remains fixed at zero to evaluate a strictly quantum-mechanical approach, and the biased VQC, where the bias is integrated into the optimization pipeline as an additional trainable parameter.

In the biased configuration, the parameter is optimized through classical gradient-based methods alongside the variational rotation angles. The objective of this distinction is to

systematically observe the performance variance between a purely quantum model and a hybrid system, where the quantum output is either bolstered or refined by a classical offset. It is hypothesized that for architectures characterized by low qubit counts and minimal circuit depth,where the problem complexity is relatively low,the contribution of the bias term may be negligible. However, its utility is expected to become more pronounced as the system scales in complexity, specifically when increasing the number of qubits and variational layers.

To identify the optimal configuration of trainable parameters, whether strictly quantum or a hybrid of quantum and classical, we evaluate various layer depths while training the Variational Classifier over a set number of epochs, adjusted according to the qubit count. The optimization process begins with the stochastic initialization of the circuit's variational parameters to small random values, while the classical bias is initialized at zero. Subsequently, we utilize the objective loss function and a classical optimizer to iteratively update the quantum rotation angles; in the specific case of the biased classifier, the classical bias parameter is optimized concurrently.

Throughout the optimization process, the classifier is evaluated using the previously unseen validation set to assess its generalization capability. Using a batch size consistent with the training phase, validation data is processed to generate predicted labels, from which the accuracy, precision, recall, and F1-score are calculated. To facilitate a comprehensive analysis of the model's evolution, performance metrics are simultaneously recorded for the training set

This comparative approach allows for a detailed assessment of the model's learning trajectory and helps identify potential occurrences of overfitting or underfitting. Furthermore, tracking the progression of the cost function across both training and validation inputs is imperative for understanding convergence behavior and the stability of the optimization.

The specific parameters associated with the highest validation performance are preserved, along with the epoch count required to achieve this optimal state. Ultimately, these optimized parameters are utilized to perform inference on the held-out test set, providing an unbiased evaluation of the Variational Quantum Classifier's performance and its efficacy in solving the tumor classification problem.

To ensure a robust assessment of the generalization capabilities of the Variational Quantum Classifier, the aforementioned experimental procedure is repeated ten times for each specific layer configuration. This iterative approach facilitates a rigorous statistical analysis of the correlation between circuit depth, and the concomitant increase in the total number of trainable parameters, and the resulting model performance. By conducting multiple independent trials, the methodology accounts for the stochasticity inherent in parameter initialization and the non-convex nature of the optimization landscape, thereby providing a more statistically significant estimation of the model's predictive stability and performance across different initial conditions.

Furthermore, the mean number of epochs required for achieving these results within each layer configuration is recorded to evaluate the efficiency and stability of the training process. The computational latency, measured in seconds, is also documented for each

training cycle to provide a detailed perspective on the temporal requirements associated with scaling the quantum architecture. Ultimately, the specific parameters from the most efficacious model across all experimental iterations and layer depths are preserved for final evaluation, irrespective of the circuit depth at which they were attained. This comprehensive data collection enables a thorough characterization of the trade-offs between quantum circuit complexity, computational cost, and classification accuracy.

Several technical considerations regarding the optimization protocol warrant further clarification. The Mean Squared Error (MSE) is employed as the objective function for the training process, subject to a specific methodological refinement. While the final classification labels are determined by the algebraic sign of the model's output, specifically the sum of the VQC measurement and the bias term, the loss function is calculated using the raw continuous values. This approach effectively treats the binary classification problem as a regression task where a zero-threshold is applied post-measurement. This choice is critical as it ensures a smooth and differentiable cost landscape, avoiding the discontinuous or step-like functions that typically arise when calculating loss based on discrete labels. Consequently, this allows for more effective monitoring of the training trajectory and provides a more granular signal for gradient updates.

The optimization of the parameters is executed using the Nesterov Momentum optimizer, also known as Nesterov Accelerated Gradient **[CITE]**. This method is an advanced iteration of standard momentum-based gradient descent, frequently utilized in deep learning for its ability to anticipate the loss landscape and achieve superior convergence rates. The input data is processed in stochastic mini-batches of size five, with the order of samples randomized in each epoch. Given the relative scarcity of the dataset, a batch size of this magnitude is considered sufficient to provide stable gradient estimates while maintaining the stochasticity necessary for effective exploration of the parameter space.

It is important to clarify that the comprehensive training procedure, which spans various layer depths across ten independent trials, is executed in two distinct phases. The initial phase involves the implementation of unbiased classifiers, followed by a secondary phase dedicated to biased models. Consequently, for each specific qubit configuration, we obtain two distinct model types: an unbiased model defined by purely quantum variational parameters and a biased model characterized by a combination of quantum and classical parameters.

This dual-track approach is designed to elucidate which specific methodology provides the optimal solution for the tumor classification task at a given qubit count. Furthermore, it enables a comparative analysis of the respective advantages of each paradigm as the problem scales, providing deeper insight into the efficacy of hybrid quantum-classical optimization versus purely quantum strategies. This methodology ensures that the performance of the Variational Quantum Classifier is evaluated not only on its quantum depth but also on its ability to integrate classical components to improve diagnostic accuracy.

3.1 1-qubit Results

Following the establishment of the overarching methodology, the initial phase of implementation utilizes a single-qubit Variational Quantum Classifier as a baseline model. Given the constraints of the Amplitude Efficient State Preparation method, a single-qubit register is limited to encoding a maximum of $2^1 = 2$ features. This necessitates a significant reduction in dimensionality from the original dataset, which initially comprised 30 distinct features. To address this while preserving the fundamental statistical characteristics of the data, Principal Component Analysis is performed across the training, validation, and testing subsets. To maintain the integrity of the experimental results and prevent data leakage, the transformation is fitted solely on the training partition and subsequently applied to the validation and test sets.

This reduction yields two primary principal components that collectively explain 62.97% of the total variance within the dataset. Although this captured variance is slightly lower than the 63.24% that would be achievable if the transformation were fitted on the entire dataset simultaneously, it remains a robust representation of the underlying data distribution. By condensing the high-dimensional feature space into two components, the model retains over half of the original information, providing a manageable and statistically significant input for the 1-qubit variational circuit. This configuration serves as an essential starting point for evaluating the classification performance of quantum architectures at their most fundamental scale, allowing for a clear assessment of how effectively minimal quantum resources can handle the compressed feature space.

To characterize the distribution and interrelationships of the identified principal components, we utilize the pairplot functionality provided by the Seaborn library **[CITE]**. This visualization tool facilitates a comprehensive analysis of the pairwise interactions between variables within a dataset while simultaneously providing an estimation of the univariate distribution for each individual feature. The resulting output is structured as a symmetrical matrix-style grid, where both the rows and columns correspond to the principal components derived from the dimensionality reduction process. Within this grid, each off-diagonal cell represents a bivariate scatter plot comparing two specific variables, while the diagonal cells display the univariate distribution of each component, typically through histograms or kernel density estimates. This approach provides a condensed yet detailed summary of both the univariate and bivariate structures of the transformed tumor dataset, offering critical insights into the spatial separation between the benign and malignant classes before they are processed by the quantum classifier.

The plots situated along the principal diagonal of the grid represent marginal univariate distributions. Each cell on the diagonal corresponds to an isolated variable, facilitating an analysis of its independent statistical profile rather than a bivariate interaction. While typically rendered as histograms, these visualizations can be adapted to employ kernel density estimates to provide a smoothed representation of the underlying probability density function. These diagonal plots are essential for inspecting fundamental statistical properties, including the range, dispersion, skewness, and modality of each principal component, as well as identifying potential outliers within the feature space. Effectively, the diagonal

matrix elements characterize the distribution of each variable in isolation.

The remaining cells in the grid, located off the principal diagonal, illustrate the bivariate relationships between distinct pairs of variables. These cells primarily utilize scatter plots where each data point represents an individual observation from the transformed dataset. The axes for these plots are determined by their respective positions in the grid matrix: the variable associated with the column is mapped to the x-axis, while the variable associated with the row is mapped to the y-axis. These bivariate visualizations enable a qualitative assessment of how pairs of variables interact, revealing whether they exhibit positive or negative correlation, identifying the presence of linear or nonlinear dependencies, and highlighting structural nuances such as spatial clustering or heteroscedasticity across the range of values.

The pairplot matrix exhibits inherent symmetry relative to its principal diagonal. This structural organization implies that the bivariate relationship between any two variables is represented twice within the grid; specifically, the interaction between components i and j is visualized once with variable i on the y-axis and again with the axes transposed. Although this configuration introduces a deliberate degree of visual redundancy, it significantly enhances data interpretability by providing mirrored perspectives of feature interactions. This repetition allows for a more efficient cross-comparative analysis of patterns and correlations across the various principal component pairings.

When the target categorical variable is assigned to the hue parameter, the pairplot introduces an additional layer of dimensionality by color-coding individual data points according to their respective class or group membership. In this context, the off-diagonal scatter plots facilitate a comparative analysis of class-specific spatial distributions within the feature space, while the diagonal elements provide stratified marginal density estimates for each class.

This multidimensional representation is particularly advantageous for exploratory data analysis in classification tasks, as it allows for a rigorous assessment of class overlap, linear or non-linear separability, and the degree to which inter-variable relationships remain consistent across different target categories. By visualizing how classes occupy distinct regions or clusters within the principal component space, we can anticipate the potential efficacy of the Variational Quantum Classifier and identify specific regions where class boundaries may be ambiguous.

In summary, the pairplot serves as a comprehensive visual framework for exploratory data analysis. The marginal distributions situated along the diagonal facilitate an understanding of each individual feature in isolation, whereas the off-diagonal scatter plots elucidate the complex interactions and correlations between variable pairs. Collectively, these visualizations offer a holistic perspective on the underlying data structure, enabling the identification of significant patterns, linear dependencies, and potential anomalies prior to the commencement of formal modeling. Figure **3.2** illustrates the resulting pairplot for the training partition following the application of Principal Component Analysis and the selection of two principal components.

The visualization highlights the spatial distribution of the classes within the reduced feature

space, providing a qualitative baseline for the expected performance of the single-qubit Variational Quantum Classifier. By observing the degree of overlap between the malignant and benign clusters in these two dimensions, one can gauge the relative difficulty the quantum model will face in establishing an optimal decision boundary. This visual confirmation is a prerequisite for the training phase, as it ensures that the dimensionality reduction process has preserved sufficient variance and class-specific structure to support the subsequent classification task.

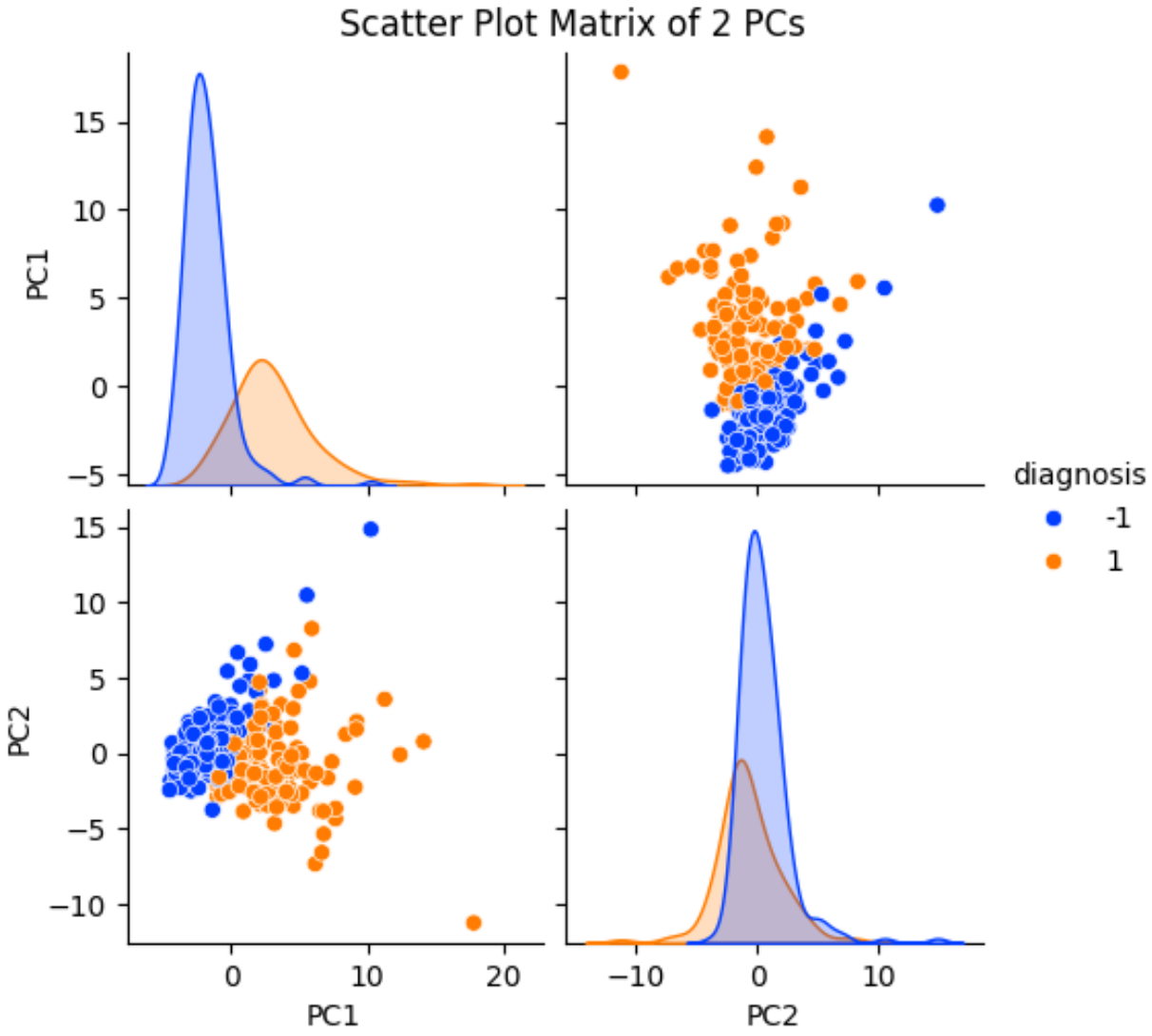

Figure 3.2: Pairplot of the 2 PCs for the training subset

A close examination of Figure **3.2** yields several significant insights regarding the class-specific distributions within the reduced feature space. By analyzing the marginal distributions along the main diagonal, it is evident that the two classes exhibit minimal spatial overlap with respect to the first principal component, suggesting that this dimension captures features with high discriminatory power. Conversely, regarding the second principal

component, the probability density of the positive class appears to be largely enveloped by the distribution of the negative class. This configuration implies that while the first component provides a clear basis for separation, the second component encompasses a region of high statistical ambiguity where the feature values of the positive class are effectively a subset of those plausible for the negative class.

In a general classification setting, the relative overlap between class-conditional distributions is a primary factor in determining how effectively a given feature can separate distinct classes. When the distribution of one class largely envelops that of another, it indicates that the majority of feature values characteristic of the enveloped class are also highly plausible under the encompassing class. In such instances, the classes share a substantial region of high probability density, which introduces a profound intrinsic ambiguity into the data. From a probabilistic perspective, there are few, if any, regions of the feature space uniquely associated with the enveloped class; consequently, even an optimal Bayes classifier is compelled to make errors in these overlapping regions, resulting in a high irreducible error that no modeling approach can entirely overcome.

This enveloping behavior has direct consequences for classification performance. Because the encompassing class assigns non-negligible probability to nearly all values characteristic of the other, misclassification errors tend to be asymmetric. Samples from the enveloped class are particularly susceptible to misidentification, as they reside in regions where the likelihood of the enveloping class remains significant. In practical terms, this often manifests as low recall for the enveloped class and a heightened sensitivity of performance metrics to decision thresholds and class priors. Importantly, this limitation does not reflect a failure of the learning algorithm itself, but rather indicates that the feature in question is weakly discriminative. Improving performance in such scenarios typically necessitates the incorporation of additional features or a transition toward probabilistic predictions rather than hard class assignments.

In contrast, when class-conditional distributions do not overlap extensively, the feature space contains regions that are specific to each class. In this case, each distribution occupies a portion of the feature domain where the other assigns little or no probability mass. This partial separation implies the existence of feature values that provide strong evidence in favor of one class over the other. Consequently, a decision boundary can be positioned such that most samples are classified correctly, ensuring that errors are largely confined to the relatively narrow regions where the distributions intersect.

From a theoretical standpoint, the non-enveloping case corresponds to a lower Bayes error rate, resulting in a more favorable classification problem. The existence of class-specific regions leads to more balanced precision and recall, as well as generally higher discrimination metrics, such as the area under the ROC curve (AUC). In these settings, simple models like linear classifiers are often sufficient to achieve near-optimal performance, where increasing model complexity yields only marginal improvements. Overall, the absence of enveloping behavior indicates that the feature is genuinely informative and that classification performance is primarily limited by noise rather than a fundamental overlap between the classes.

Shifting focus to the off-diagonal cells of the pairplot, we examine the bivariate relationship between the first and second principal components. It is evident that, despite localized class overlap or ”spillage,” the two categories appear largely linearly separable within this two-dimensional projection. This observation is particularly encouraging, as it suggests that even the most fundamental Variational Quantum Classifiers may achieve satisfactory performance. This structural behavior, including the marginal distribution overlaps, is consistent with expectations: the first component accounts for 42.34% of the total variance, while the second describes the remaining 20.63%. While the variance explained by each individual component typically decreases as the number of components increases, the current level of separation provides confidence that the quantum approach will yield robust results.

As previously stated, we investigate the impact of the circuit depth on classifier performance and evaluate whether the introduction of a classical bias parameter enhances or inhibits the model's classification capabilities. Given that the dimension of the associated Hilbert space is $4^n - 1$, we restrict our experimentation to configurations with one and two layers. Since the architecture utilizes a single qubit, each layer consists exclusively of a single Rot gate, comprising three trainable parameters, without the possibility of quantum entanglement. This structural constraint further justifies limiting the depth to two layers, as empirical evidence indicates that successive rotations in the absence of entangling $CNOT$ gates offer no additional expressive advantage. Each model configuration is trained for 150 epochs, a duration considered sufficient for the proper optimization of the variational and classical parameters.

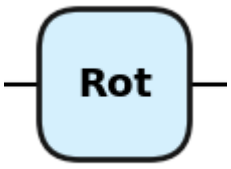

Figure 3.3: A single layer of the 1-qubit VQC

An examination of the evaluation results reveals several noteworthy observations regarding the classifier's performance. Foremost, the 1-qubit configuration yields highly satisfactory results even with a single variational layer. In the unbiased case, the model achieves average performance metrics across ten independent runs of 91.93% accuracy, 91.95% precision, 90.63% recall, and a 91.19% F1-score, with all indicators consistently exceeding the 90% threshold. The introduction of a classical bias parameter produces a marginal performance shift, resulting in an average accuracy of 92.11%, 91.83% precision, 91.12% recall, and a 91.45% F1-score. Transitioning to a two-layer architecture yields comparable outcomes; the unbiased classifier achieves an accuracy of 91.93%, a precision of 91.85%, a recall of 90.73%, and an F1-score of 91.21%. Similarly, the biased two-layer model demonstrates stable performance with 92.02% accuracy, 91.68% precision, 91.10% recall, and a 91.37% F1-score. These findings suggest that for a single-qubit

system, both configurations are remarkably effective, and increasing circuit depth beyond a single layer provides negligible performance gains.

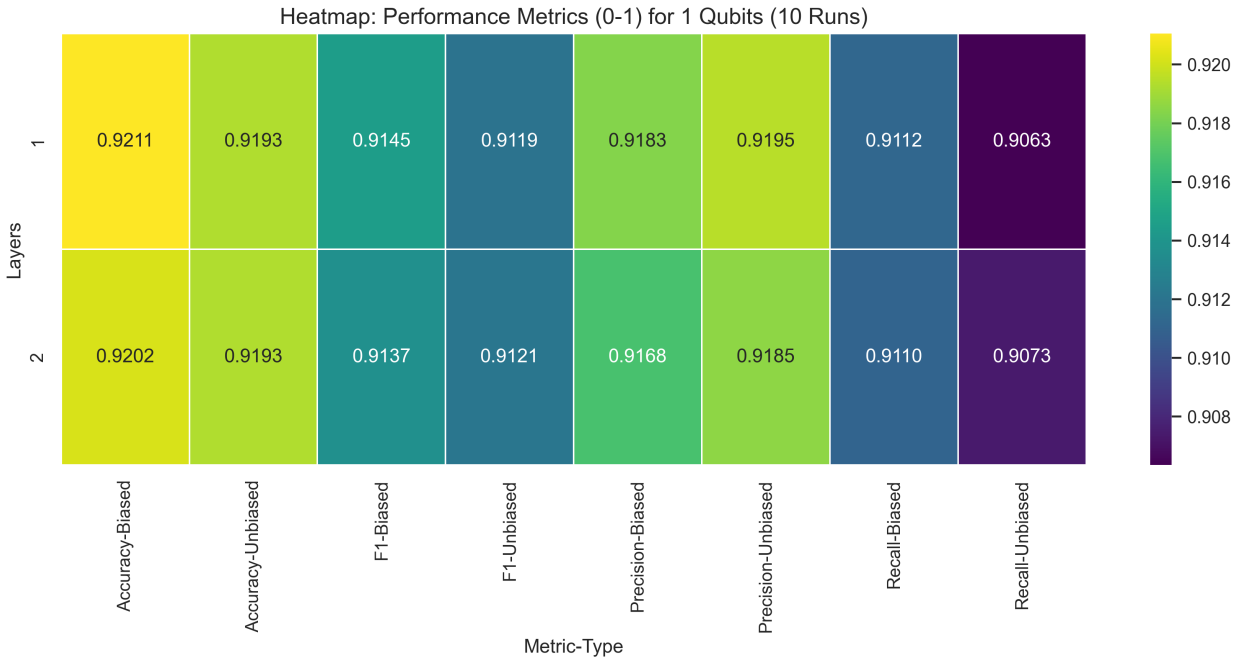

Figure 3.4: Performance Results for 1-qubit classifiers after 10 runs

Examining the efficiency metrics for the two approaches, we observe that for the single-layer configuration, the unbiased classifier converges to its optimal parameters after approximately 81 epochs, with the training procedure lasting an average of 4.9 seconds. In comparison, the biased classifier requires nearly 78 epochs to reach optimization, with an average training time of 4.78 seconds. Upon adding an extra layer, the unbiased classifier finds its optimal values significantly faster, at approximately 44 epochs, though the training procedure extends to an average of 6.12 seconds. For the biased two-layer variety, the model identifies its best parameter values after only 37 epochs, completing its training in an average of 4.91 seconds.

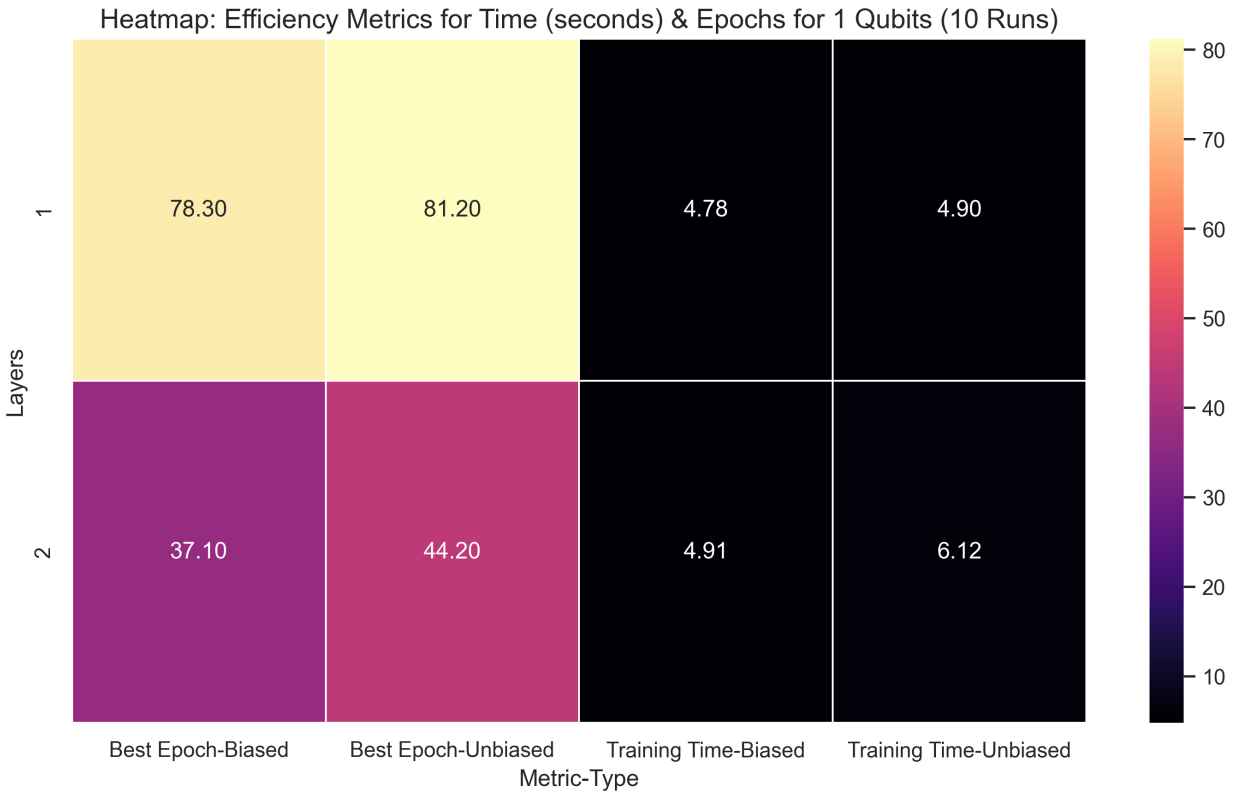

Figure 3.5: Efficiency Results for 1-qubit classifiers after 10 runs

From a comparative perspective, the biased classifier appears to demonstrate superior performance, achieving higher scores across the four key metrics regardless of the number of layers. Furthermore, despite requiring the optimization of an additional parameter, it exhibits greater efficiency by identifying optimal parameter values more rapidly and requiring less training time, particularly in the two-layer configuration. It is important to note, however, that these differences are minuscule and fall within the margin of statistical error. Even the approximate one-second variation in training time may be attributable to external system processes occupying computational resources during the training phase rather than the inherent properties of the models themselves.

It is particularly noteworthy that, regardless of whether a biased or unbiased classifier is employed, our initial intuition regarding the impact of circuit depth, and consequently, the number of parameters, is validated. Aside from marginal variations attributable to statistical error, there is no significant performance improvement when transitioning from a single-layer to a two-layer classifier. In certain instances, such as the biased classifier, a slight decrease in performance is even observable. Regarding efficiency, the results align with expectations: as the number of trainable parameters increases, the computational time per training instance also rises. Simultaneously, increasing the parameter count while maintaining a constant Hilbert space dimension allows the model to converge to optimal results in fewer epochs.

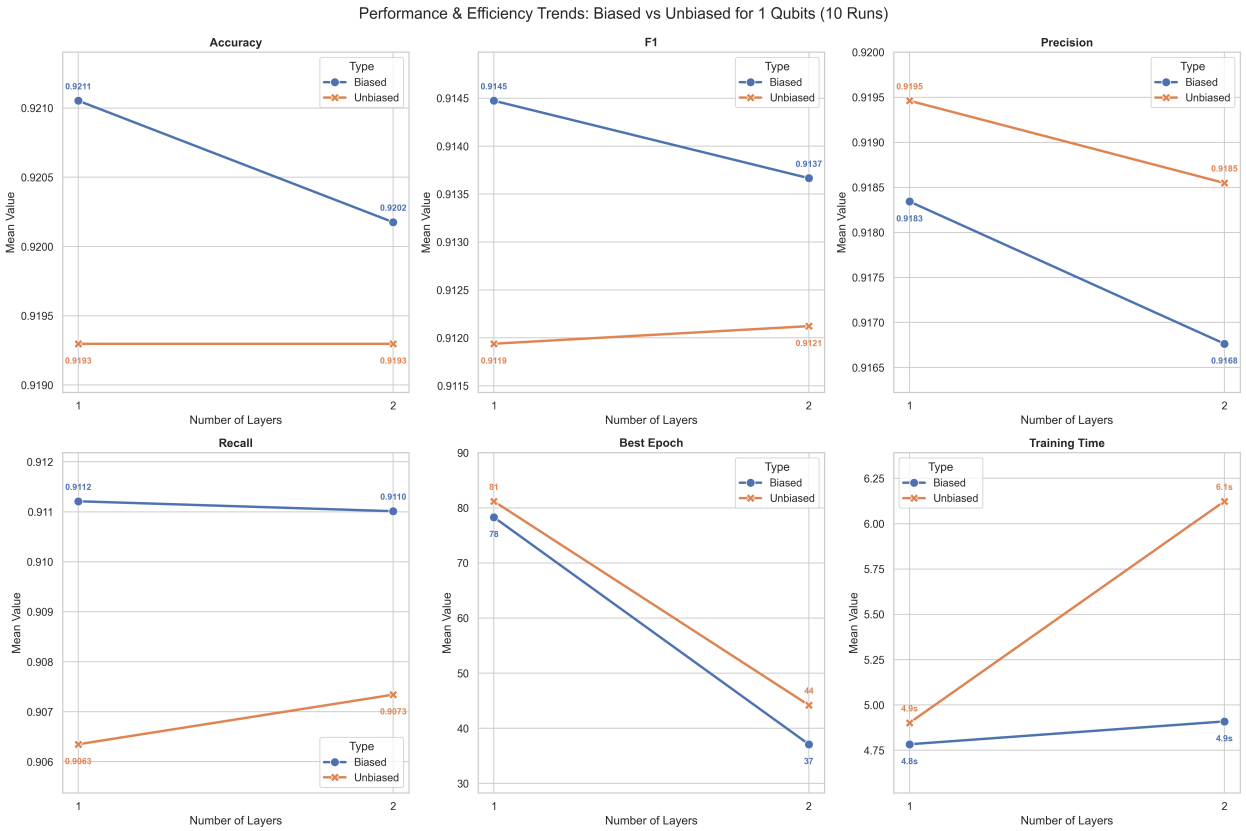

Figure 3.6: Performance and Efficiency Trends of Biased and Unbiased Classifier for 1-Qubit

In the final phase of our evaluation, we examine the highest-performing models from both the biased and unbiased configurations across all experimental runs. We present the specific variational quantum circuit (VQC) architectures alongside the evolution of the loss function and key performance metrics over the 150-epoch duration for both the training and validation sets.

This analysis facilitates an investigation of the loss landscape to identify plateaus and peaks, offering valuable insight into the progression and stability of the optimization procedure. Similarly, monitoring the evolution of these metrics throughout the training process allows us to detect potential signs of overfitting or underfitting while providing a comprehensive overview of the model's general learning trajectory and convergence behavior.

For the best-performing biased classifier, the VQC architecture consists of a single layer, which aligns with our previous average findings. Despite its structural simplicity, this circuit achieved an accuracy of 92.11%, a precision of 91.70%, a recall of 91.27%, and an F1-score of 91.47% at the 84th epoch.

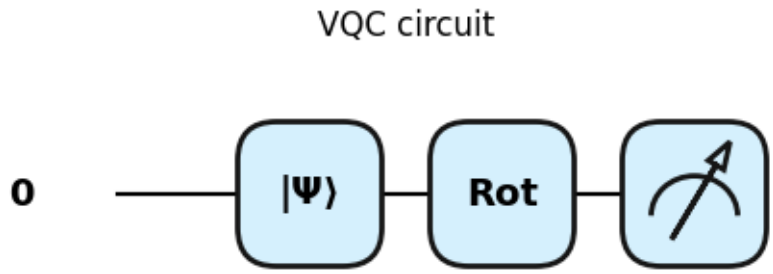

Figure 3.7: VQC of the best 1-qubit biased classifier

Regarding the evolution of the loss function, there is a prominent decline during the initial 20 epochs, followed by an anomalous period between the 80th and 100th epochs where the loss increases and then decreases sharply. Remarkably, the model identifies its optimal parameters during this volatile interval. While this might appear surprising, it is important to note that neither the minimization of the training cost nor the validation cost exclusively guarantees high performance; indeed, such fluctuations can be a concerning sign of potential overfitting.

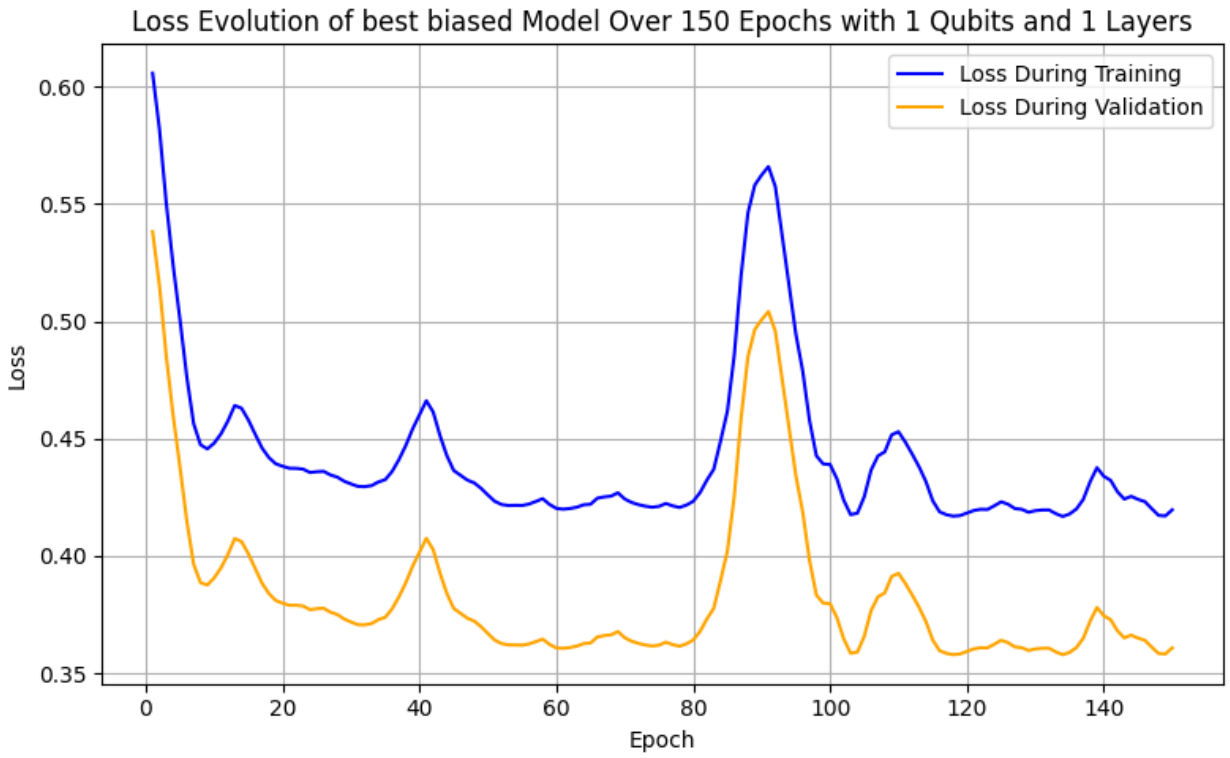

Figure 3.8: Evolution of the loss function of the best 1-qubit biased classifier

This observation is further reinforced by a closer examination of the key metric progression for the training and validation sets throughout the 150 epochs. Notably, the model demonstrates strong generalization, as the trajectories of the training and validation curves are remarkably similar; with the exception of brief intervals where validation performance marginally exceeds training metrics, the divergence between the two remains minimal and consistently within a statistical error margin of 0.05. Furthermore, in relation to the loss function behavior, the peak scores are indeed observed at epoch 84, where the loss function paradoxically exhibits an upward trend. Conversely, around epoch 100, when the loss function reaches its absolute minimum, the performance metrics begin to decline, indicating that the point of lowest loss does not necessarily coincide with the model's most accurate predictive state.

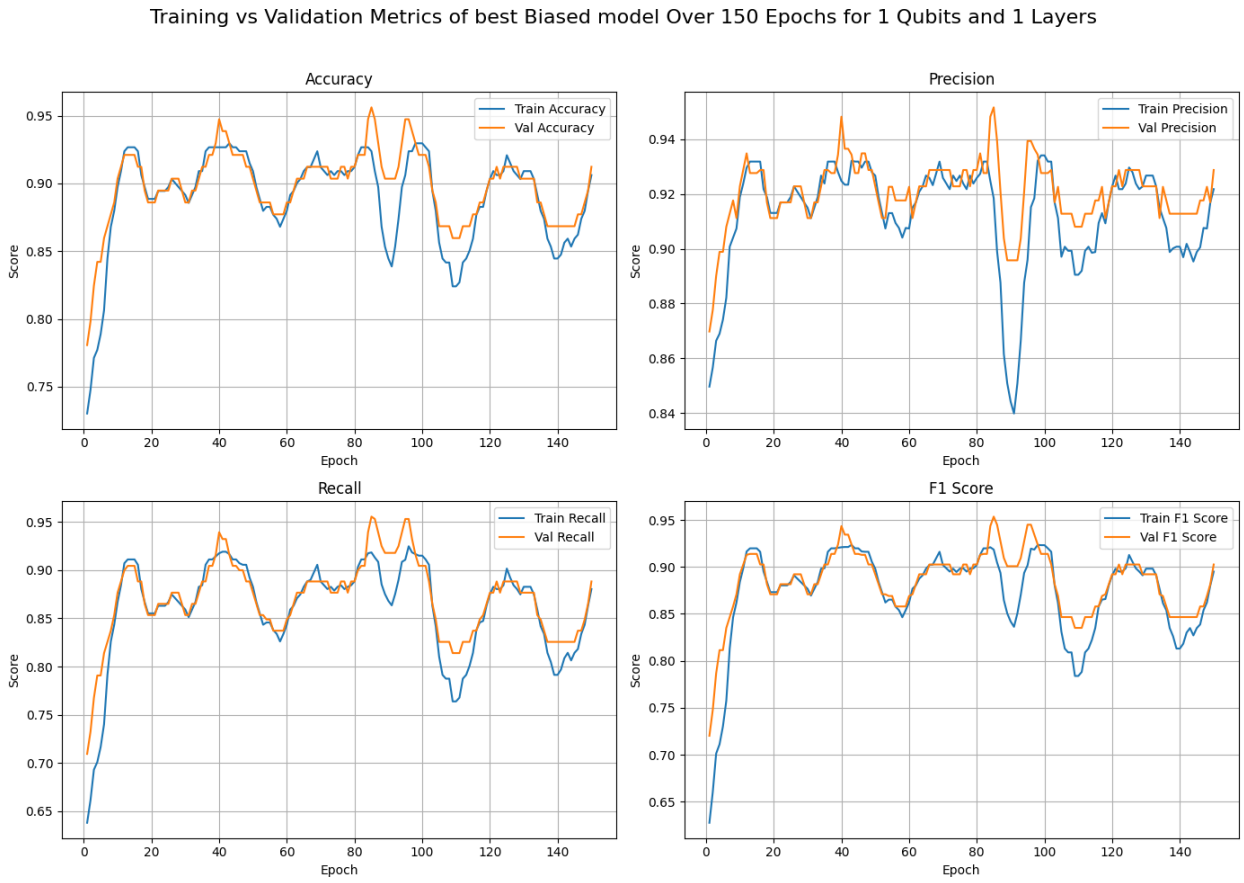

Figure 3.9: Validation and Training curves for the 4 key metrics of the best 1-qubit biased classifier

The results for the best-performing unbiased classifier follow a similar trend. Consistent with our average findings, its VQC architecture consists of a single layer. This configuration achieved an accuracy of 92.11%, a precision of 91.70%, a recall of 91.27%, and an F1-score of 91.47% at the 86th epoch.

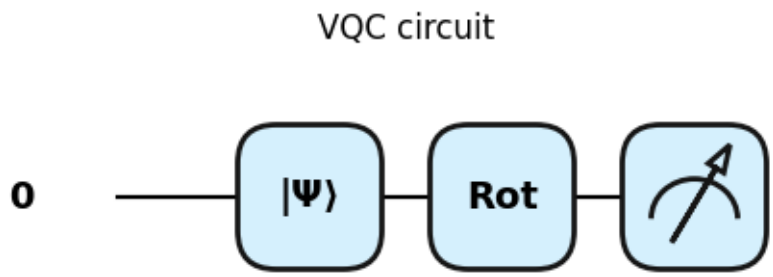

Figure 3.10: VQC of the best 1-qubit unbiased classifier

Regarding the evolution of the loss function, the trajectory for the unbiased classifier differs notably from the frequent fluctuations observed in the biased variant. While the cost function exhibits a rapid initial decline during the first 20 epochs, it subsequently enters a plateau lasting approximately 20 epochs following a minor increase and decrease. The loss then rises for nearly 10 epochs before descending again around epoch 100, where it reaches its minimum and ultimately stabilizes. Notably, the optimal performance metrics are once again identified at an epoch where the cost function is not at its absolute minimum, but is instead characterized by an increasing trend.

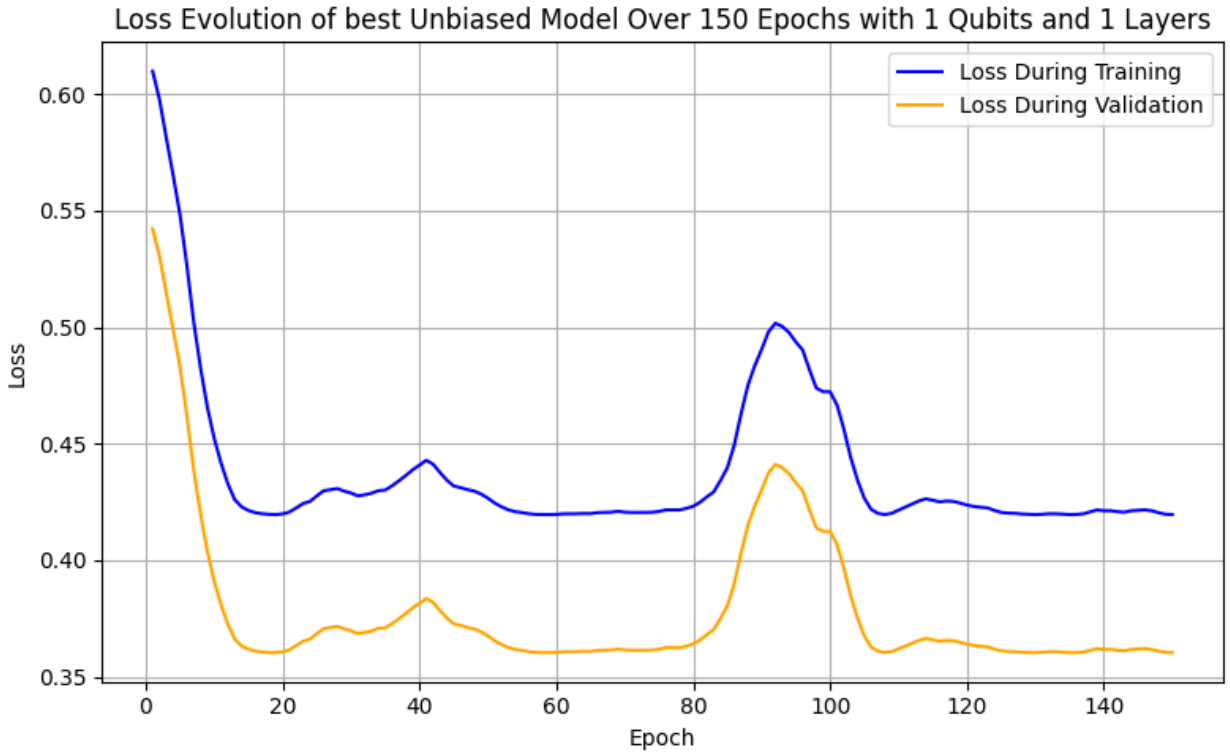

Figure 3.11: Evolution of the loss function of the best 1-qubit unbiased classifier

The progression of key metrics for the unbiased classifier is quite similar to that of the biased case. Both curves exhibit comparable trends, and with the exception of the interval between the 80th and 100th epochs, their divergence remains largely insignificant, suggesting a lack of substantial overfitting or underfitting. The sudden increase in divergence during this specific interval coincides with the point where the cost function begins to rise. Precision appears to be the metric most affected by this development, while the others remain well within the 0.05 error margin. Interestingly, after the 100th epoch, as the cost function reaches its plateau, the performance metrics follow suit. However, this stability is not sufficient to surpass the peak scores recorded at the 86th epoch, further reinforcing the notion that while minimizing the loss function is essential to the training process, it is not an absolute panacea for achieving optimal classification results.

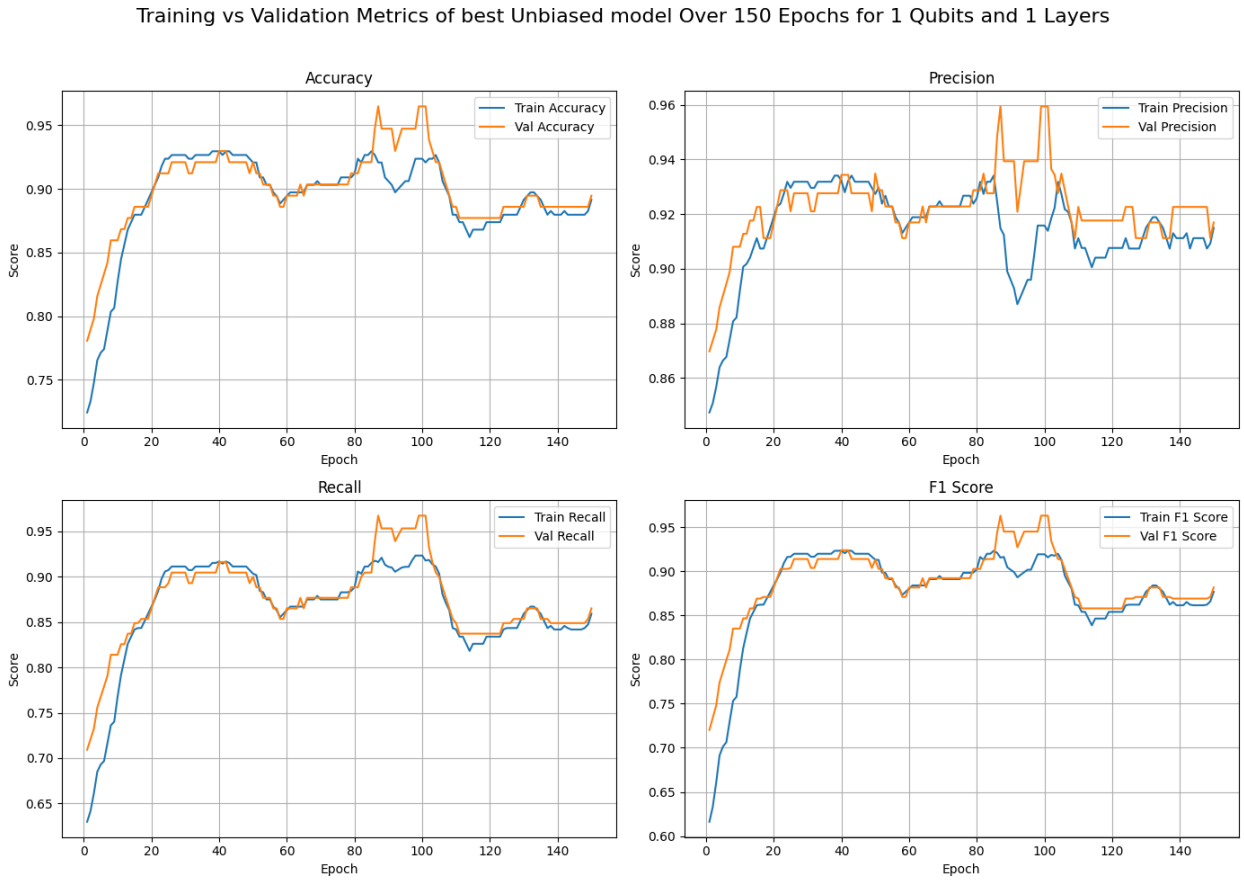

Figure 3.12: Validation and Training curves for the 4 key metrics of the best 1-qubit unbiased classifier

Having successfully addressed the classification problem using a single-qubit configuration, we now extend our investigation to a two-qubit architecture. This transition allows us to explore a larger Hilbert space and evaluate whether the increased dimensionality and the potential for quantum entanglement offer superior discriminative capabilities compared to the 1-qubit model.

3.2 2-qubits Results

By utilizing a two-qubit architecture, we can incorporate four principal components ($2^2 = 4$), a significant increase over the single-qubit case. These four features account for 79.79% of the total variance within the training subset of the tumor classification problem, a figure that closely aligns with the 79.24% variance explained by four components across the entire dataset. Specifically, the first and second components account for 42.34% and 20.63% of the total variance, respectively, while the third and fourth components contribute an additional 10.80% and 6.01%.

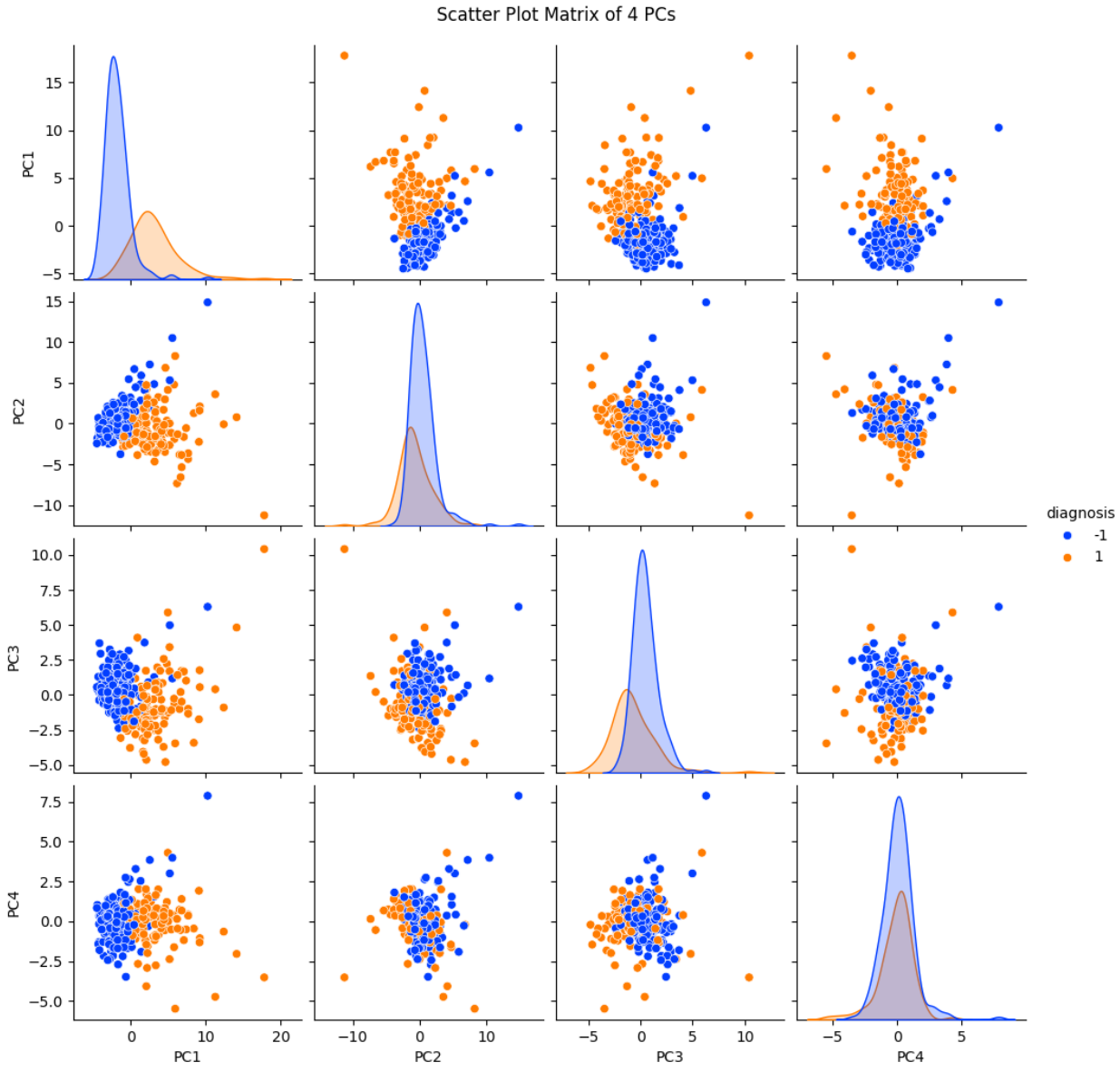

Figure 3.13: Pairplot of the 4 PCs for the training subset

A close inspection of the pairplot in Figure **3.13** for the four principal components reveals that the relationship between the first two dimensions remains consistent with the single-qubit case. More notably, however, are the characteristics of the third and fourth features. The class distributions for the third component exhibit overlap, though to a lesser extent than that observed in the second component. In contrast, the positive class distribution for the fourth component is almost entirely enveloped by that of the negative class. Consequently, it can be argued that the third component is significantly more discriminative and informative than the fourth.

Turning our attention to the pairwise scatterplots, we observe that feature pairs involving the fourth component, when analyzed without the influence of the first, lack clear linear separability. In the PC_2–PC_4 pairing specifically, the majority of samples cluster within the same spatial region, with the negative class distribution effectively eclipsing the positive one. Consequently, we anticipate that the classification task will increase in complexity, as these higher-order components are less inherently informative and the classes become increasingly difficult to distinguish.

Utilizing two qubits allows us to leverage quantum entanglement to our advantage. Consequently, the Rot gates in each layer are followed by a ring of $CNOT$ gates that entangle the qubits. This introduction of entanglement is intended to enhance the classifier's expressive power, assisting the model in navigating the increased dimensionality of the feature space. Reflecting on the limitations and empirical findings discussed previously, we evaluate both biased and unbiased versions of the classifier across configurations of 1, 4, 5, 6, and 7 layers. This results in 6, 24, 30, 36, and 42 trainable parameters for the unbiased cases, respectively, while the biased counterparts require 7, 25, 31, 37, and 43 parameters. To accommodate this expanded parameter space, we have extended the training duration to 200 epochs to ensure sufficient optimization.

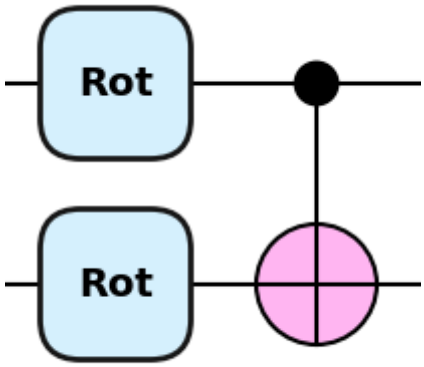

Figure 3.14: A single layer of the 2-qubit VQC

An inspection of the evaluation results across ten independent runs allows for several key conclusions. Most notably, the performance of the single-layer classifier has decreased significantly compared to the 1-qubit model. In the unbiased configuration, the model

achieves an accuracy of 74.30%, a precision of 72.61%, a recall of 70.58%, and an F1-score of 71.21%. The biased case yields slightly higher results, with 75.61% accuracy, 74.18% precision, 72.01% recall, and a 72.68% F1-score. In both instances, these metrics represent a substantial decline from the 1-qubit baseline and only marginally surpass the random chance threshold of 62.70%. However, increasing the circuit depth to four layers leads to a dramatic improvement: the unbiased classifier reaches 90.18% accuracy and an 89.16% F1-score, while the biased version achieves 91.40% accuracy and a 90.73% F1-score. These gains suggest that the expanded feature dimensionality necessitates a higher number of trainable parameters to achieve satisfactory performance. This trend continues as the depth increases to five, six, and seven layers. For these configurations, the unbiased classifier yields accuracies of 91.40%, 92.28%, and 91.84%, and F1-scores of 90.65%, 91.51%, and 90.95%, respectively. Similarly, the biased classifier scores 91.67%, 92.72%, and 91.84% in accuracy, and 90.92%, 92.12%, and 91.16% in F1-score, indicating that peak performance is reached at approximately six layers before stabilizing or slightly declining.

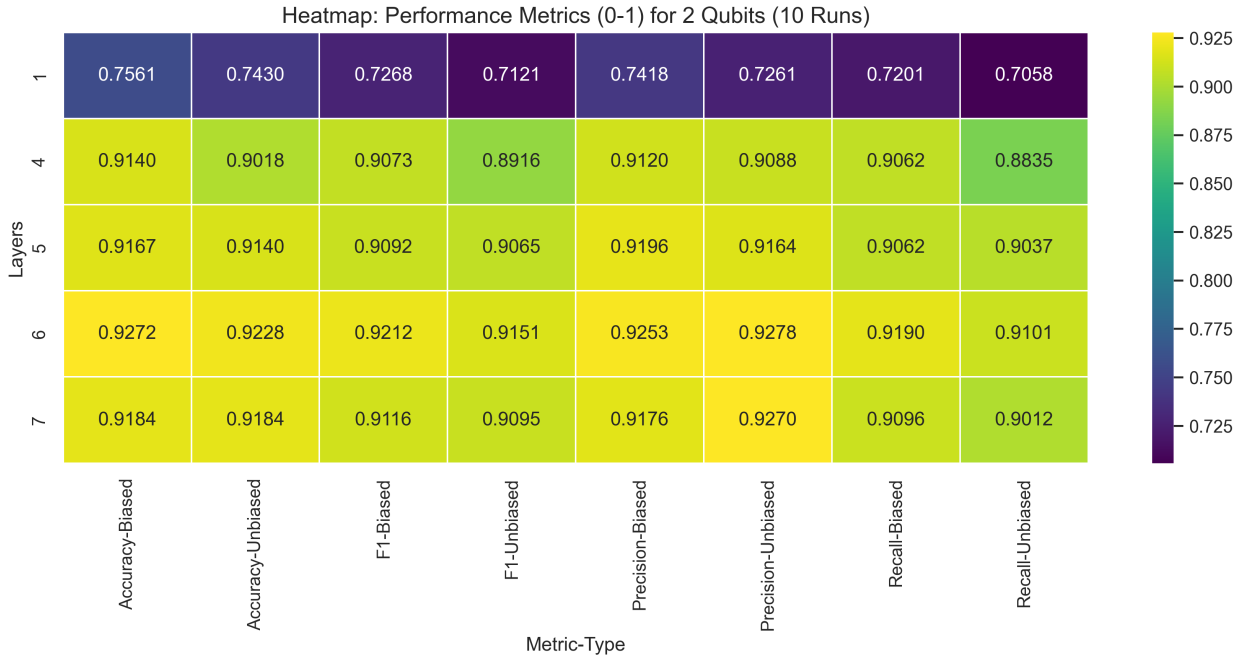

Figure 3.15: Performance Results for 2-qubit classifiers after 10 runs

In terms of training efficiency across the varying circuit depths, the unbiased classifier identified its optimal parameter values after 8, 152, 142, 131, and 127 epochs for each respective layer configuration, with average training durations of 7.48, 13.91, 15.95, 18.53, and 21.23 seconds. For the biased variant, the required optimization periods reached 161, 156, 139, 118, and 126 epochs, while the corresponding training procedures averaged 7.36, 14.03, 16.41, 18.70, and 21.57 seconds.

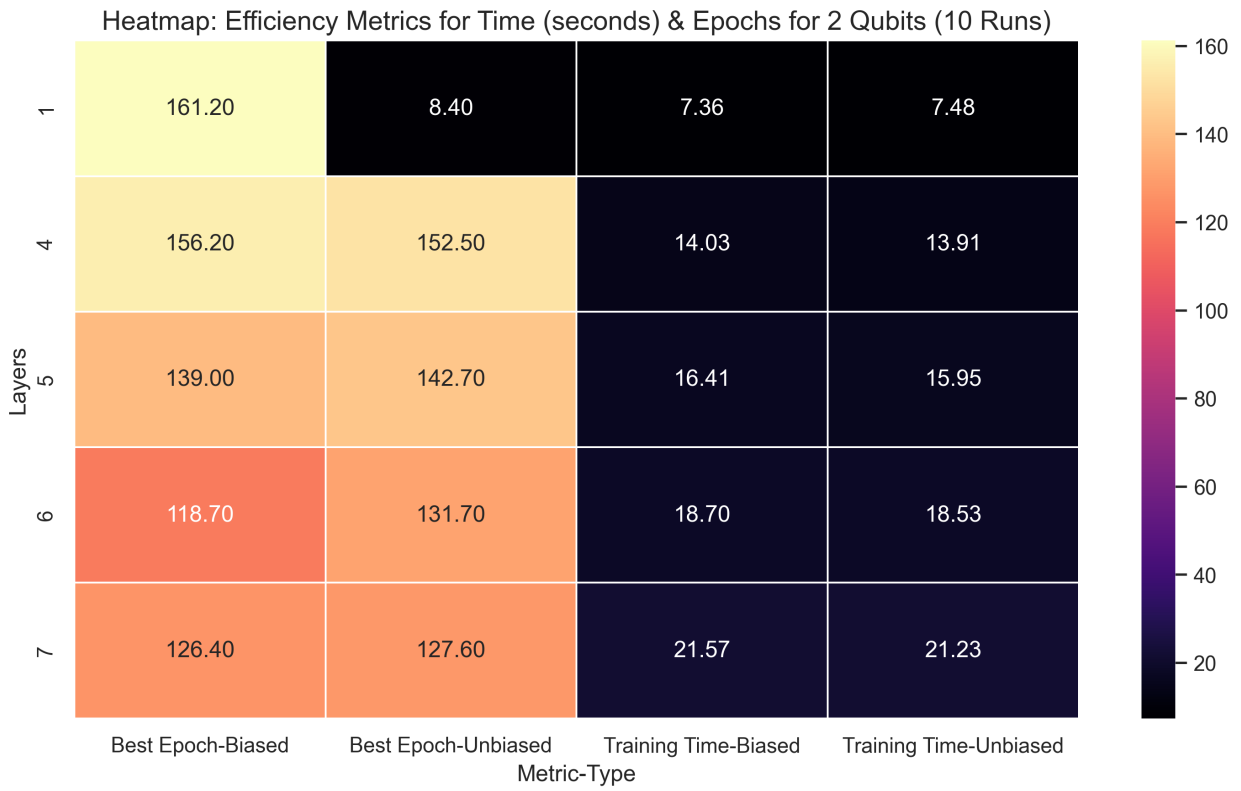

Figure 3.16: Efficiency Results for 2-qubit classifiers after 10 runs

From a comparative perspective, a clear trend emerges in our results. First, the single-layer classifier significantly underperforms in both configurations. This is expected as the dimensionality of the Hilbert space has increased, and the limited number of trainable parameters in a single layer is insufficient to navigate it effectively. To improve performance, we must increase the expressivity of the circuit by introducing more parameters. Consequently, the key performance metrics improve drastically as the number of layers increases. However, this approach reaches a point of diminishing returns, as performance gains become marginal beyond the fourth layer.

Interestingly, both versions of the classifier demonstrate relatively comparable performance. While the biased classifier achieves marginally higher scores across all metrics, these differences remain within the scope of statistical error. A more significant observation is that the biased classifier appears to be at least as efficient as its unbiased counterpart, if not more so. Aside from the single-layer case, where both models yielded unsatisfactory results, the biased classifier converges to optimal parameter values in fewer epochs while maintaining training times similar to the unbiased version. This slight advantage in efficiency justifies a preference for the biased approach, notwithstanding its negligible superiority in predictive accuracy.

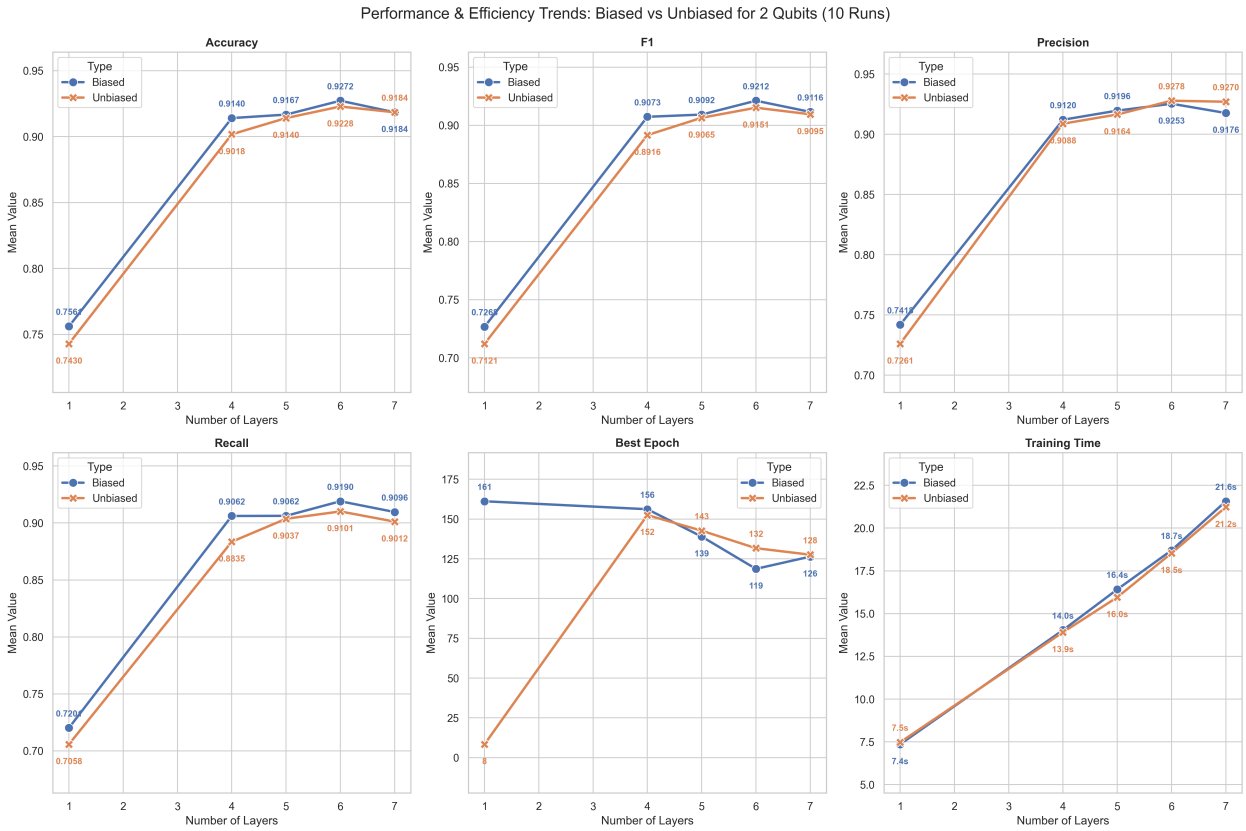

Figure 3.17: Performance and Efficiency Trends of Biased and Unbiased Classifier for 2 Qubits

In regard to the top performers across all circuit depths and experimental runs, the most effective unbiased classifier utilized a 6-layer VQC. This configuration reached an accuracy of 94.74%, a precision of 94.80%, a recall of 93.85%, and an F1-score of 94.29% at the 94th epoch.

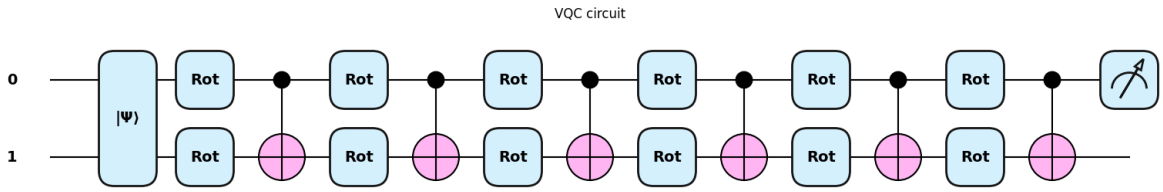

Figure 3.18: VQC of the best 2-qubit unbiased classifier

Regarding the evolution of the loss function, we observe that the sharp decline in cost begins after the 30th epoch and continues until approximately the 75th epoch, at which point it appears to reach a plateau. Subsequently, the loss fluctuates sporadically before reaching its absolute minimum after the 100th epoch. Although the timing for finding the best parameters is closer to this minimum than in previous instances, we still observe

that the absolute minimization of the cost does not coincide perfectly with the highest performance scores.

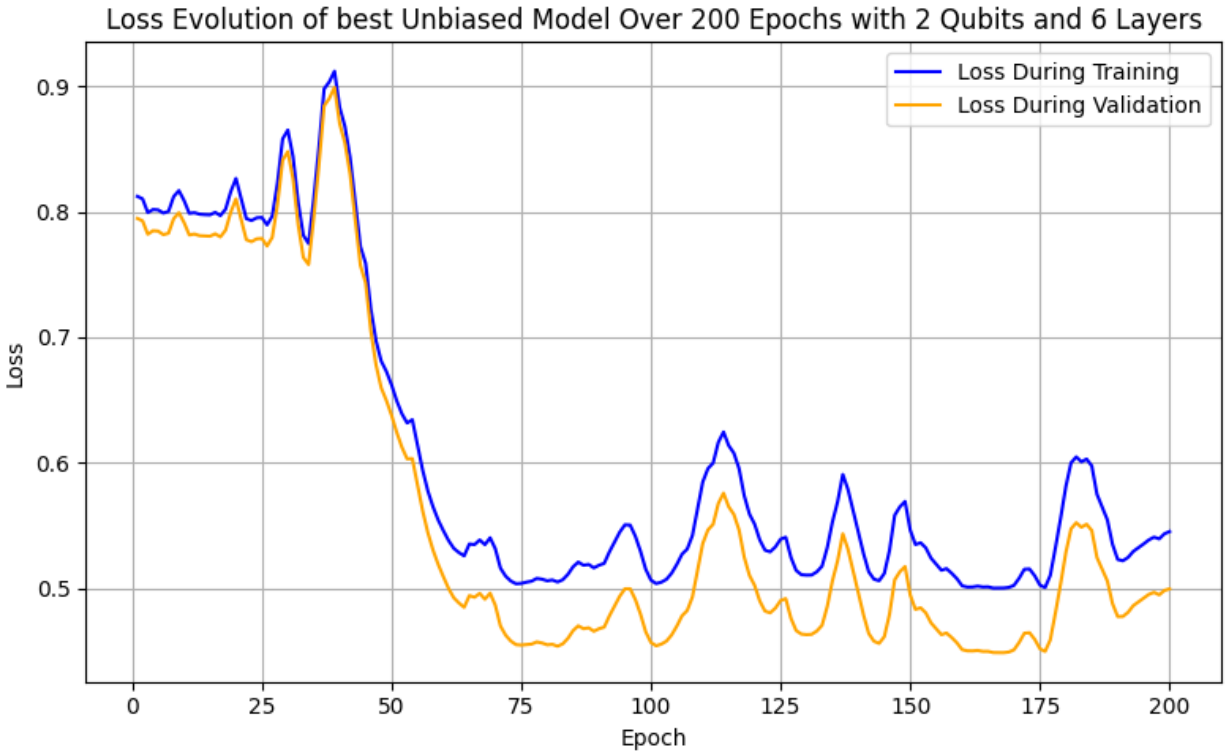

Figure 3.19: Evolution of the loss function of the best 2-qubit unbiased classifier

Upon observing the performance curves of the model, we detect somewhat erratic behavior. The model undergoes an extensive period of underperformance until the 50th epoch, at which point it begins to improve, ultimately reaching its peak by the 94th epoch. Following this peak, performance declines rapidly before once again exhibiting signs of improvement. Despite these fluctuations, there is no evidence of overfitting or underfitting, as the training and validation curves follow a consistent trend.

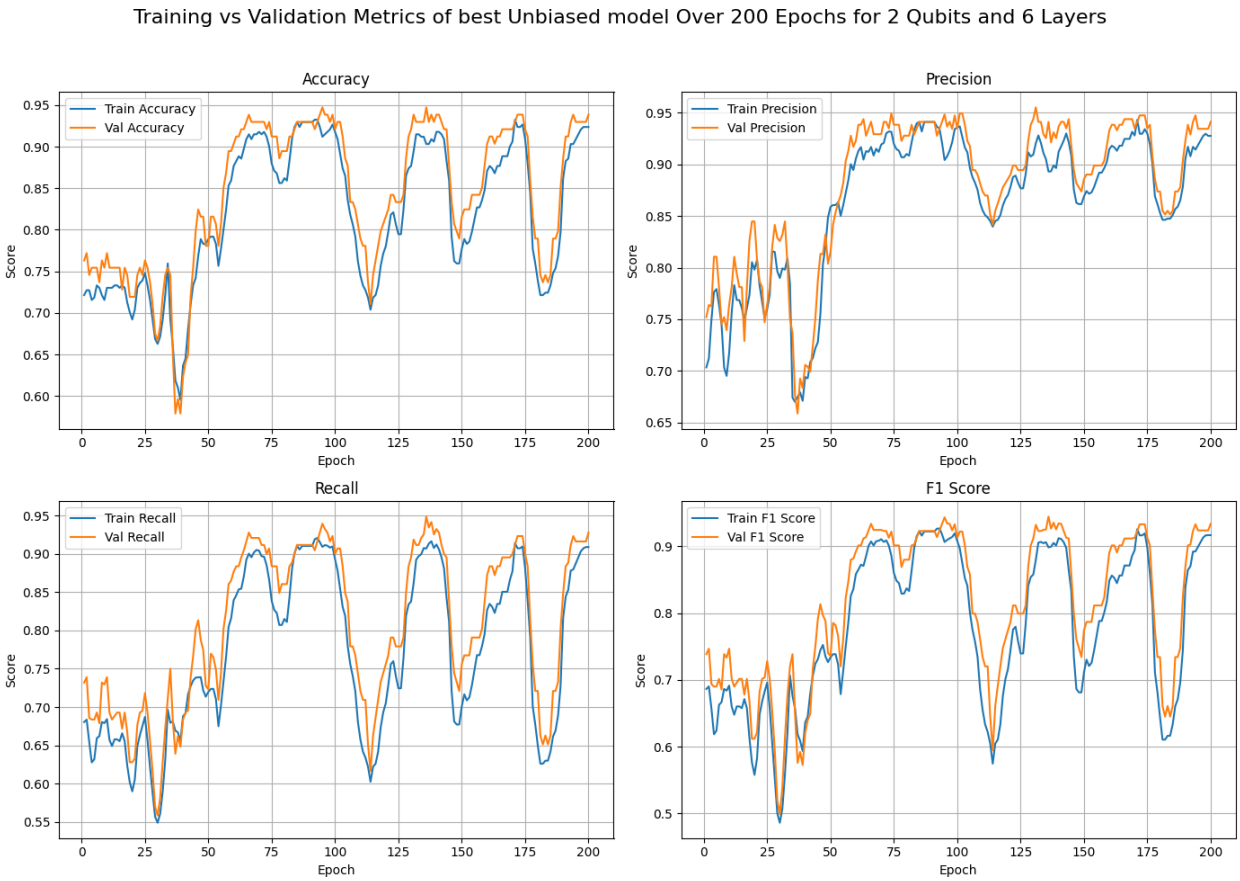

Figure 3.20: Validation and Training curves for the 4 key metrics of the best 2-qubit unbiased classifier

For the best-performing biased model, we identify a classifier with a VQC architecture consisting of 6 layers. This classifier achieves an accuracy of 95.61%, a precision of 95.10%, a recall of 95.54%, and an F1-score of 95.31% at the 75th epoch.

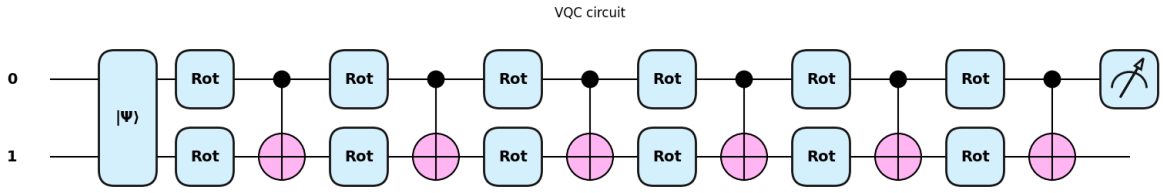

Figure 3.21: VQC of the best 2-qubit biased classifier

The evolution of the loss function for this classifier is quite similar to that of its unbiased counterpart, with the sharp decrease appearing after 30 epochs and lasting until the 70th epoch. Subsequently, we observe the same pattern in which the cost increases and then decreases until it reaches a plateau after 190 epochs. It is evident that the discovery of

the best parameters does not occur when the cost is at its lowest, as the loss function exhibits an upward trend at the 75th epoch.

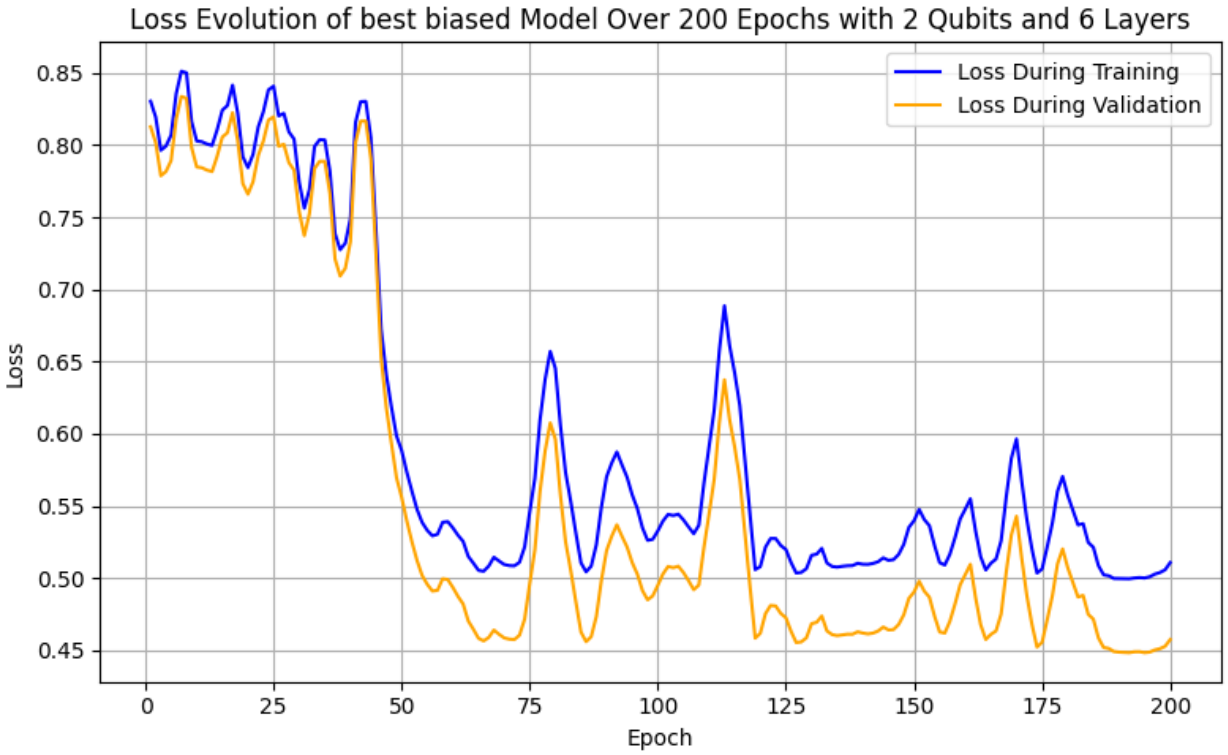

Figure 3.22: Evolution of the loss function of the best 2-qubit biased classifier

An inspection of the progression of key metrics throughout the 200 epochs reveals that the biased model exhibits the same erratic behavior as its unbiased counterpart. Performance begins with relatively low scores, not exceeding 80%. Around the 45th epoch, the performance increases drastically, reaching its peak at the 75th epoch. This is followed by a series of sharp decreases and subsequent increases in performance until a local maximum occurs at approximately epoch 175, after which it begins to decline. Nevertheless, because the behavior of the training and validation curves remains similar, there is no evidence to suggest significant overfitting or underfitting of the model.

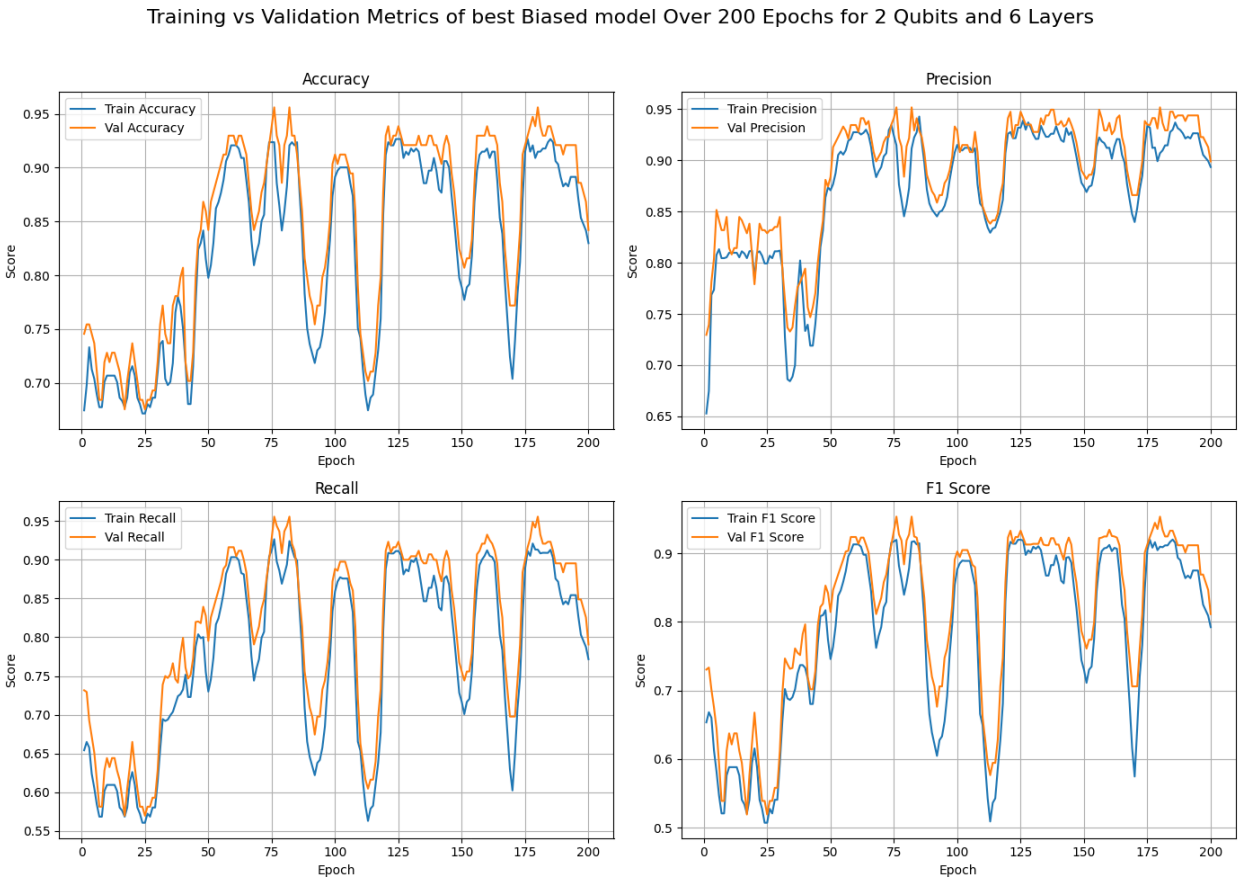

Figure 3.23: Validation and Training curves for the 4 key metrics of the best 2-qubit biased classifier

Having concluded our investigation of the two-qubit paradigm, we proceed to increase the number of available qubits and, consequently, the number of principal components upon which the model will rely.

3.3 3-qubits Results

Since we are now working with three qubits, our data encoding scheme allows us to utilize eight features. This enables us to perform PCA with eight principal components serving as our features. With this approach, we can express 92.71% of the variance in the training data for our classification task, which is a substantial increase compared to the 79.79% achieved in the two-qubit case. This result was expected, as the exploratory data analysis of the classical approach had already determined that eight principal components can explain up to 92.60% of the total variance in our data. With each of the eight components accounting for 42.34%, 20.63%, 10.80%, 6.01%, 4.65%, 4.20%, 2.46%, and 1.60% of the total explained variance respectively, we anticipate that each new component added will be less discriminatively informative than the previous ones.

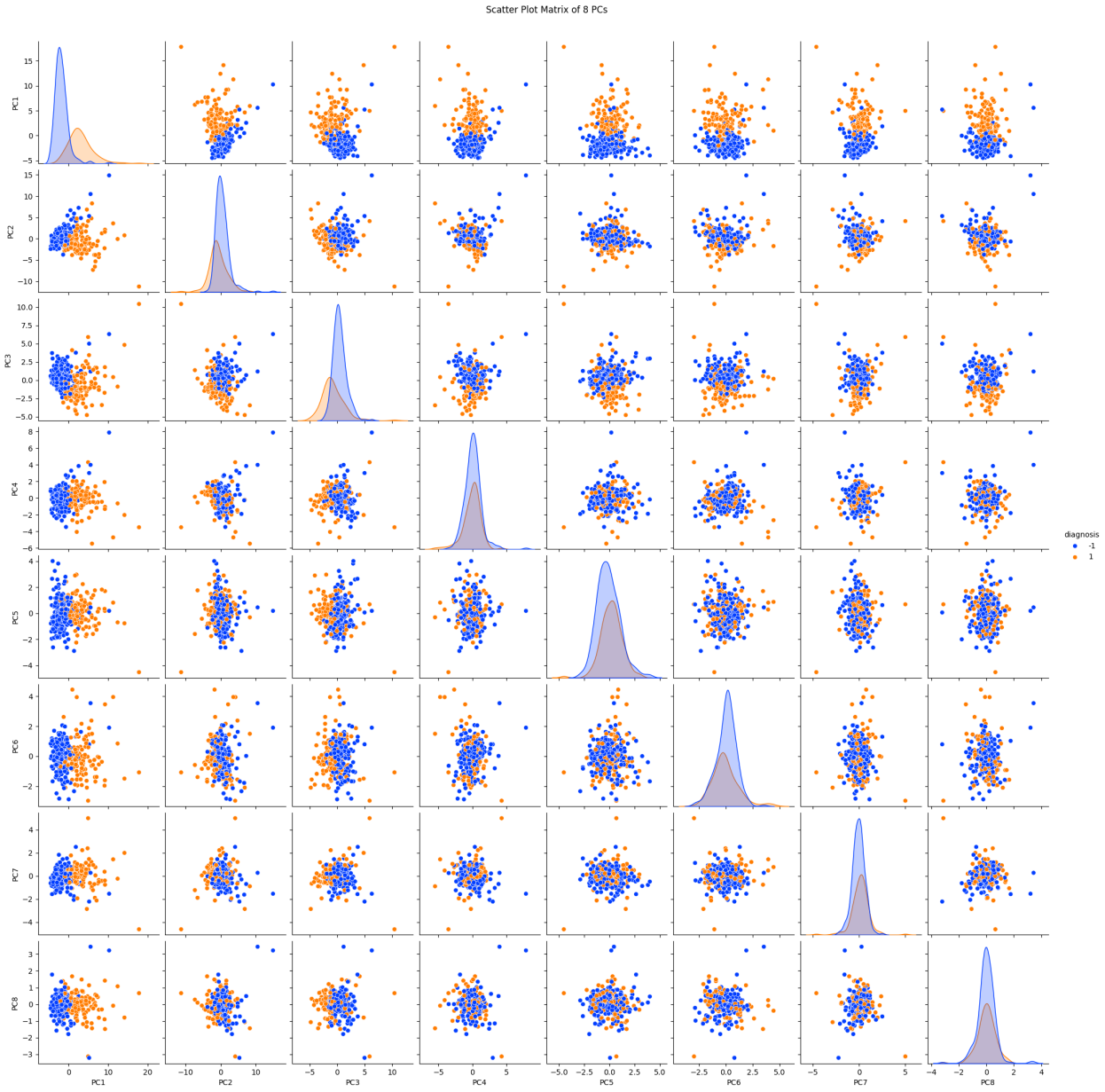

Figure 3.24: Pairplot of the 8 PCs for the training subset

A closer examination of the pairplot in Figure **3.24** appears to validate this trend. Along the main diagonal, we can observe that as we progress through the principal components, their distributions exhibit increasing overlap. Specifically, in the eighth principal component, the negative class distribution encapsulates the positive class almost entirely. This pattern is also evident, though to a lesser extent, in the fifth, sixth, and seventh features, which is consistent with the fact that each of these components individually accounts for less than 5% of the total explained variance.

Inspecting the remaining scatterplot cells reveals a distinct lack of linear separability, particularly in pairs involving the newly introduced components. For instance, the pairwise

scatterplot between the fourth and fifth principal components exhibits little to no separability. Interestingly, all scatterplot pairs involving the first principal component appear to maintain a clear class distinction, which is a direct result of the significant amount of variance this component captures. Ultimately, we expect the classification task to become more challenging as the new features, while providing a more descriptive representation of the dataset, appear to be less informative for discriminating between classes.

To address our problem, we evaluate both biased and unbiased variational quantum classifiers across configurations of 1, 5, 6, and 8 layers. These configurations involve 9, 45, 54, and 72 trainable parameters for the unbiased version, and 10, 46, 55, and 73 parameters for the biased variant, respectively. The layer architecture remains consistent with our previous models, utilizing a combination of rotation gates and $CNOT$ gates. Although several three-qubit gates are available, we maintain simplicity by using only single-qubit rotations and two-qubit $CNOT$ gates. To accommodate the increased number of trainable parameters relative to earlier experiments, we have extended the training duration to 250 epochs.

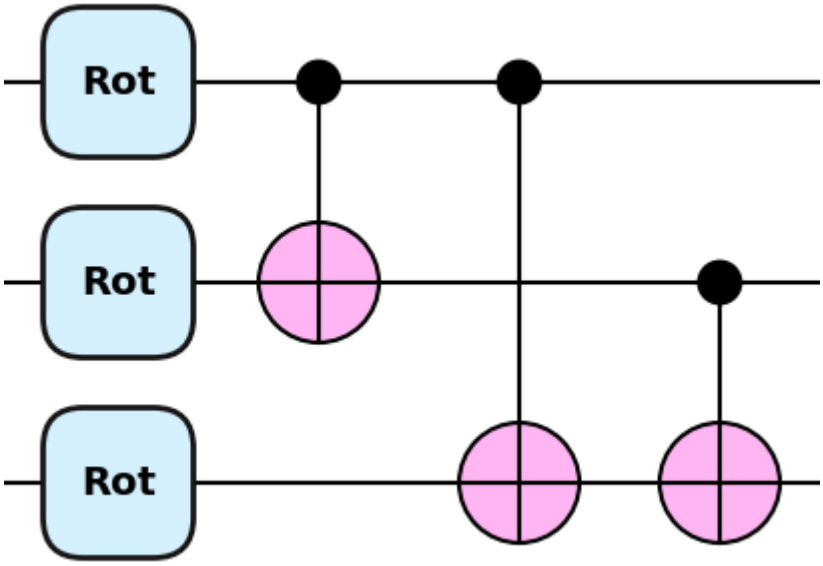

Figure 3.25: A single layer of the 3-qubit VQC

An inspection of the average results across ten runs reveals that the simplest single-layer classifier significantly underperforms, failing to surpass the random chance baseline. The unbiased version achieves an accuracy of 61.40%, a precision of 53.96%, a recall of 52.03%, and an F1-score of 49.01%. Similarly, the biased version performs slightly better but remains below standards, reaching an accuracy of 61.84%, a precision of 54.72%, a recall of 52.33%, and an F1-score of 49.18%. Performance improves notably as more layers and trainable parameters are introduced. For the unbiased classifier, the 5, 6, and 8-layer configurations yield accuracies of 91.58%, 91.40%, and 92.02%, with corresponding F1-scores of 90.78%, 90.64%, and 91.37%. Likewise, the biased classifier achieves

accuracy scores of 91.49%, 92.37%, and 92.46%, and F1-scores of 91.05%, 91.71%, and 91.92% for the same respective layer counts.

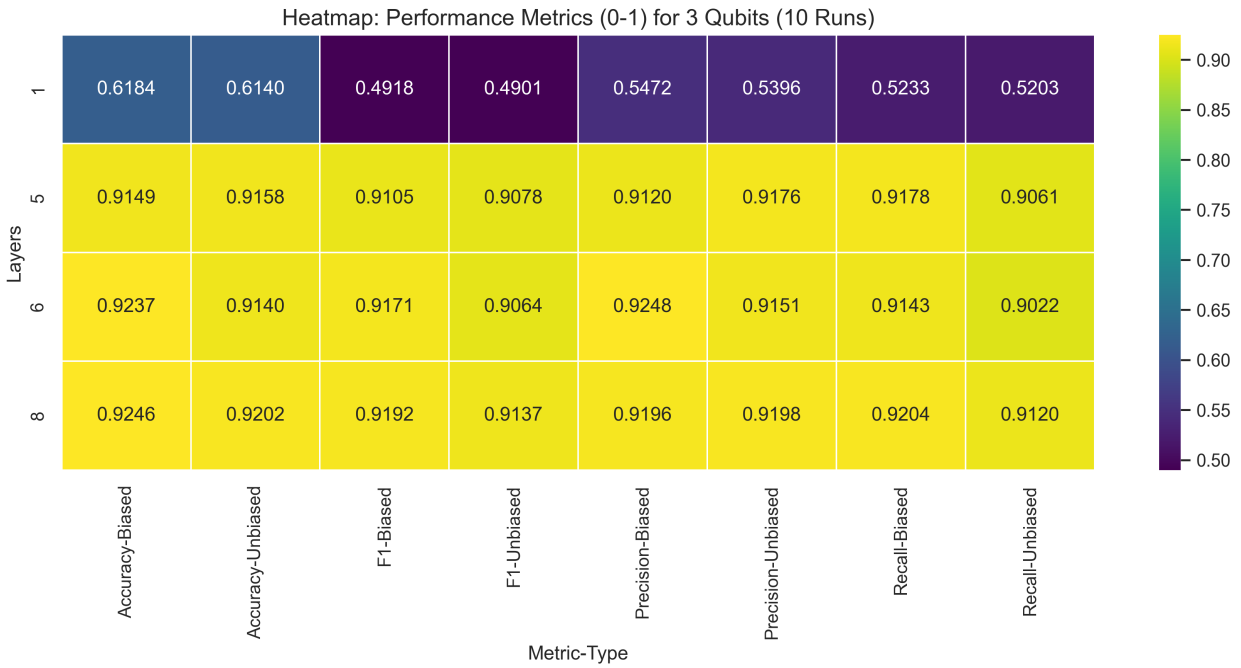

Figure 3.26: Performance Results for 3-qubit classifiers after 10 runs

Regarding the efficiency of the classifiers, the unbiased model reached its optimal parameter values after 92, 231, 214, and 191 epochs for the 1, 5, 6, and 8-layer configurations, respectively. These training procedures lasted an average of 10.42, 29.57, 35.46, and 45.50 seconds for each corresponding depth. Similarly, the biased classifier identified its best parameters after 49, 211, 209, and 196 epochs, with the training sessions averaging 10.55, 31.43, 36.08, and 46.78 seconds.

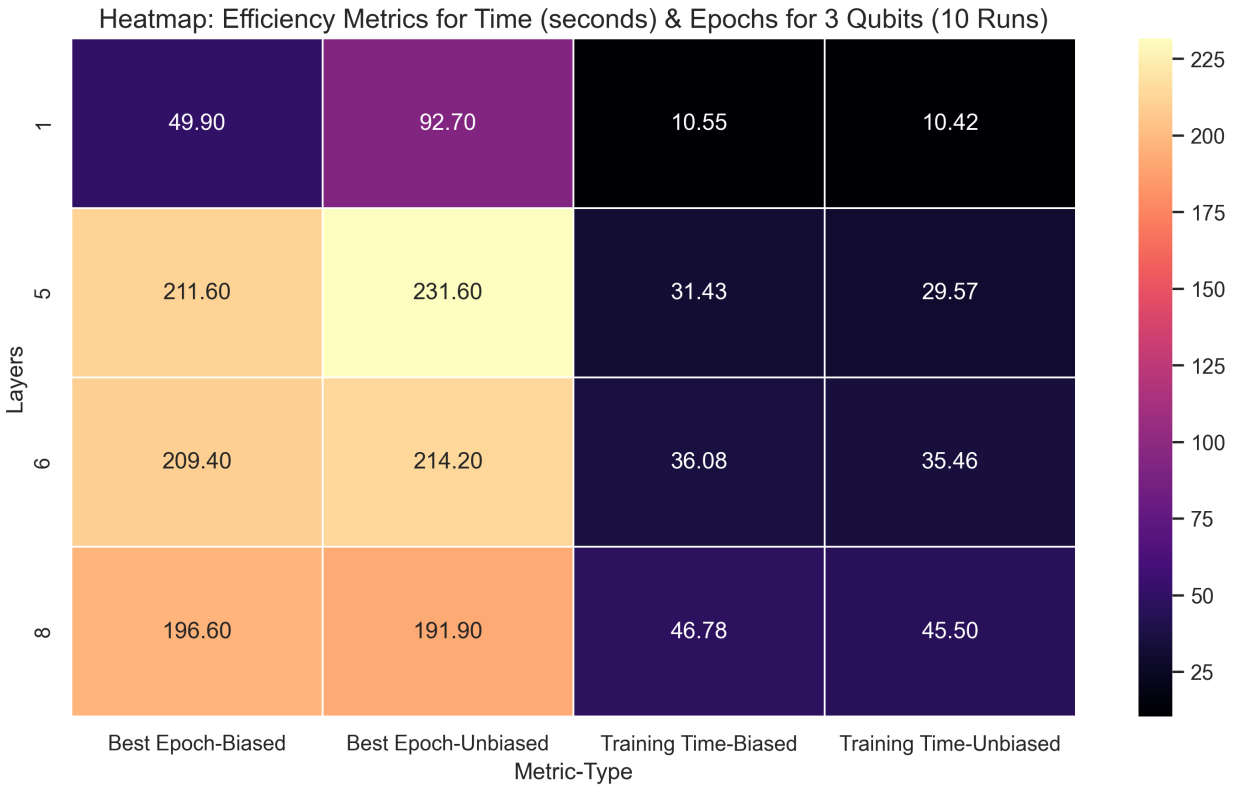

Figure 3.27: Efficiency Results for 3-qubit classifiers after 10 runs

A comparison of the biased and unbiased approaches reveals several clear insights. First, the single-layer version of both configurations fails to generalize, yielding disappointing results. To provide context, a baseline classifier that simply categorizes every instance as the negative class, the dominant class in this dataset, would achieve an accuracy of 62.7%. Consequently, it is apparent that increasing the number of trainable parameters by extending the depth of the circuit is essential. By doing so, we are able to reach far more substantial results, regardless of which classifier version is utilized.

Considering that recall is critical for this tumor classification problem, a case can be made for preferring the biased classifier, as it consistently achieves higher scores in this metric. Additionally, the biased approach maintains training times comparable to the unbiased version while converging to optimal parameter values slightly faster. However, it remains important to note that these marginal differences in performance and efficiency may be attributable to statistical variance rather than a definitive architectural advantage.

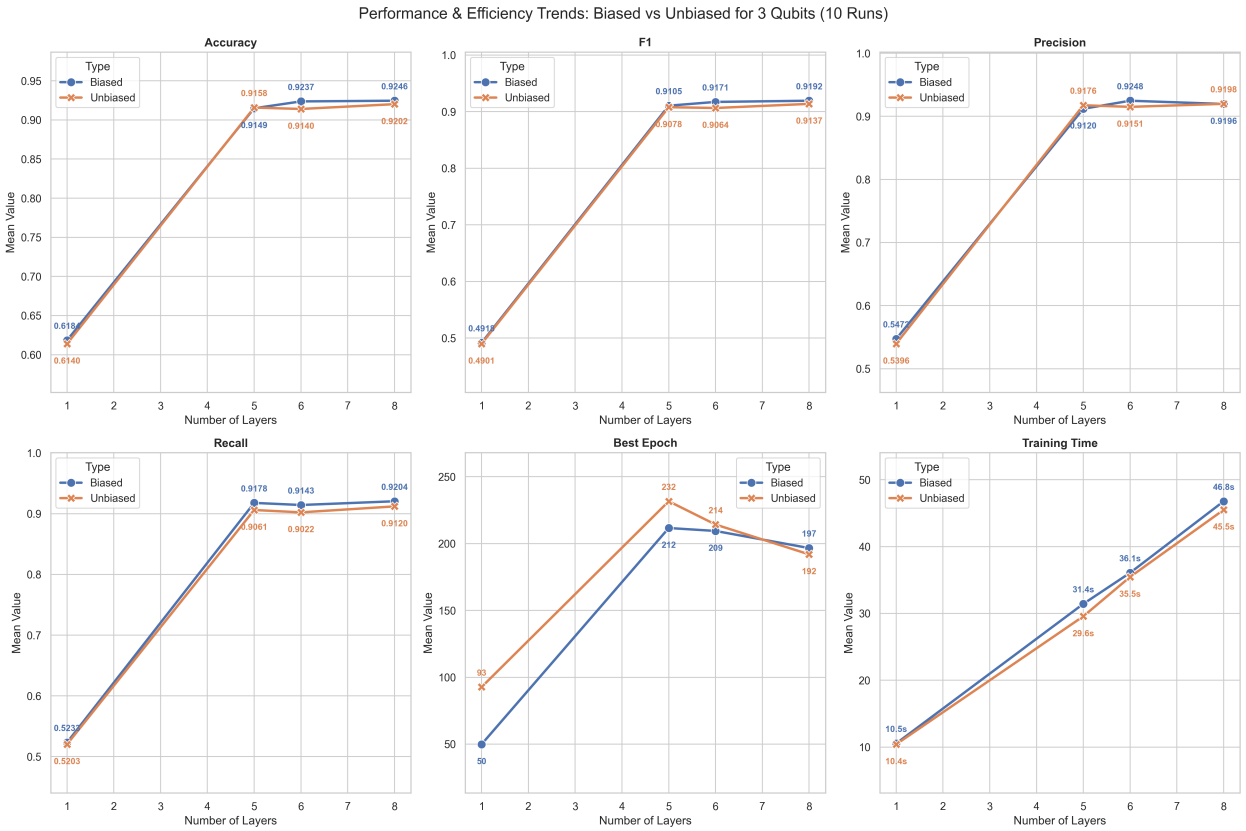

Figure 3.28: Performance and Efficiency Trends of Biased and Unbiased Classifier for 3 Qubits

When identifying the top-performing unbiased classifier, we report a model whose VQC consists of 5 layers. After 234 epochs, this configuration achieved an accuracy of 95.61%, a precision of 94.83%, a recall of 96.03%, and an F1-score of 95.35%.

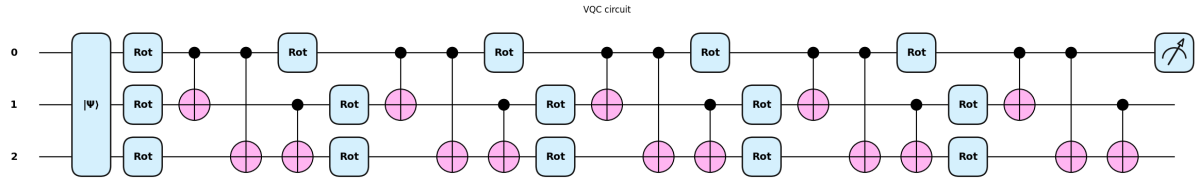

Figure 3.29: VQC of the best 3-qubit unbiased classifier

A closer examination of the cost function evolution reveals that it is no longer characterized by the sharp decreases observed in previous models. This effect is visible only to a much lesser extent between the 100th and 150th epochs. Prior to that, the loss function undergoes minor fluctuations, though a general downward trend remains present. Following the 150th epoch and the subsequent minor peaks, the loss function reaches a plateau, consistently remaining between 0.6 and 0.9.

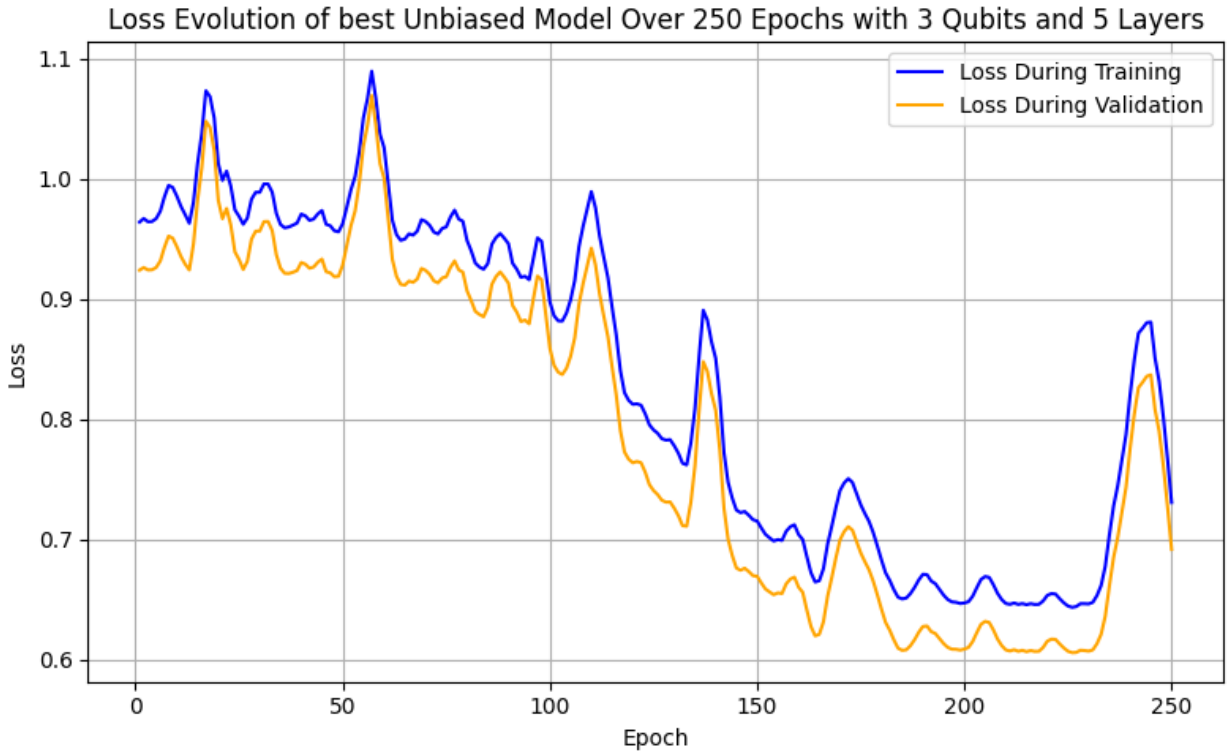

Figure 3.30: Evolution of the loss function of the best 3-qubit unbiased classifier

There appears to be a slight difference in the progression of the key metrics. For most metrics, although the training and validation curves are not identical, they follow similar patterns by increasing and decreasing in the same regions. The only exception is the precision metric, where the validation and training curves appear significantly divergent initially. This gap narrows after the 125th epoch, after which the two curves share many similarities, particularly in their general trends. This is encouraging since the best parameters are identified at the 234th epoch, a point where the model shows no indications of overfitting or underfitting across any of the metrics.

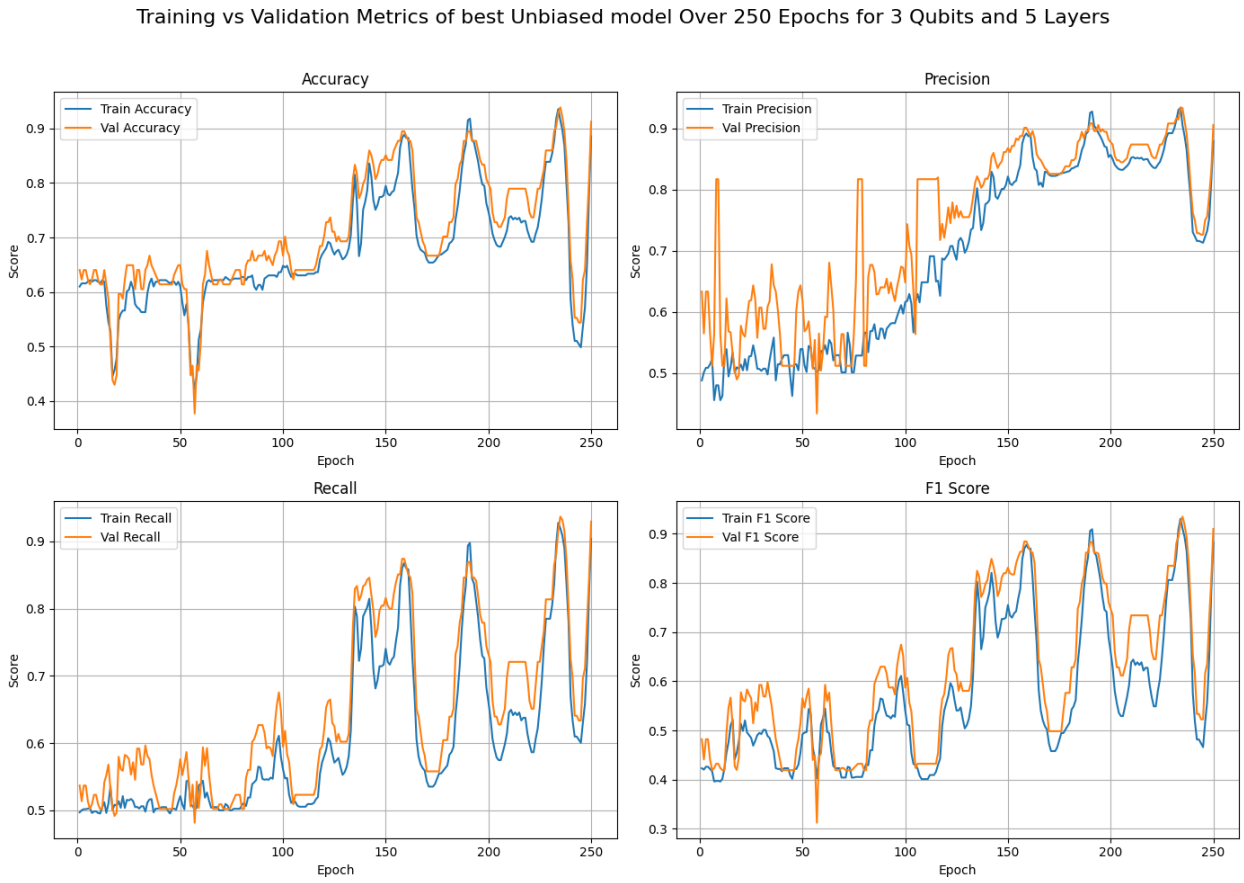

Figure 3.31: Validation and Training curves for the 4 key metrics of the best 3-qubit unbiased classifier

For the biased method, the best-performing classifier consists of 8 layers and, after 201 epochs, yields an accuracy of 95.61%, a precision of 96.05%, a recall of 94.54%, and an F1-score of 95.21%.

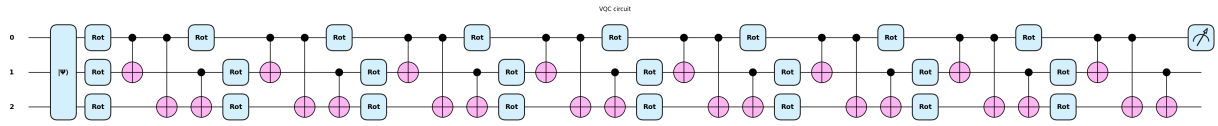

Figure 3.32: VQC of the best 3-qubit biased classifier

If we examine the evolution of the loss function more closely, we notice a pattern similar to that of the unbiased case. The sharp and immediate decrease in cost observed in previous iterations of our classifiers is absent. Instead, it is replaced by a series of minor fluctuations, followed around the 50th epoch by a gradual yet steep decline in cost that continues until the 110th epoch. Subsequently, the loss appears to reach a plateau, notwithstanding some inconsequential fluctuations. Notably, the cost is not at its absolute minimum at the 201st epoch, when the best parameters are identified.

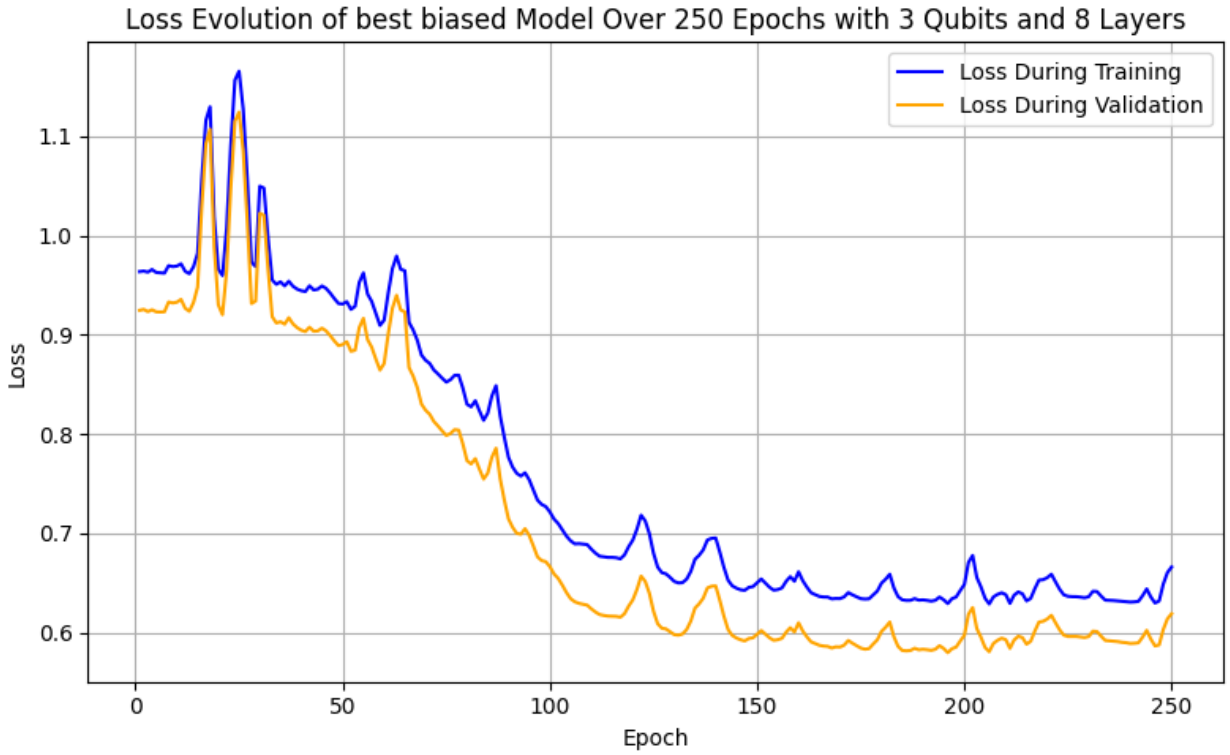

Figure 3.33: Evolution of the loss function of the best 3-qubit biased classifier

Unlike its unbiased counterpart, the validation and training curves for the key metrics are quite similar. With the exception of the precision metric, they remain close throughout the training process, and the distance between them remains minimal. Even regarding precision, the behavior stabilizes and the curves align more closely once the training passes the 150th epoch.

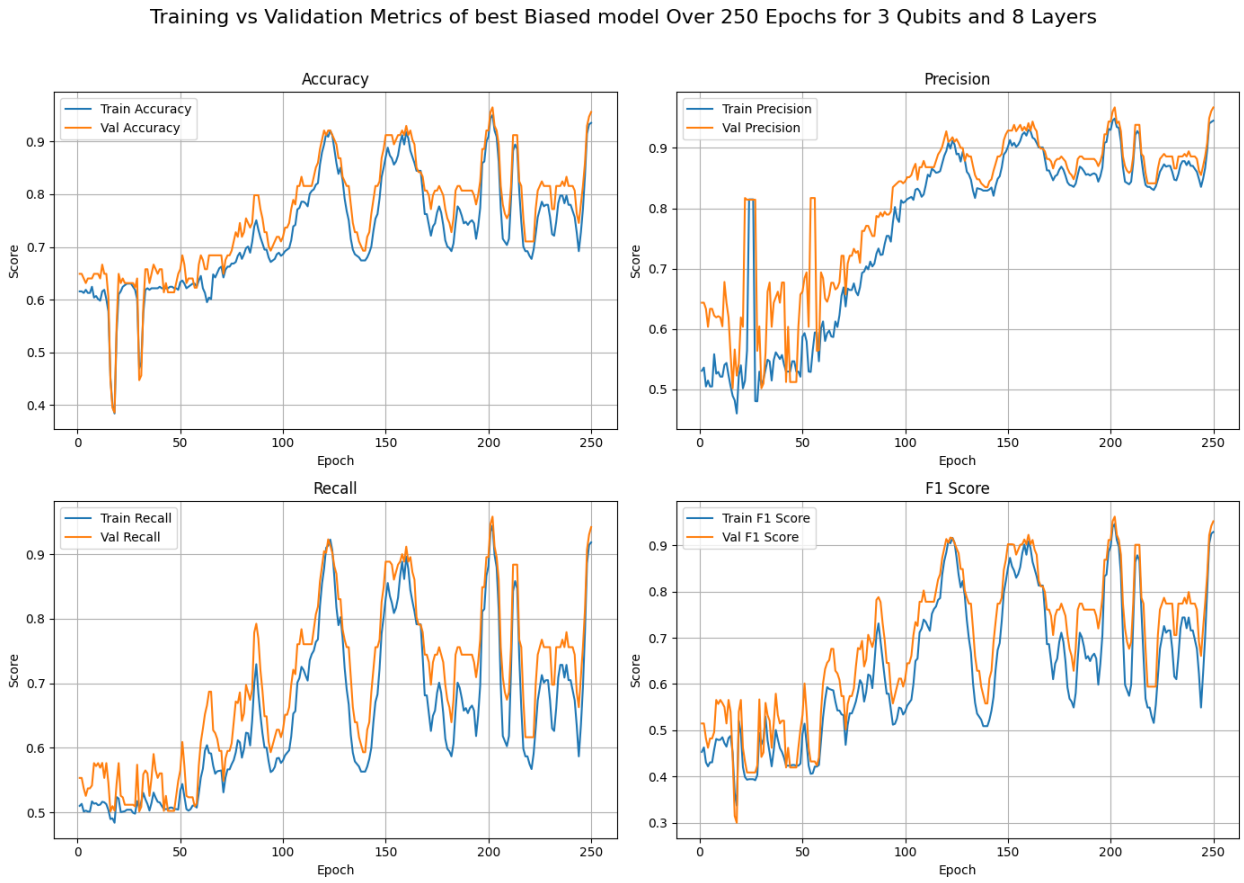

Figure 3.34: Validation and Training curves for the 4 key metrics of the best 3-qubit biased classifier

Having completed the classification task using the three-qubit approach, and consequently the eight features associated with it, we now move on to the four-qubit configuration.

3.4 4-qubits Results

Our 4-qubit implementation allows us, through the use of the Amplitude-Efficient State Preparation routine, to encode 16 features. These features are the 16 principal components obtained by fitting the PCA transformer onto the training subset and using it to transform all three data subsets. With this strategy, we are able to explain 99.08% of the total variance in the training data, which is slightly higher than the 98.92% that could be explained if the transformer were fitted on the entirety of the original dataset. Each of the principal components accounts for 42.34%, 20.63%, 10.80%, 6.01%, 4.65%, 4.20%, 2.46%, 1.60%, 1.38%, 1.27%, 1.11%, 0.92%, 0.72%, 0.39%, 0.28%, and 0.25% of the total explained variance ratio, respectively. We observe that with 16 principal components, we are able to describe our problem almost in its entirety, with the drawback being that some of the newly introduced features account for only miniscule amounts of variance.

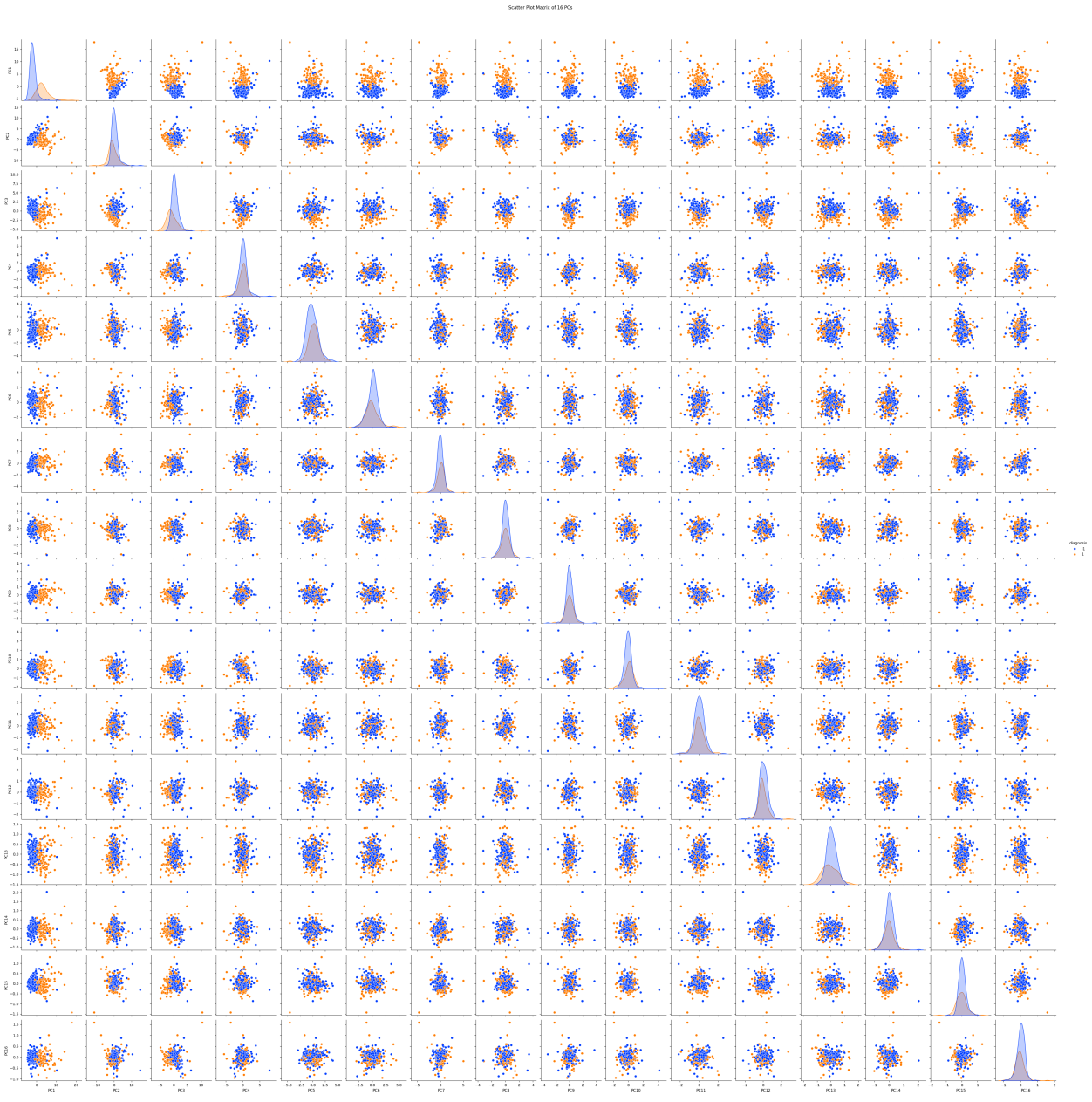

Figure 3.35: Pairplot of the 16 PCs for the training subset

As with the eight-principal-component case, each subsequent component becomes less informative. A close examination of the main diagonal reveals that the primary body of the positive class distribution is entirely enveloped by the negative distribution across the eight newly added components. This is expected, given that several of these components account for less than 1% of the total explained variance ratio. Consequently, the introduction of these features is unlikely to contribute significantly to the classification of the input. This effect becomes even more apparent when focusing on the scatterplots involving these eight new features. No linear class separability is visible, as the blue negative class and the orange positive class conglomerate at the center of each corresponding cell.

To address the problem, we maintain the existing layer architecture, where each layer consists exclusively of rotation gates and $CNOT$ gates. To account for the increased complexity of the classification task, we evaluate biased and unbiased classifiers with variational quantum circuit depths of 1, 5, 10, and 15 layers. Consequently, the unbiased configurations require training 12, 60, 120, and 180 parameters, while the biased versions involve 13, 61, 121, and 181 trainable parameters, respectively. Given the increase in trainable parameters and the larger Hilbert space, each model is trained for 300 epochs, representing a slight increase over previous experiments.

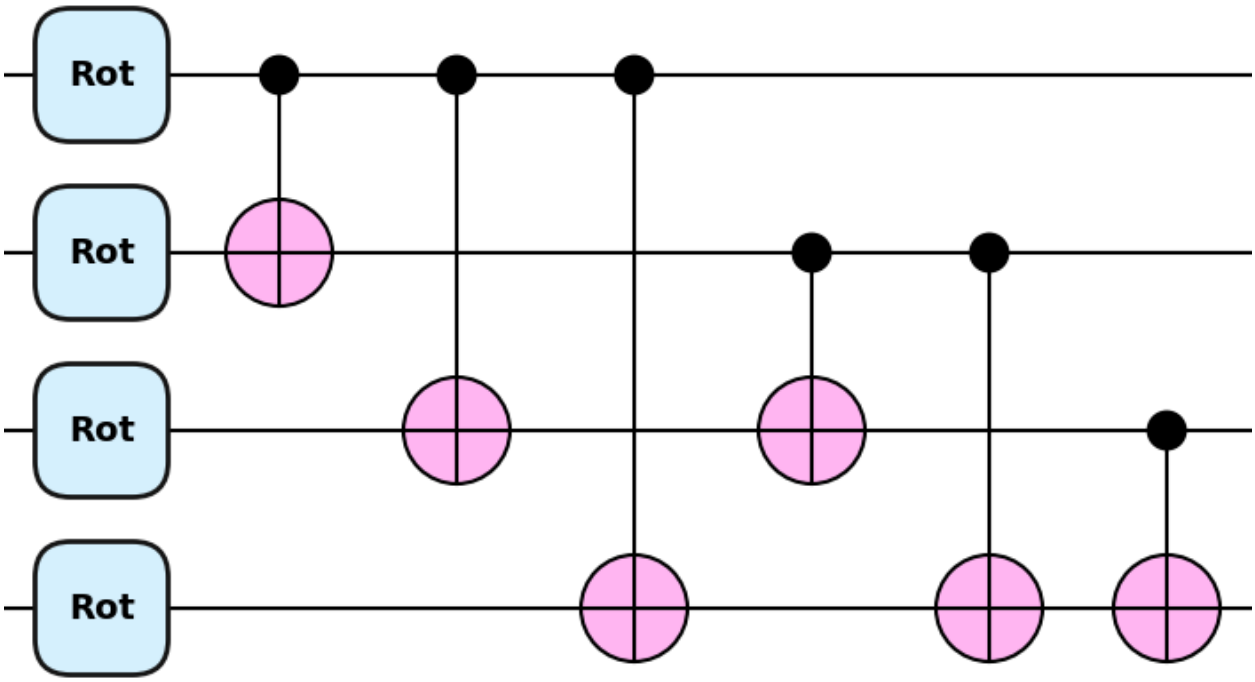

Figure 3.36: A single layer of the 4-qubit VQC

An inspection of the average scores obtained after ten runs reveals that the simple one-layered classifier performs poorly, with some metrics falling below 50%. Specifically, the unbiased version achieves an accuracy of 61.40%, a precision of 50.35%, a recall of 50.10%, and an F1-score of 43.64%, while the biased counterpart yields an average accuracy of 61.23%, a precision of 49.90%, a recall of 49.96%, and an F1-score of 43.55%. Interestingly, increasing the circuit depth to five layers shows improvement, yet the results remain underwhelming; the unbiased classifier reaches an average accuracy of 72.02%, a precision of 72.99%, a recall of 65.55%, and an F1-score of 64.52%, whereas the biased version performs notably better with an accuracy of 82.46%, a precision of 82.03%, a recall of 80.65%, and an F1-score of 80.69%. The results gain significant weight as we move to the ten-layer circuit, where the unbiased classifier achieves an accuracy of 89.56%, a precision of 90.71%, a recall of 87.32%, and an F1-score of 88.35%, and the biased version reaches an accuracy of 89.91%, a precision of 90.65%, a recall of 88.44%,

and an F1-score of 89.01%. Performance improves further with 15 layers, as the unbiased classifier returns an accuracy of 93.42%, a precision of 94.47%, a recall of 91.72%, and an F1-score of 92.70%, while the biased counterpart yields an accuracy of 92.72%, a precision of 92.88%, a recall of 91.81%, and an F1-score of 92.10%.

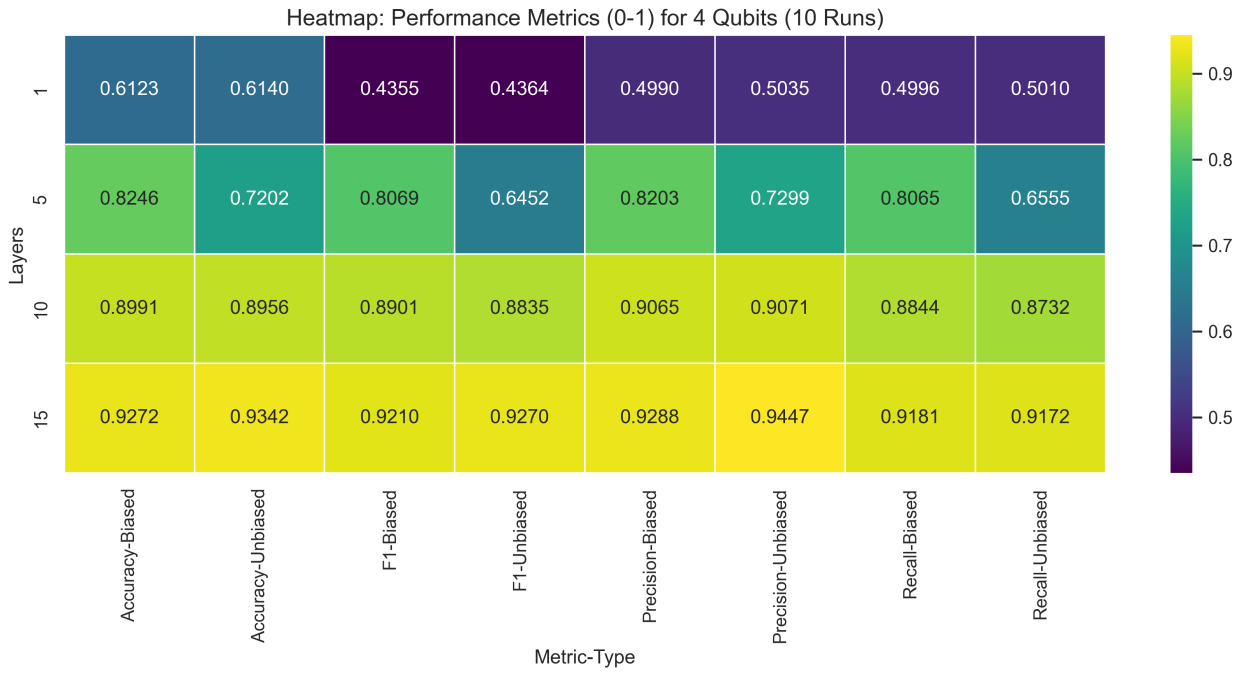

Figure 3.37: Performance Results for 4-qubit classifiers after 10 runs

Regarding the efficiency of the 4-qubit classifiers across depths of 1, 5, 10, and 15 layers, the unbiased method identified its optimal parameters after 108, 252, 241, and 226 epochs, respectively, with training procedures lasting an average of 16.06, 56.19, 98.02, and 146 seconds for each corresponding circuit depth. For the biased method, the best values for the trainable parameters were found after 29, 252, 243, and 206 epochs, while the training sessions lasted an average of 15.70, 56.59, 98.92, and 151.06 seconds, respectively.

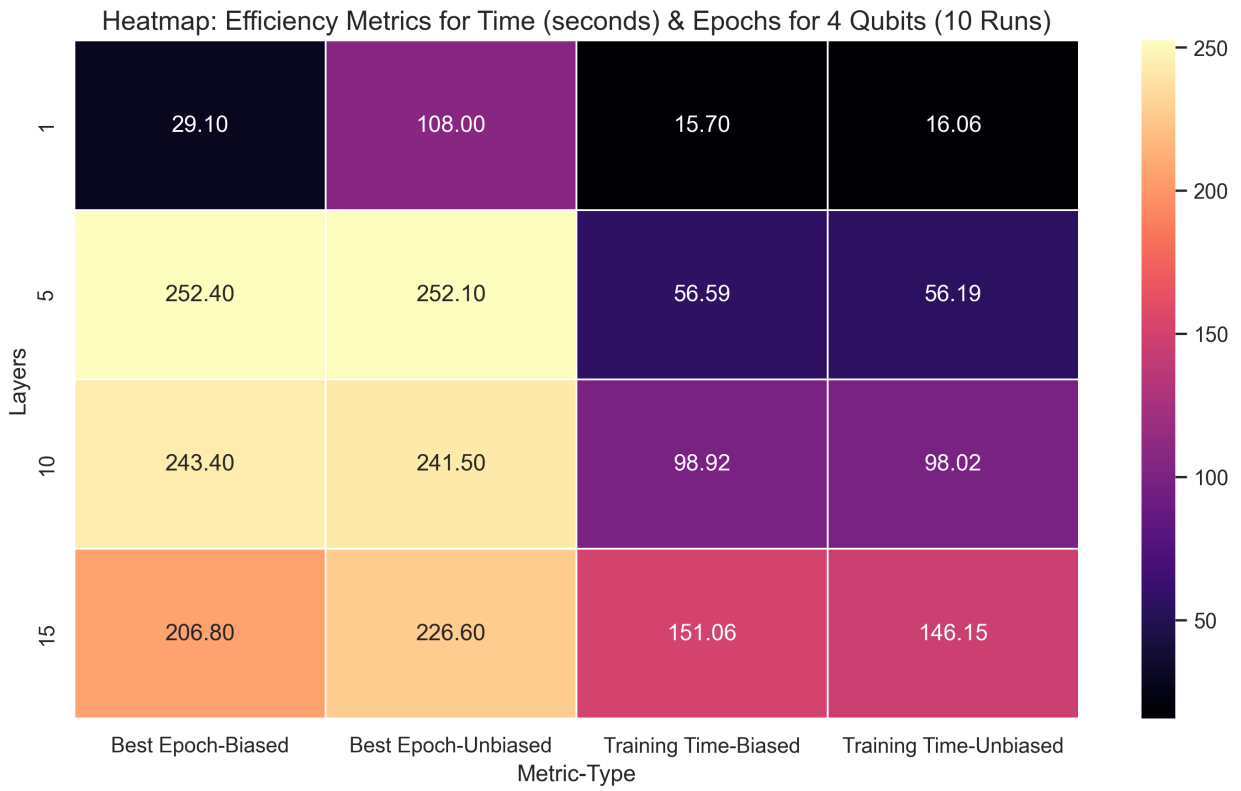

Figure 3.38: Efficiency Results for 4-qubit classifiers after 10 runs

Several important conclusions emerge from the comparison of the two methods. It is immediately clear that with a low number of layers and a limited count of trainable parameters, both approaches severely underperform. While the biased method shows a performance advantage in the 5-layer configuration, it remains considerably substandard. This outcome is expected, as the significantly larger Hilbert space of a 4-qubit system requires a higher number of parameters to be traversed efficiently. Substantial results are only achieved by increasing circuit depth; specifically, at 15 layers, the unbiased method demonstrates a slight edge over its biased counterpart. Furthermore, the unbiased version requires less training time and identifies optimal parameters at approximately the same epoch as the biased model, suggesting that the efficiency advantage previously observed for the biased approach has diminished in this larger space. Nevertheless, because the differences in key metrics are so miniscule, it remains valid to argue that the two models are effectively equivalent.

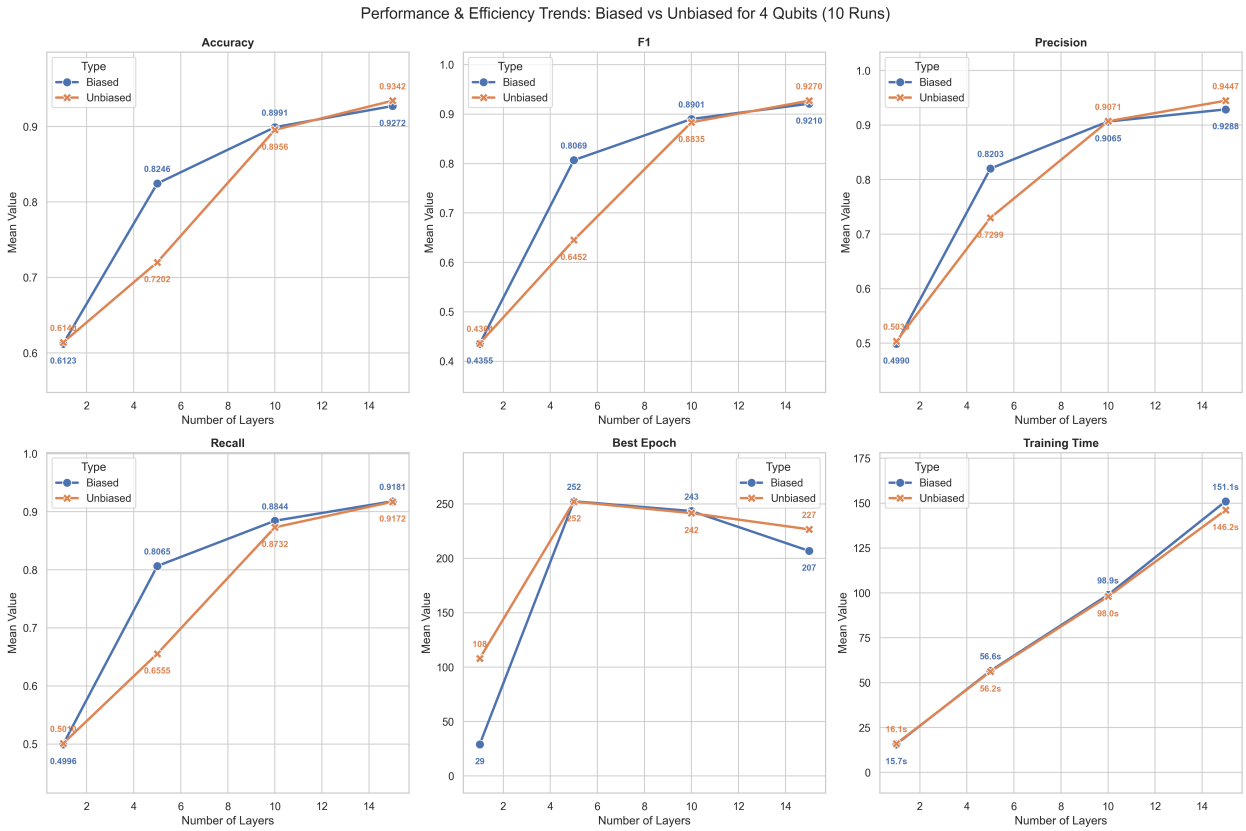

Figure 3.39: Performance and Efficiency Trends of Biased and Unbiased Classifier for 4 Qubits

When examining the top-performing models, the best unbiased classifier features a VQC with 15 layers. At the 179th epoch, this model achieves an accuracy of 96.49%, a precision of 96.72%, a recall of 95.73%, and an F1-score of 96.19%.

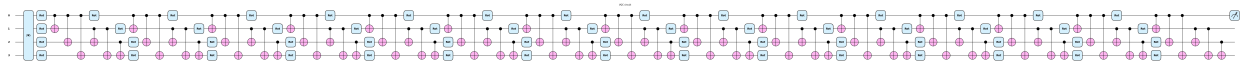

Figure 3.40: VQC of the best 4-qubit unbiased classifier

The evolution of its loss function differs slightly from previous models. Initially, the validation cost remains higher than the training cost until approximately the 160th epoch, at which point its curve crosses below the training cost before eventually reaching a plateau after the 200th epoch. While we observe some sharp increases in cost followed by similarly sharp declines, they are not as pronounced as those seen in previous cases. If anything, these fluctuations may indicate temporary periods of overfitting or underfitting, confirming that the decision to extend the training duration was well-advised.

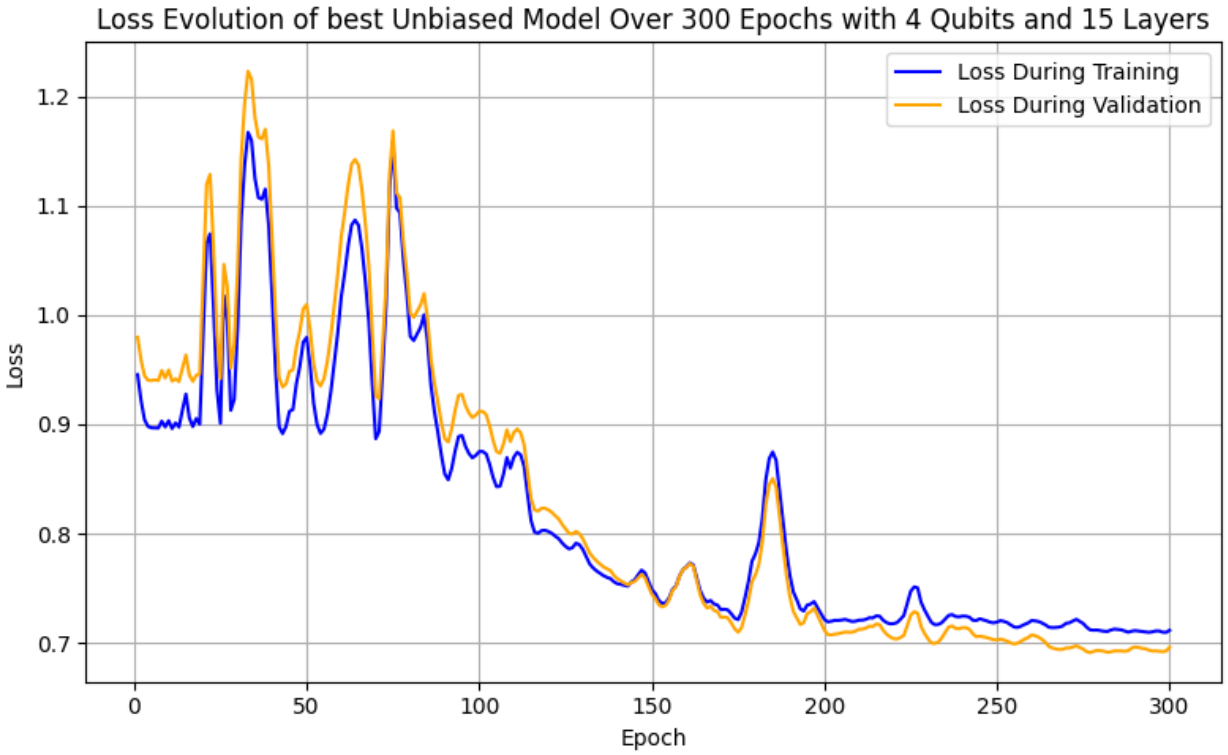

Figure 3.41: Evolution of the loss function of the best 4-qubit unbiased classifier

Observing the progression of the key metrics, we notice somewhat erratic behavior from the model. During the first 140 epochs, the classifier underperforms and fails to achieve either high accuracy or recall. However, after the 150th epoch, the performance picks up significantly, eventually reaching its peak at the 179th epoch where it scores an accuracy of 96.49%, a precision of 96.72%, a recall of 95.73%, and an F1-score of 96.19%. Generally, it appears that the unbiased classifier, with its 180 parameters, struggles to learn effectively from the data initially and only achieves strong results after 150 epochs have passed. Until that point, the scores remain relatively low, although, surprisingly, the model manages to avoid severe instances of overfitting or underfitting throughout this process.

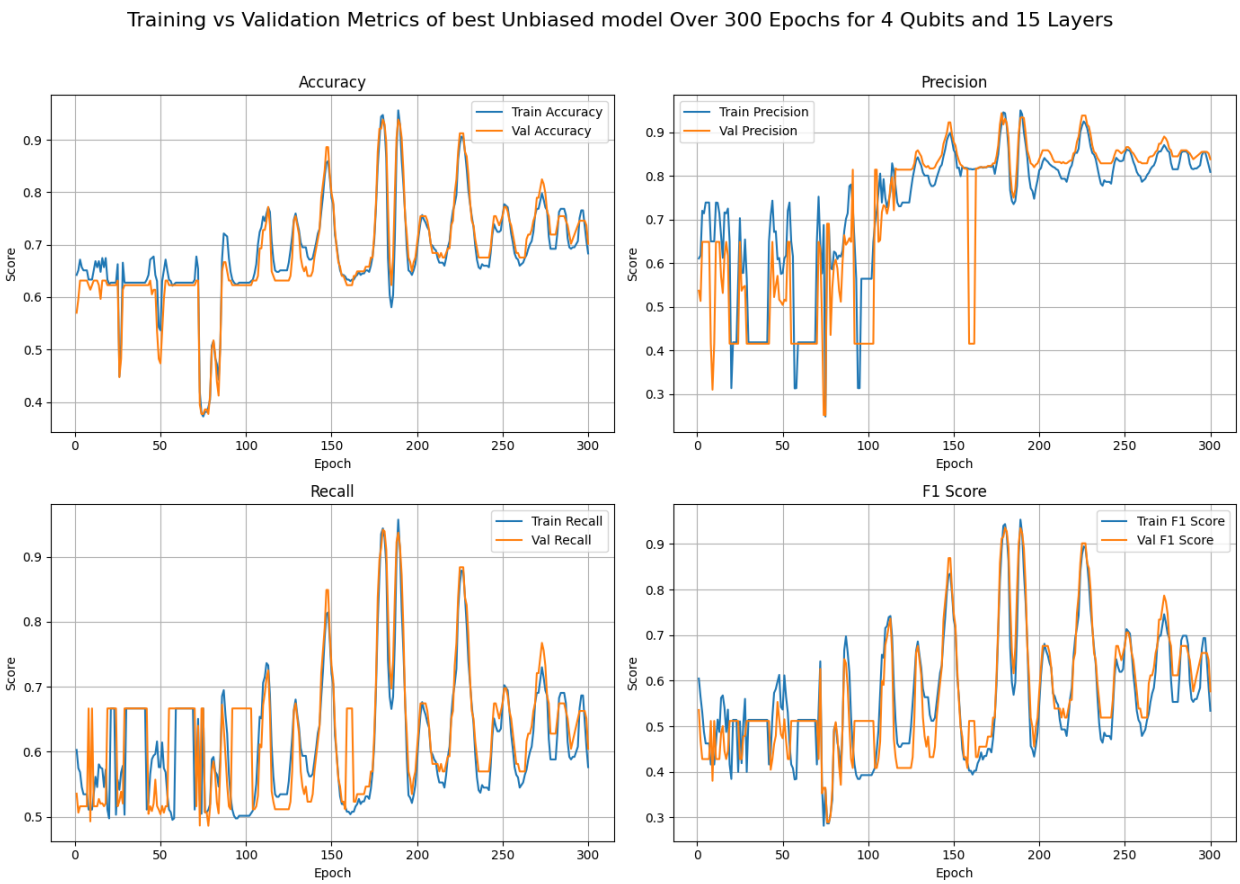

Figure 3.42: Validation and Training curves for the 4 key metrics of the best 4-qubit unbiased classifier

For the biased approach, the best-performing classifier for the 4-qubit task consists of 15 layers. After 255 epochs, this model achieves an accuracy of 95.61%, a precision of 95.51%, a recall of 95.04%, and an F1-score of 95.26%.

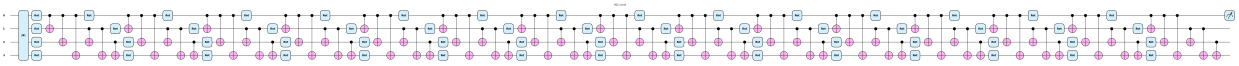

Figure 3.43: VQC of the best 4-qubit biased classifier

The loss function for the biased model begins discouragingly, with a sharp increase during the initial epochs, but it soon follows with an equally rapid decline. By the 90th epoch, the validation curve drops below the training curve, effectively alleviating concerns regarding overfitting. Unlike the unbiased case, a final fluctuation occurs toward the end of the training process after the 250th epoch; however, we successfully identified the best parameters at epoch 255, where the model achieved an accuracy of 95.61%, a precision of 95.51%, a recall of 95.04%, and an F1-score of 95.26%, just before this instability occurred.

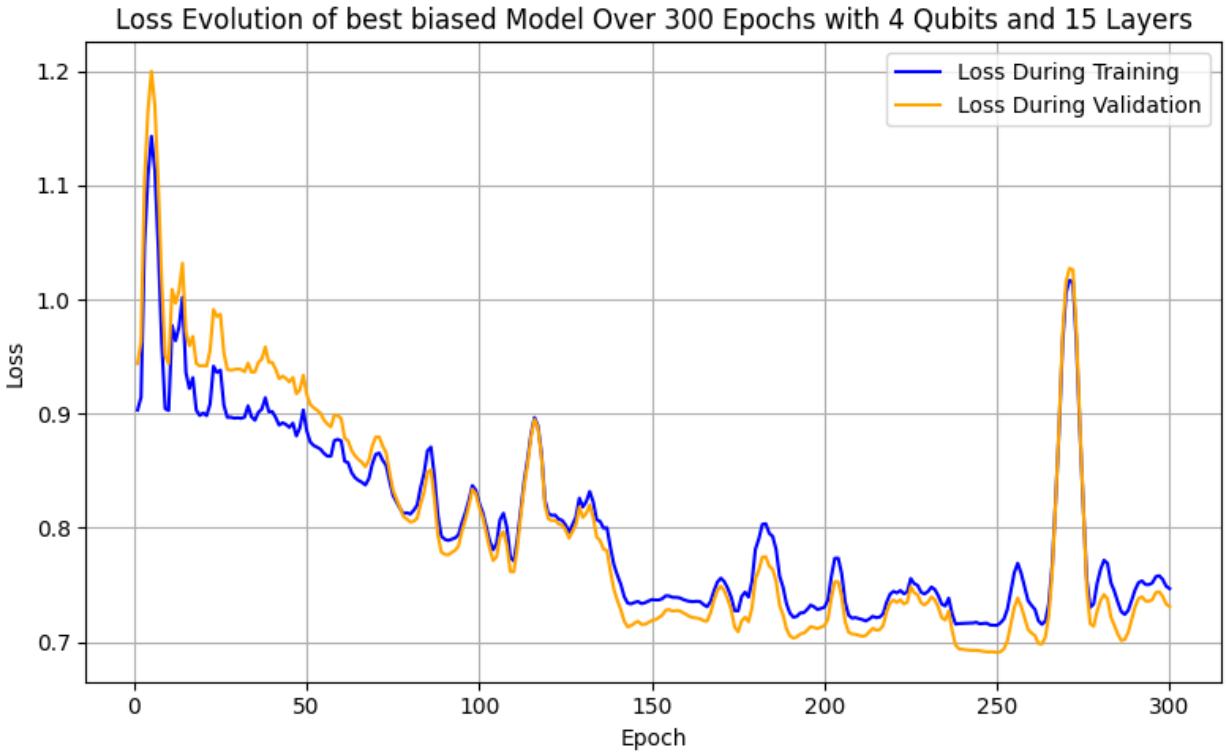

Figure 3.44: Evolution of the loss function of the best 4-qubit biased classifier

Examining the progression of the key metrics, we observe the same erratic behavior that characterized the unbiased classifier. Initially, all key metrics score poorly, followed by significant fluctuations between high and low values. It is during one of these surges that the best parameters are identified, achieving an accuracy of 95.61%, a precision of 95.51%, a recall of 95.04%, and an F1-score of 95.26%, which demonstrates that the extended training duration was pivotal to achieving high performance. Despite these constant fluctuations, the model shows minimal signs of overfitting, as the distance between the training and validation curves remains low throughout the process, even as they periodically cross one another.

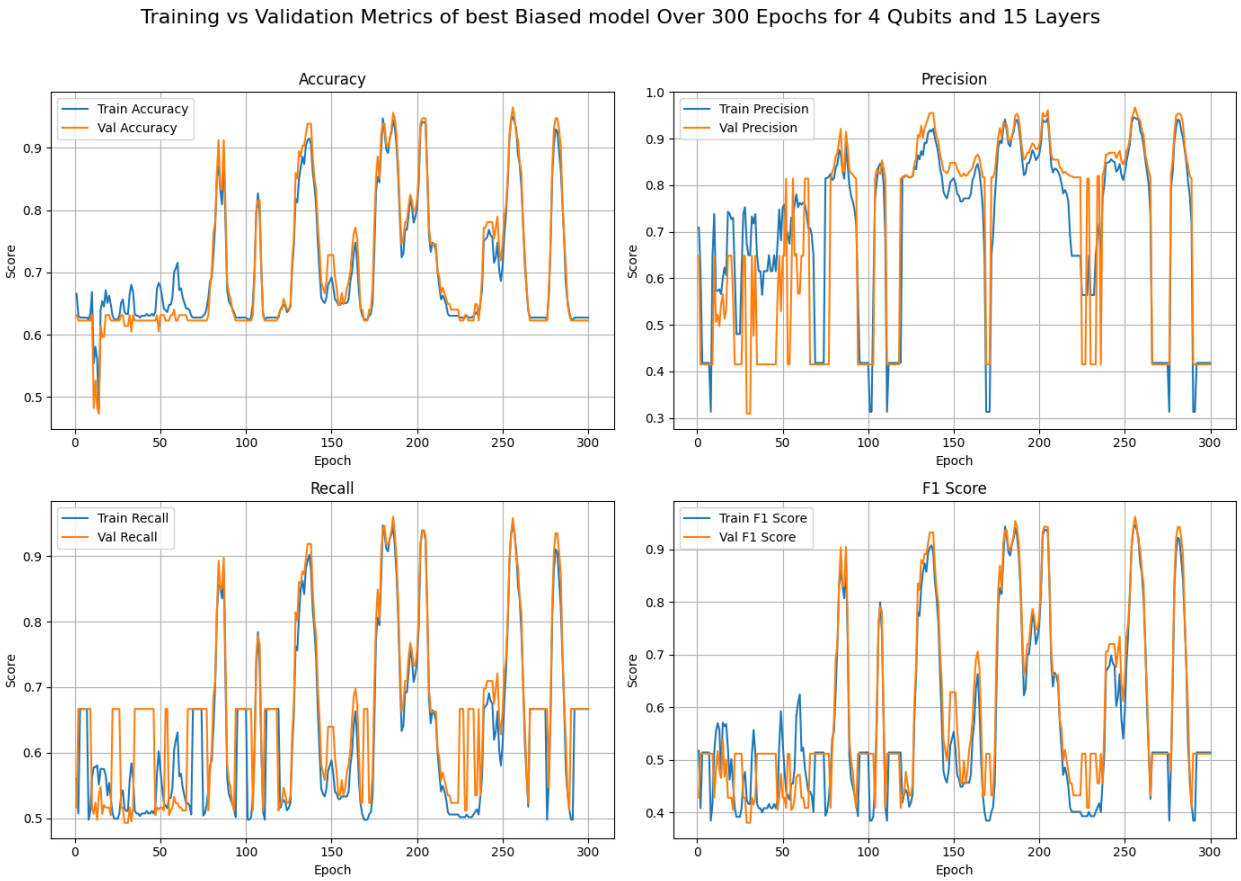

Figure 3.45: Validation and Training curves for the 4 key metrics of the best 4-qubit biased classifier

Finally, we move on to tackle the problem using a 5-qubit configuration, an approach that allows us to utilize the full set of original features without the necessity of performing dimensionality reduction.

3.5 5-qubits Results

The use of 5 qubits allows us, through the Amplitude-Efficient State Preparation routine, to encode a total of 32 features (2^5). Since we have 30 features at our disposal, we incorporate two latent dimensions by padding each normalized input vector with 0.1. Because we are not performing PCA for this approach, no further exploratory data analysis is required beyond the initial analysis conducted during our classical assessment of the problem.

Consistent with our previous methodology, the layer composition remains unchanged, consisting exclusively of Rot and $CNOT$ gates. To address the increased dimensionality of the 5-qubit problem, we employ a significantly larger number of trainable parameters by evaluating biased and unbiased classifiers at circuit depths of 1, 20, and 30 layers. Consequently, the unbiased configurations require the optimization of 15, 300, and 450 parameters, respectively, while the biased versions involve 16, 301, and 451 parame-

ters. Given the substantial increase in parameter counts, we have extended the training duration for each model to 400 epochs to ensure sufficient convergence.

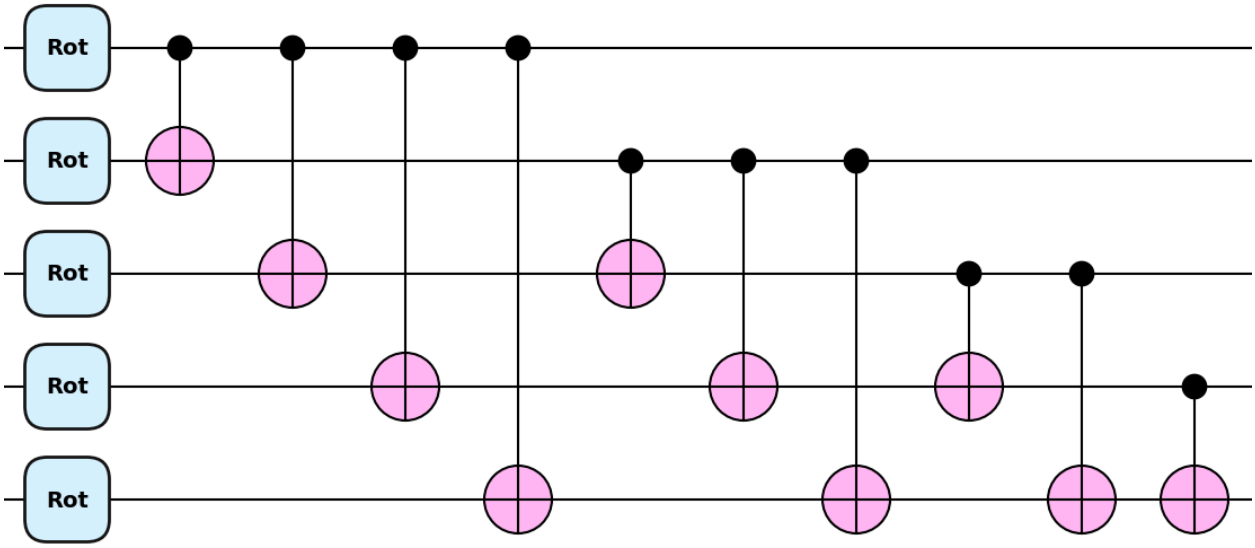

Figure 3.46: A single layer of the 5-qubit VQC

An inspection of the average scores obtained across ten runs reveals several compelling patterns. Once again, the simple single-layer classifier fails to categorize the data in a satisfactory manner; the unbiased version achieves an accuracy of 62.28%, a precision of 31.42%, a recall of 49.31%, and an F1-score of 38.38%, while the biased counterpart performs slightly better but still inadequately, scoring an accuracy of 66.67%, a precision of 70.26%, a recall of 56.70%, and an F1-score of 52.36%. It is evident that in both instances, the parameter count is insufficient to navigate the enlarged Hilbert space effectively. However, performance improves significantly as the circuit depth increases. For depths of 10, 20, and 30 layers, the unbiased classifier achieves accuracies of 82.72%, 84.56%, and 85.35%, precisions of 84.05%, 84.96%, and 85.25%, recalls of 78.63%, 81.73%, and 83.14%, and F1-scores of 80.01%, 82.67%, and 83.80%, respectively. Similarly, the biased classifier yields accuracies of 84.74%, 84.47%, and 84.74%, precisions of 85.66%, 84.80%, and 84.75%, recalls of 81.52%, 81.81%, and 82.16%, and F1-scores of 82.63%, 82.57%, and 82.99%.

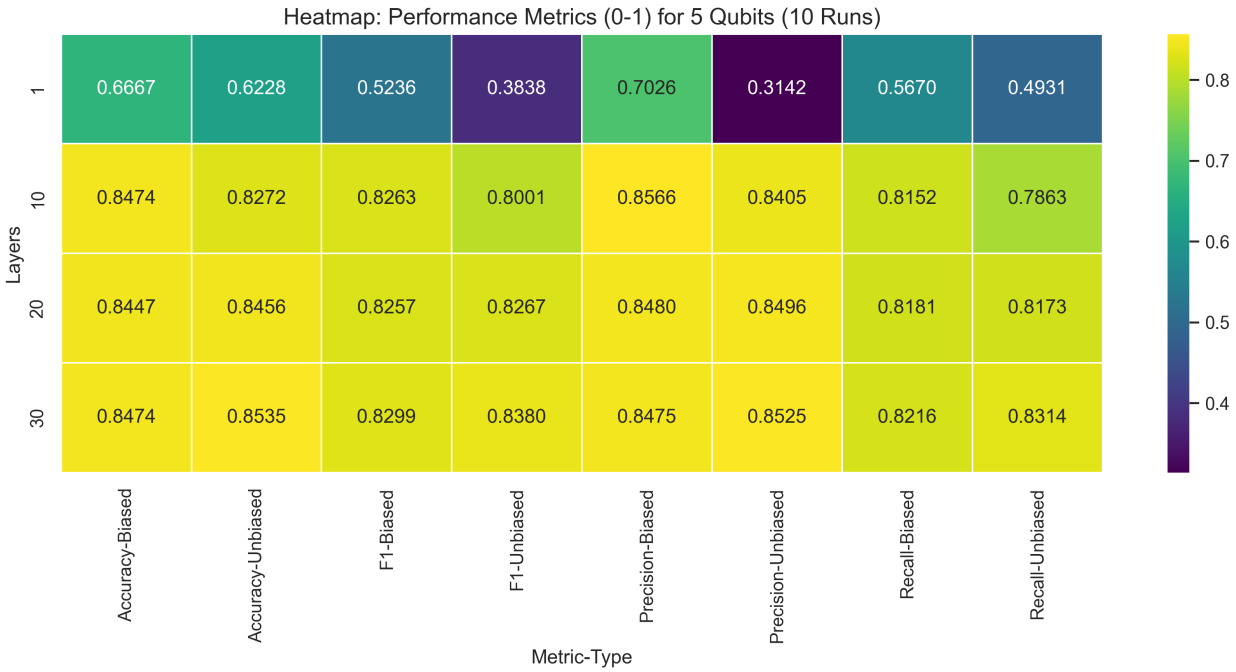

Figure 3.47: Performance Results for 5-qubit classifiers after 10 runs

In terms of efficiency, the unbiased 5-qubit classifier identified its optimal parameters after 192 epochs for 1 layer, 280 epochs for 10 layers, 214 epochs for 20 layers, and 270 epochs for 30 layers. The average training durations for these unbiased configurations were 33.77 seconds, 210.74 seconds, 390.85 seconds, and 569.58 seconds, respectively. Similarly, the biased classifier reached its best performance after 341, 255, 233, and 251 epochs, with corresponding average training times of 30.38 seconds for 1 layer, 203.97 seconds for 10 layers, 422.98 seconds for 20 layers, and 568.32 seconds for 30 layers.

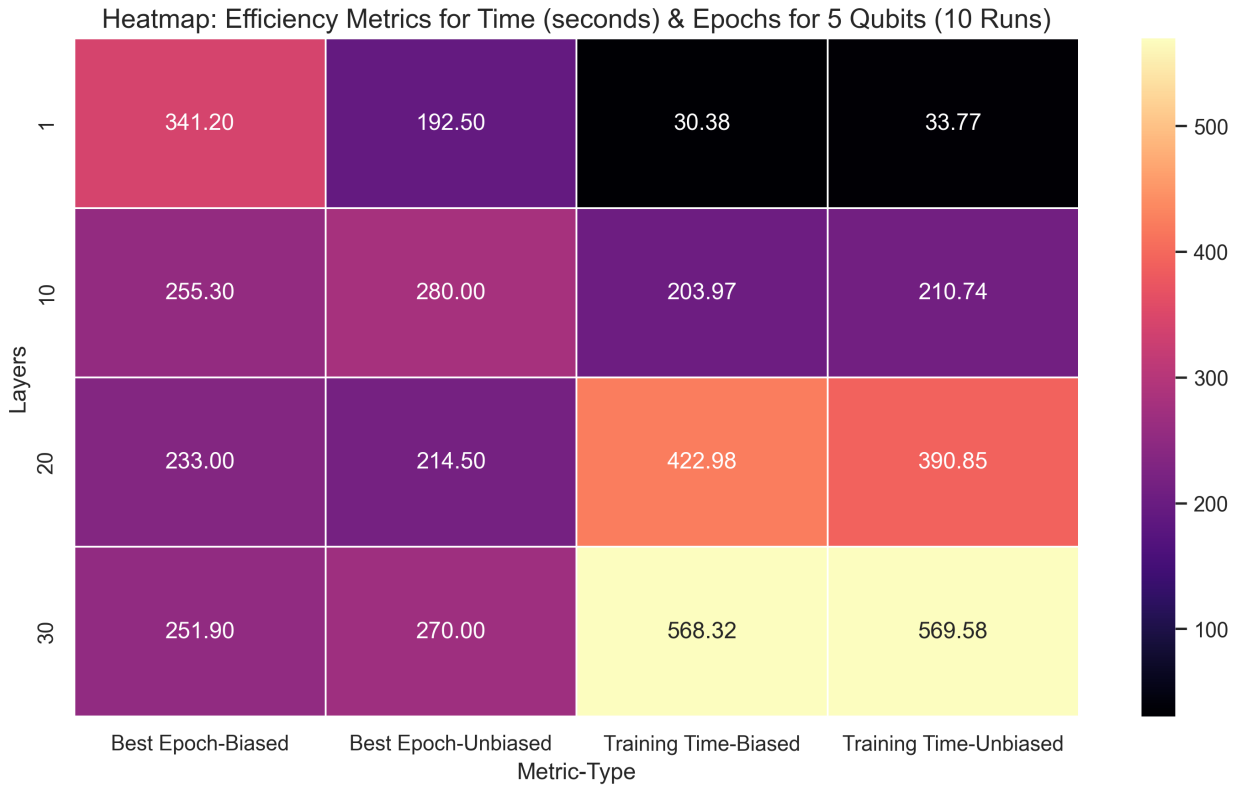

Figure 3.48: Efficiency Results for 5-qubit classifiers after 10 runs

A close comparison of the two approaches reveals several critical insights. Most notably, when utilizing only a single layer, both methods fail to generalize effectively and perform abysmally; specifically, recall and precision scores fall below 50%, leading to a consistently low F1-score. As the number of layers increases, all performance metrics improve across both methods. However, it is significant that even with 20 layers, the models are unable to surpass the 85% accuracy threshold. In fact, there is no statistically significant difference between the scores achieved at 20 layers and 30 layers. This stagnation suggests that even 450 parameters are insufficient to provide adequate coverage of the 5-qubit Hilbert space, indicating that a further increase in circuit depth or parameter count is necessary to capture the complexity of the 30-feature dataset.

In terms of efficiency, it is evident that increasing the number of layers to achieve higher scores results in a corresponding increase in training time for both methods. This trend suggests that if we were to increase the number of trainable parameters further, the practical feasibility of our approach could be compromised. Despite this potential drawback, the allocated number of epochs appears to be more than sufficient, as both models consistently identify their optimal parameter values in fewer than 300 epochs across all tested configurations.

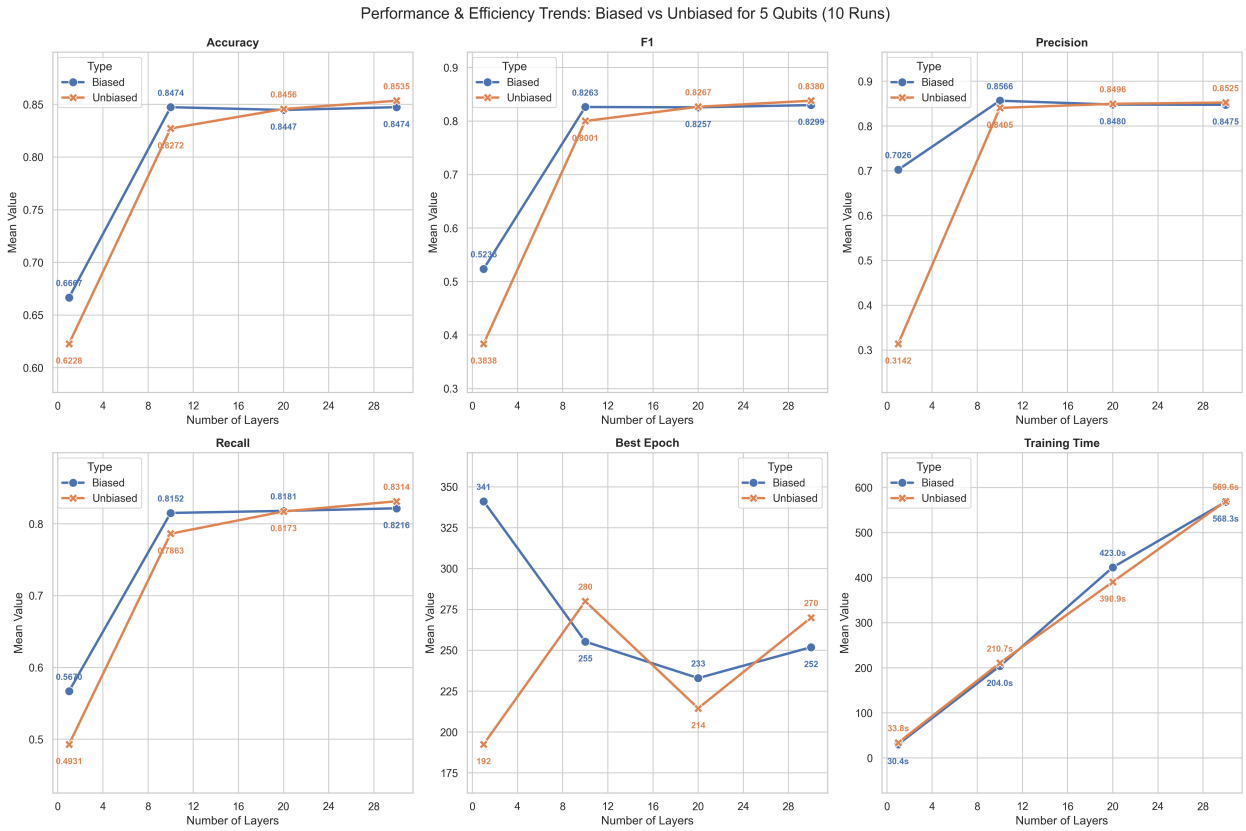

Figure 3.49: Performance and Efficiency Trends of Biased and Unbiased Classifier for 5 Qubits

Regarding the top-performing models across all iterations and layer configurations, the unbiased method identified a VQC with 20 layers that, after 321 epochs, achieved an accuracy of 88.60%, a precision of 87.44%, a recall of 88.99%, and an F1-score of 88.02%.

Figure 3.50: VQC of the best 5-qubit unbiased classifier

The evolution of the loss function in this case is more similar to our low-qubit approaches. The cost begins to decrease relatively early and, despite fluctuations that lead to some substantial spikes, it starts to plateau after the 150th epoch. It is encouraging that no signs of overfitting or underfitting are observed, as the validation and training curves remain in close proximity while consistently exhibiting the aforementioned downward tendencies.

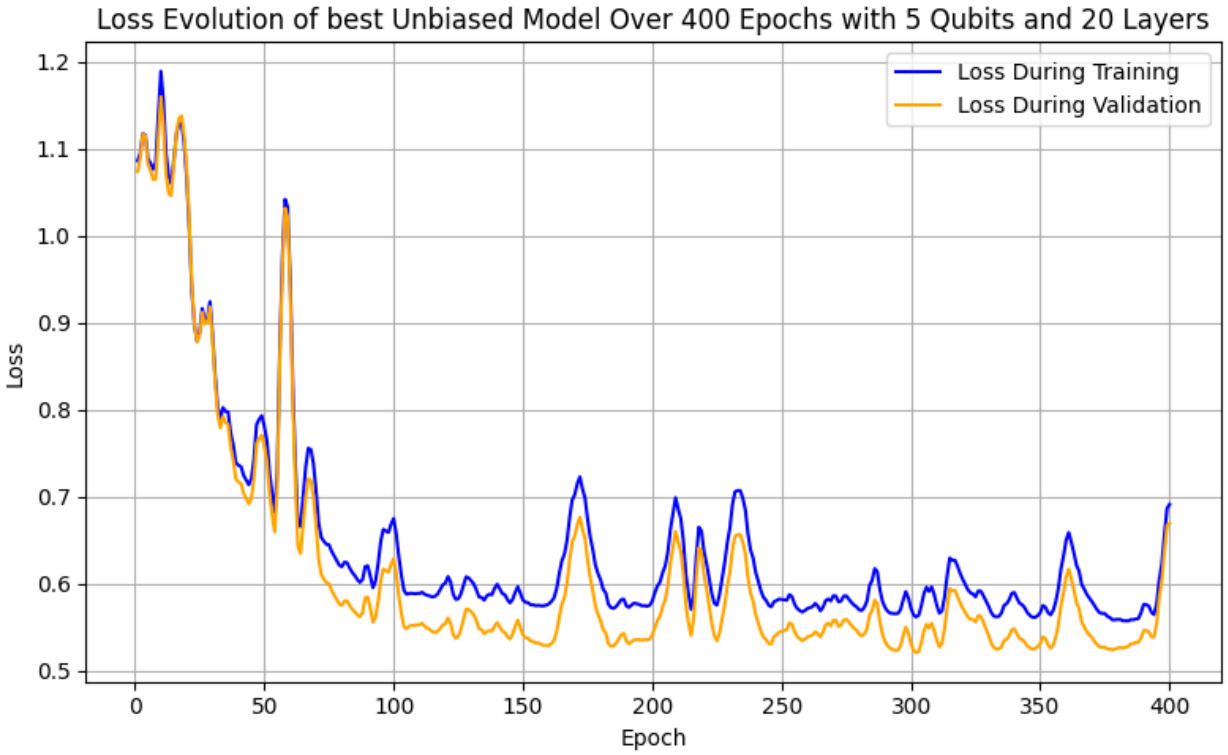

Figure 3.51: Evolution of the loss function of the best 5-qubit unbiased classifier

Observing the progression of the key metrics for this classifier, we see significant volatility with numerous fluctuations in both the training and validation curves. Interestingly, despite the initial decrease in cost, the performance metrics drop dramatically during the early epochs before rising to fluctuate between 0.7 and 0.9, failing to achieve a truly high-level performance. This behavior suggests that the model struggles to learn effectively from the data and that a higher parameter count may be required to reach more substantial results; however, since no lasting distance develops between the validation and training curves, the model appears to avoid both overfitting and underfitting throughout the process.

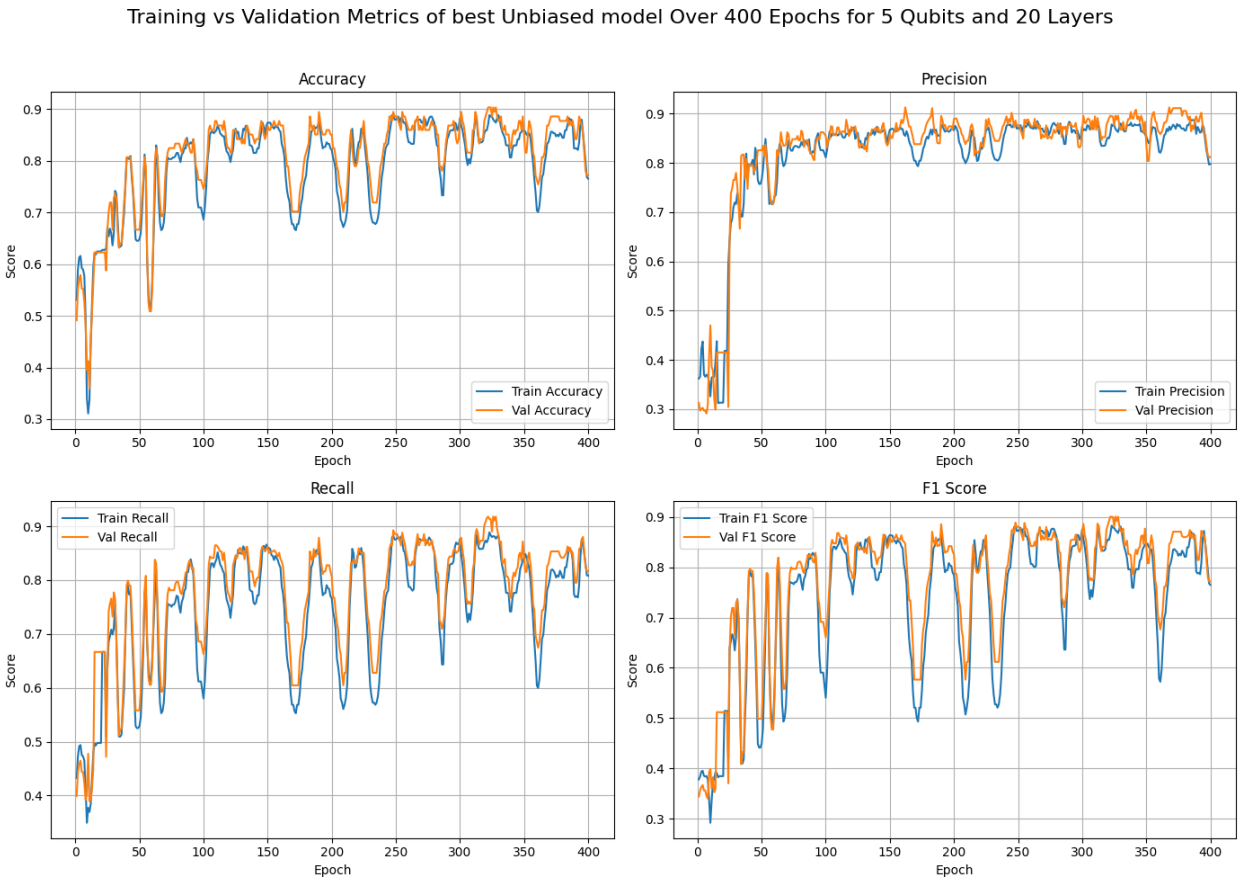

Figure 3.52: Validation and Training curves for the 4 key metrics of the best 5-qubit unbiased classifier

For the biased method, the best-performing classifier for the 5-qubit task utilizes a 10-layer VQC. After 198 epochs, this model achieves an accuracy of 88.60%, a precision of 87.64%, a recall of 88.00%, and an F1-score of 87.81%.

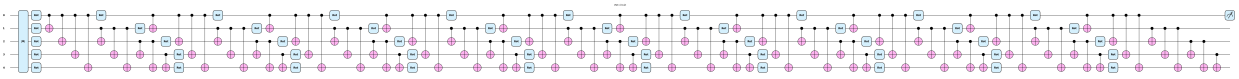

Figure 3.53: VQC of the best 5-qubit biased classifier

The evolution of the loss function for this biased model is slightly different from its unbiased counterpart, as it initially shows an increase in cost before gradually dropping until the 50th epoch. From that point, it begins to steadily decrease and eventually plateaus after the 200th epoch, a stage where no further significant increases or decreases are observed. Once again, the fact that both curves follow the same general tendencies and that the validation curve remains consistently below the training curve serves as a promising sign regarding the capabilities of the model

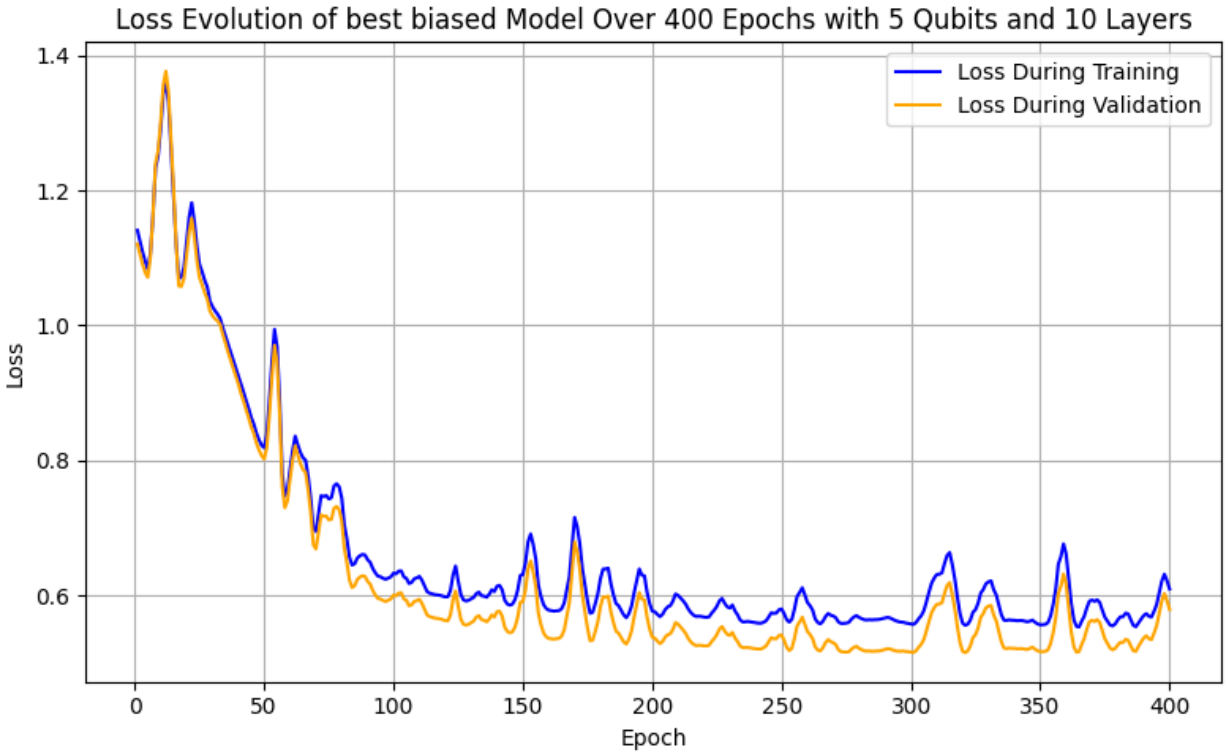

Figure 3.54: Evolution of the loss function of the best 5-qubit biased classifier

Should we observe the progression of the four key metrics for the training and validation data, it is evident that despite an initial drop in scores up until the 50th epoch, the model remains quite stable. Around the 100th epoch, it reaches relatively high scores and with the exception of some fluctuations, it never drops significantly lower. Although the model still displays some difficulty learning from the data, it appears to stabilize as the epochs progress. Most encouragingly, there is no dramatic difference in shape or distance between the training and validation curves for the majority of the training process, indicating that the model suffers from neither overfitting nor underfitting.

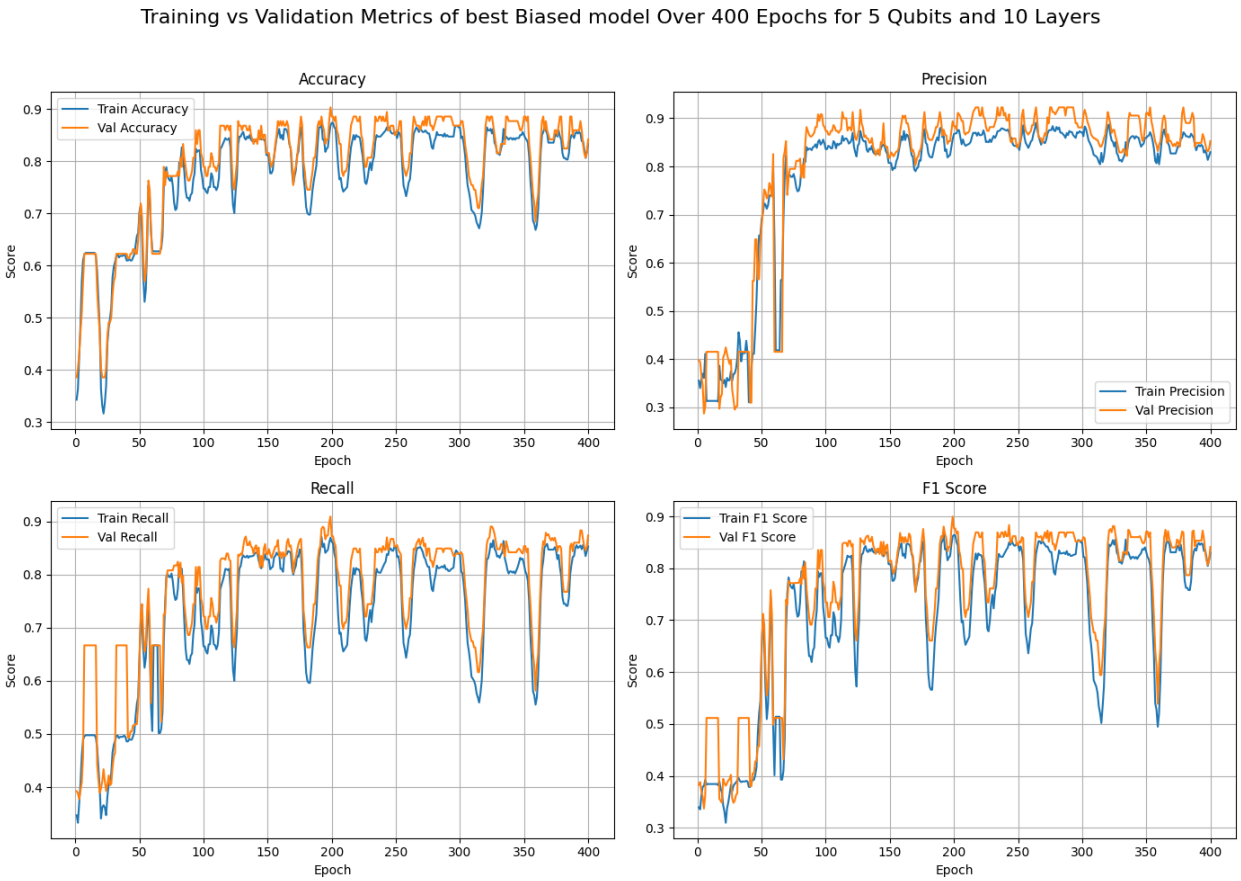

Figure 3.55: Validation and Training curves for the 4 key metrics of the best 5-qubit biased classifier

3.6 Final Discussion

In the final steps of this examination, we present a summary and comparison of the results obtained from the top-performing models for both methods across all runs and qubit configurations. This analysis focuses on the scores of the four key metrics, accuracy, precision, recall, and F1-score, while also detailing the number of layers utilized by each best performer, the total count of trainable parameters, and the number of epochs required to reach the reported scores. Finally, we conduct a brief comparison between the average performance of the two quantum methods and the Logistic Regression classifier, which was determined to be the optimal classical model for solving this tumor classification problem.

Starting with the unbiased case, we observe that the top-performing models exhibit an upward trend until reaching the 4-qubit configuration. Up to that point, all scores remained well above 90%, indicating that the models were robust and efficient in their classification tasks. In the 5-qubit case, however, performance dropped significantly, and the model was unable to score above 90% in any metric. This outcome was expected, as we have already noted that a considerably larger number of parameters is required to tackle the

problem when it consists of 30 features. We remain confident that if we were to increase the number of trainable parameters further, the scores of the key metrics would improve.

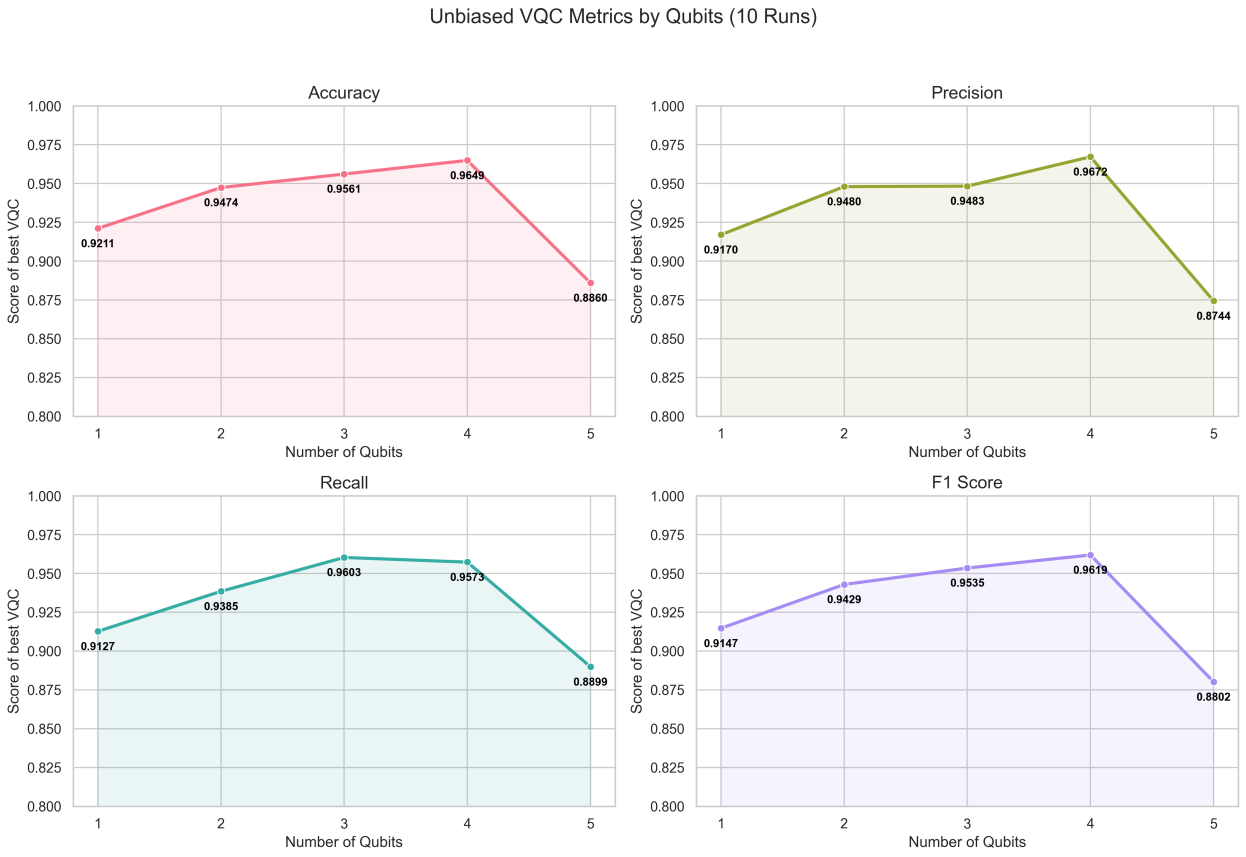

Figure 3.56: Performance of best unbiased classifier per number of qubits across all runs

In the biased case, we observe a similar pattern where the majority of the configurations maintain consistently high scores, only to drop below the 90% threshold for the 5-qubit case. It appears that the presence of the classical bias parameter is insufficient to push the model beyond this 90% mark as initially hoped, which suggests that the introduction of more layers and a higher number of trainable parameters is necessary to achieve better performance.

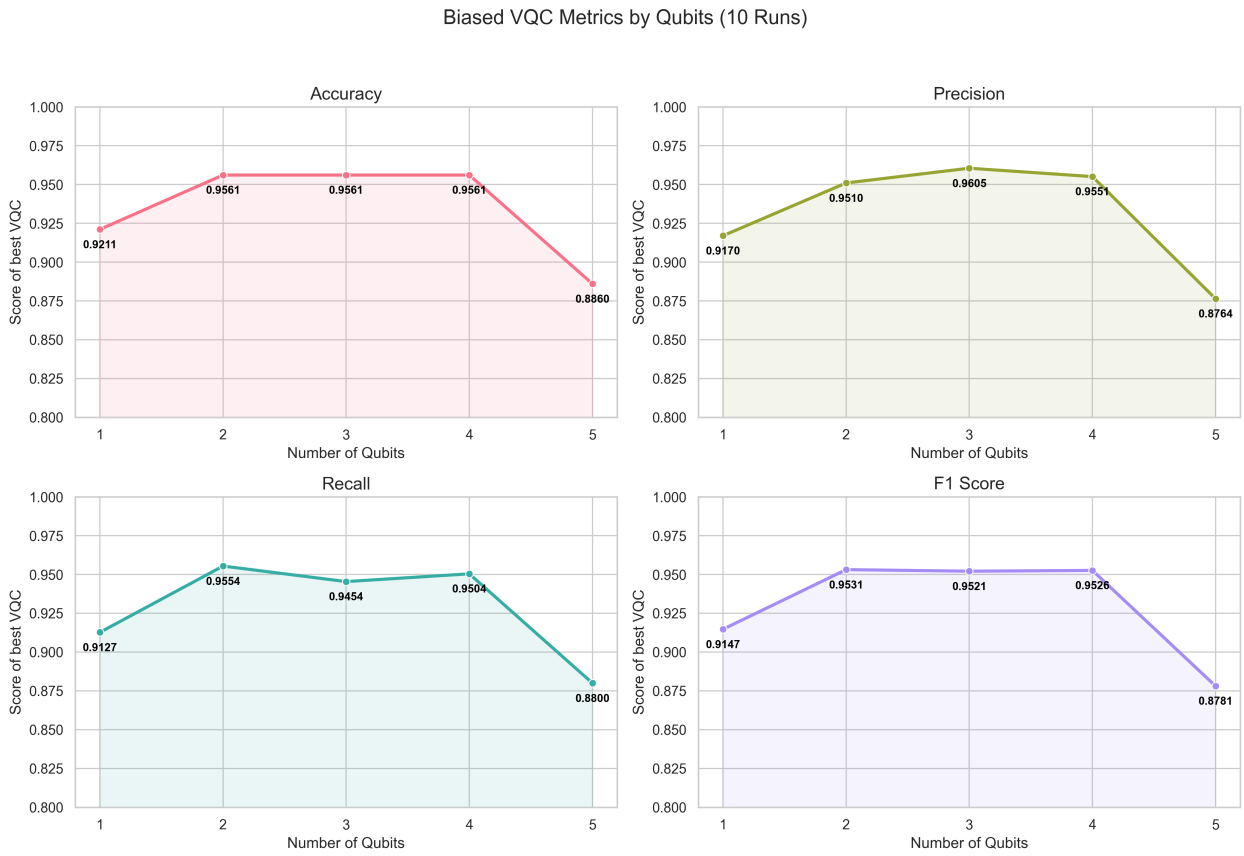

Figure 3.57: Performance of best biased classifier per number of qubits across all runs

Regarding the circuit depth for each method's best-performing model, we can observe that with the exception of the 3-qubit and 5-qubit configurations, the two methods require a similar number of layers. Notably, the biased method requires more layers for the simpler 3-qubit case where the number of trainable parameters per layer is lower, whereas the unbiased method requires more layers for the 5-qubit case where each layer is more costly in terms of parameter count.

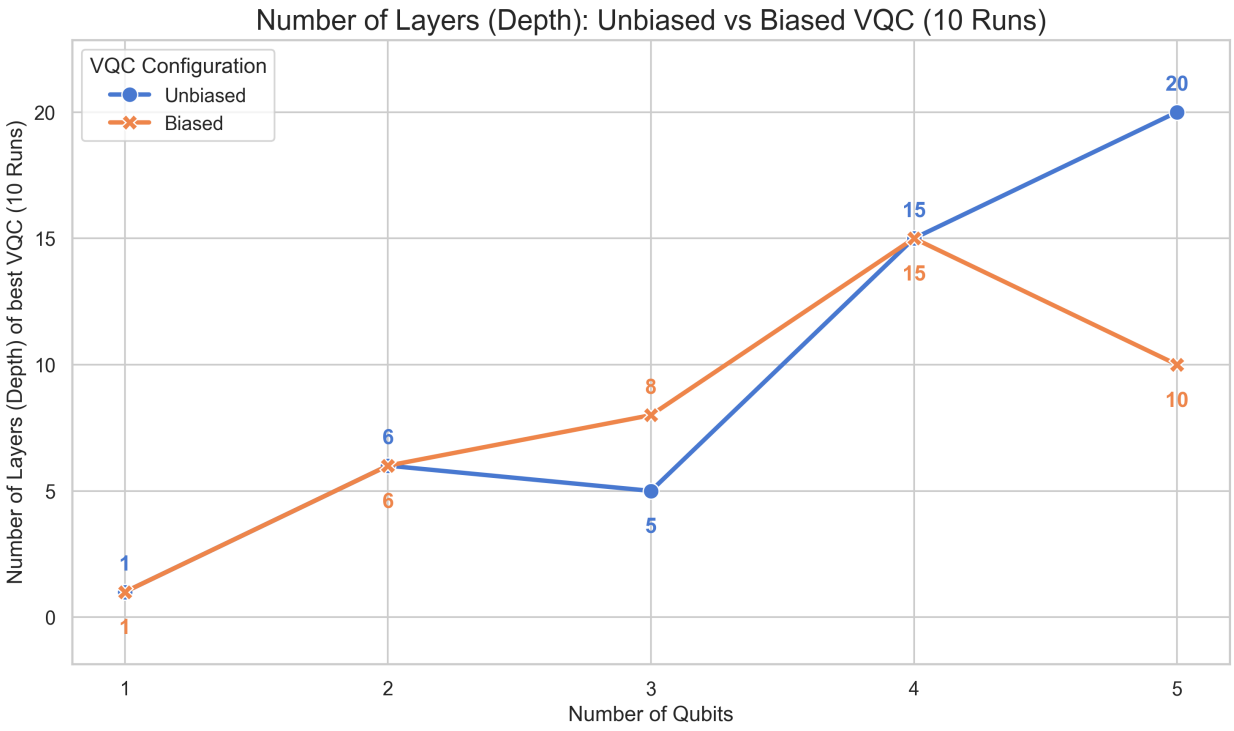

Figure 3.58: Circuit depth in respect to the number of qubits for biased and unbiased classifiers

The aforementioned trend becomes clearer when focusing on the number of parameters each top-performing model must train. For the majority of the configurations, the primary difference between the two methods is the single classical bias parameter introduced by the biased method. The only two instances where this does not hold true are the 3-qubit and 5-qubit cases. In the 3-qubit scenario, the biased method trains an additional 23 parameters; in contrast, the unbiased method must optimize an extra 149 parameters in the 5-qubit case. Consequently, the introduction of the classical parameter appears to have been beneficial in terms of efficiency, especially considering that each classifier in the 5-qubit paradigm requires a considerable amount of training time.

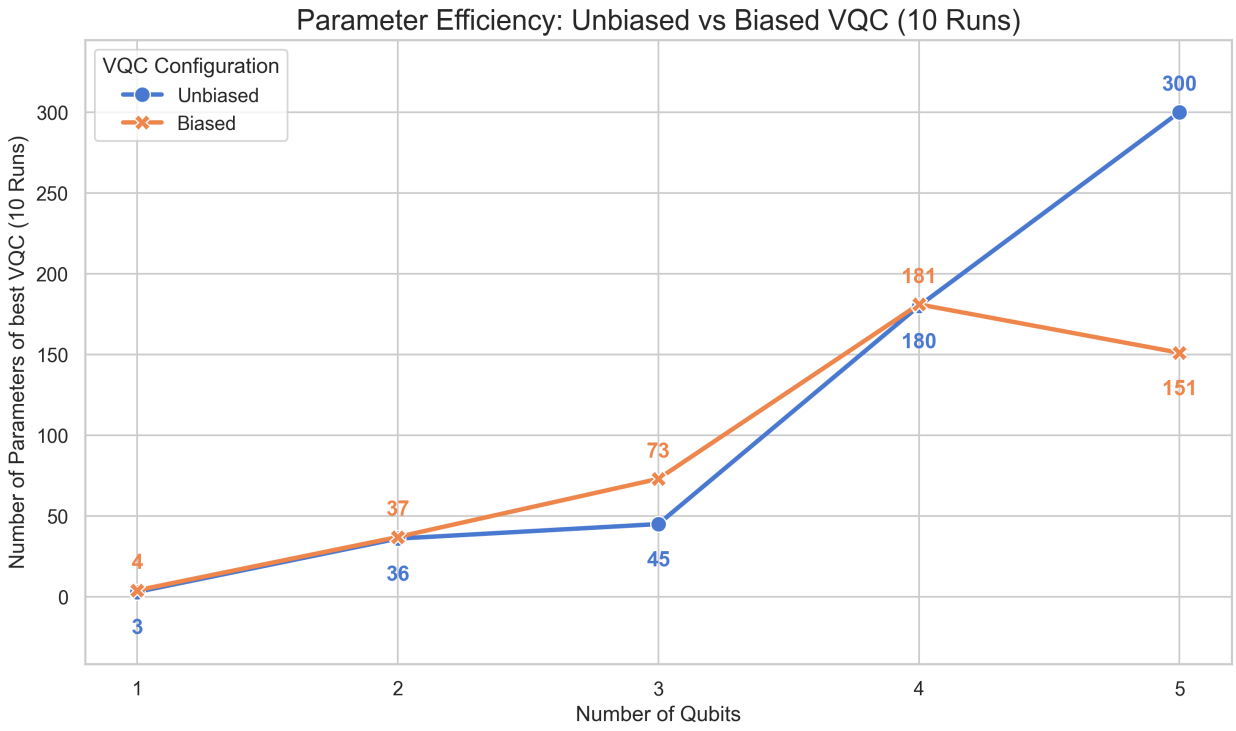

Figure 3.59: Number of trainable parameters in respect to the number of qubits for biased and unbiased classifiers

In terms of epochs, it is evident that the biased models reached their optimal parameter values faster than the unbiased models in almost every instance, with the 4-qubit configuration being the only exception. This provides further evidence that the introduction of the classical bias parameter effectively benefits the performance and convergence of our quantum classifier.

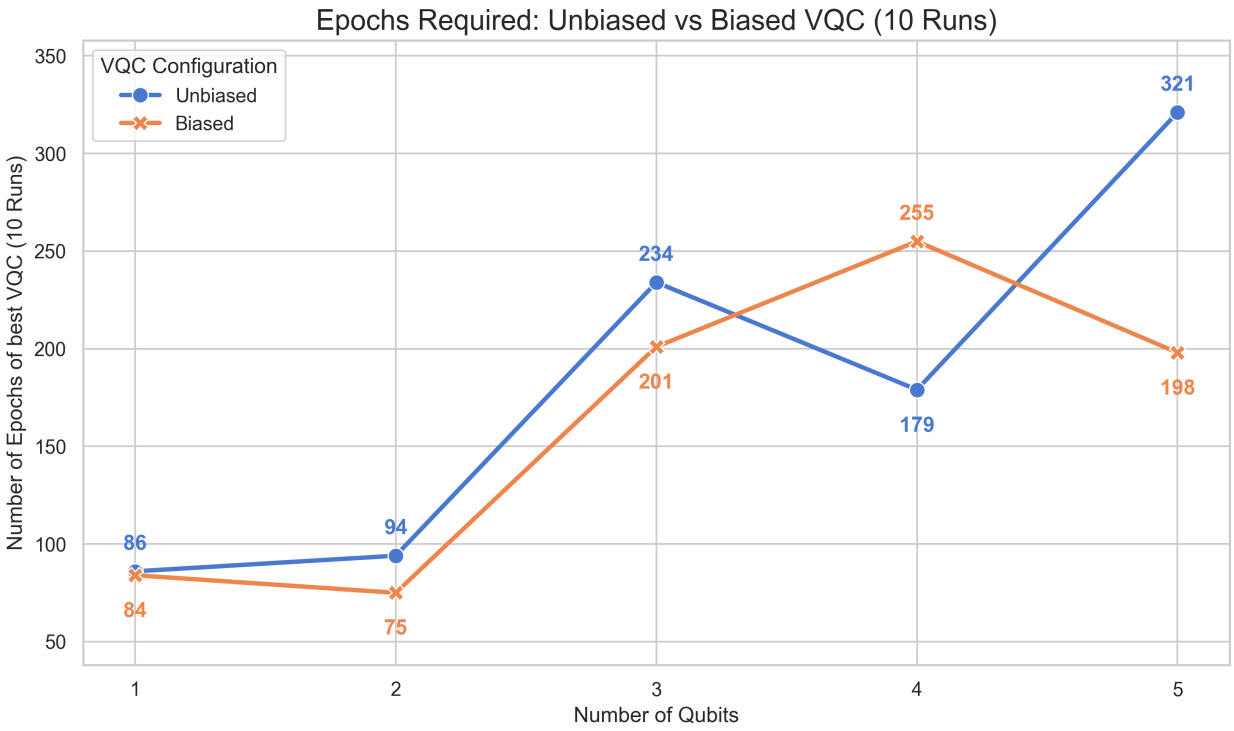

Figure 3.60: Number of epochs required to reach the best results in respect to the number of qubits for biased and unbiased classifiers

To ensure the final comparison between the quantum methods and Logistic Regression is equitable, an evaluation process similar to that of the quantum approaches was adopted. The classical method was trained and tested using the same datasets as the quantum models, with the process repeated across ten runs and various hyperparameter values for the Logistic Regression model. Although this approach is less rigorous than the exhaustive repeated nested cross-validation technique employed previously, it is appropriate here as the primary objective is to provide a fair and direct comparison between the models.

For the quantum methods, we report only the highest average results across all layers for each configuration and method. For instance, in the 5-qubit case, we consider the results of both the unbiased and biased classifiers comprised of 30 layers, and we apply this same logic to the remaining configurations. In essence, we set aside the number of parameters required to achieve these results and focus exclusively on the best average performance attained by each model.

With that in mind, we can see that our quantum approach demonstrates clear superiority only in the 1-qubit configuration, where the dataset consists of only 2 features. In this instance, the biased classifier clearly outperforms the Logistic Regression model across all key metrics while performing slightly better than its unbiased counterpart. In all other scenarios, the Logistic Regression model remains superior in terms of accuracy. Interestingly, it is not always more effective than its quantum counterparts regarding recall and maintains a similar performance in terms of F1-score for the most part. While the

dimensionality remains relatively low, the difference between the classical and quantum approaches is not particularly severe. However, this shifts significantly when addressing the complete original problem comprising 30 features; in this case, the quantum methods are not comparable to the classical model, as they consistently fail to surpass the 85% threshold across most performance metrics.

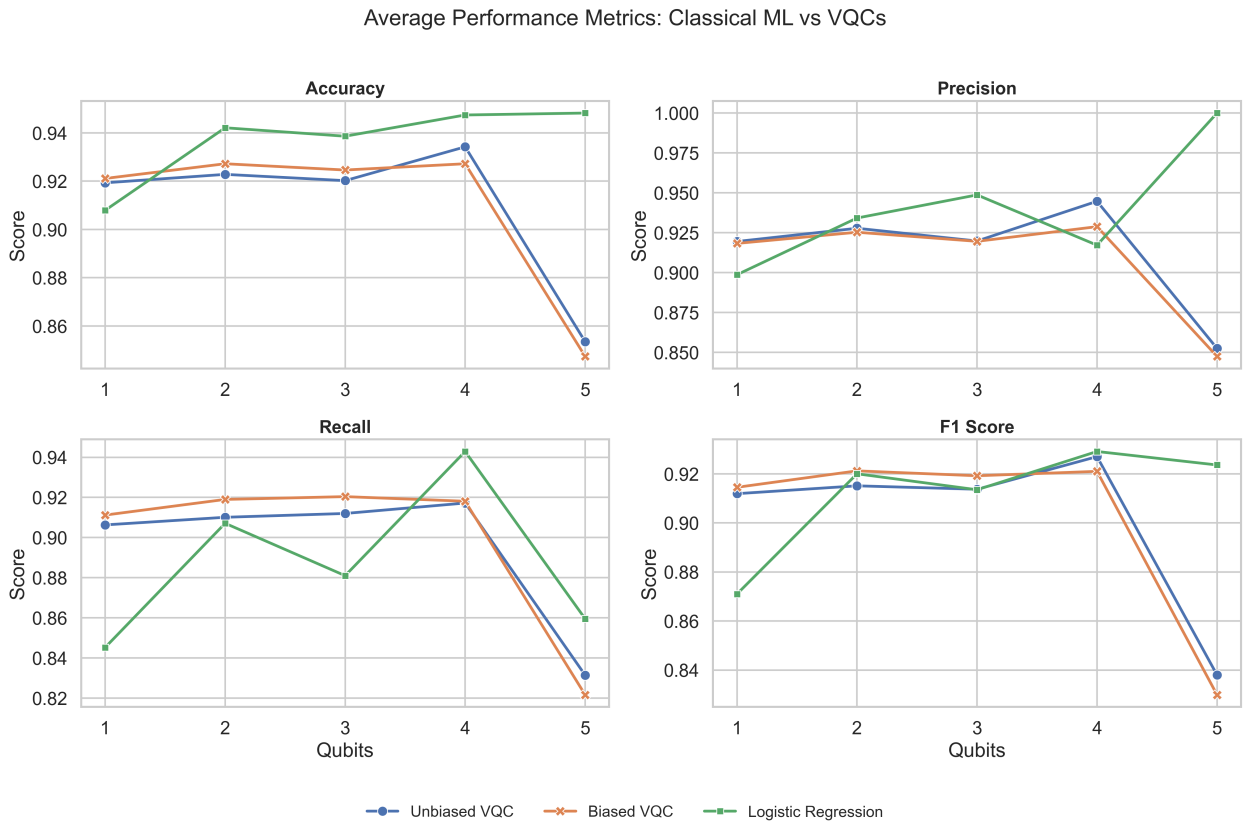

Figure 3.61: Average performance of the Quantum Models and the classical Logistic Regression model across 10 different runs for all the different number of qubits

This LaTeX example is a template showing how to format postgraduate theses. This sample focuses on theses written in English, where some pages and titles are changed from their Greek counterparts. Make sure you pass the *en* option in the `\documentclass` on the top of the .tex file.

In order to make the gray bibliography uniform throughout the library—including postgraduate thesis—this document class uses formatting enforced by the Department of Informatics & Telecommunication. Formatting details are as follows:

3.6.1 Page size

Page size shall be **A4**.

3.6.2 Printing pages

Page margin sizes, and header and footer placement depend on whether printing will be two-sided, like in a standard book, or one-sided.

3.6.2.1 Two-sided printing

This document class assumes two-sided printing by default. On odd pages, which are on the right of the book, the right margins are narrower than the left margins. On even pages, on the left side of the book, the reverse is true: left margins are narrower than the right.

The header contains the thesis title, and is placed on the right of odd right-facing pages, and on the left of even left-facing pages. The footer contains the page number on the center of all pages.

3.6.3 Dates

The dates in the pages, such as the month and year, refer to the examination date.

3.6.4 Cover and front matter pages

Formatted like in this sample, centered unless noted otherwise, in order:

1. Emblem of Athena: top-center
2. Name of university (NATIONAL AND KAPODISTRIAN UNIVERSITY OF ATHENS): Arial, bold, uppercase, 14pt.
3. Name of school (SCHOOL OF SCIENCES): Arial, bold, uppercase, 12pt.
4. Name of department (DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS): Arial, bold, uppercase, 12pt.
5. Name of postgraduate program: Arial, bold, uppercase, 12pt.
6. Type of thesis (MSc THESIS): Arial, bold, uppercase (except the ’c’ in ”MSc”), 12pt.
7. Title of thesis: Arial, bold, 16pt.
8. First name, Middle initial, and Last name of author: Arial, bold, 12pt.
9. Left-aligned Supervisor (or Supervisors if more than one): First and Last name: Arial, bold, 12pt.; Supervisor’s title: Arial, 12pt.
   If there are non-staff supervisors, they are placed right after staff supervisors with their appropriate titles.

10. Place where thesis was completed (ATHENS): Arial, bold, uppercase, 12pt.
11. Month and year of completion: Arial, bold, uppercase, 12pt.
12. Line spacing: 1pt.
13. Page numbering starts at 1 from the cover page, but not printed on any of the cover or front matter pages.
14. The back of those pages shall remain blank.

3.6.5 2nd front matter page (Examination committee page)

Formatted like in this sample, center-aligned unless stated otherwise, in order:

1. Type of thesis (MSc THESIS): Arial, bold, uppercase (except the 'c' in "MSc"), 12pt.
2. Title: Arial, 12pt.
3. First & last name of author: Arial, bold, 12pt.
4. Author's serial number: Arial, uppercase, 12pt.
5. Left-aligned: Supervisor(s) and 3-member examination committee: Arial, bold, uppsercase, 12pt.
   Both sections contain:
   - Name of committee member: Arial, bold, 12pt.
   - Title of committee member: Arial, 12pt.
   - Committee signatures
6. Examination Date: Arial, bold, 12pt.

Their reverse pages are left blank.

3.6.6 Abstract

Two abstracts, one in English and one in Greek, follow. The abstract contains the purpose and scope of this thesis, methodology, the experiments and result thereof.

The abstract text is followed by the subject area and up to 5 keywords, in English and Greek after their respective abstracts. Each abstract plus subject area and keyword definition can span no more than 2 pages. Even-numbered pages are left blank if abstracts span only 1 page.

3.6.7 Other front matter pages

The rest of the front matter pages are as follows:

1. Inscription. Optional, can be included by passing *inscr* in the `\documentclass` options. Reverse page is left blank.
2. Acknowledgements. Optional, can be included by passing *ack* in the `\documentclass` options. Reverse page is left blank.
3. Table of contents. Mandatory, contains the chapters, sections and any subdivisions, as well as a list of figures and a list of tables, if any.
4. Preface. Optional, can be included by passing *preface* in the `\documentclass` options.

3.6.8 Page numbering

Page numbering starts at 1 from the front cover, the English title page in this case, without printing the numbers. When printing on both sides of the sheet (two-sided by default), any blank pages are included towards the page count, but do not include any headers or footers including page numbers. If printing on one side of the page (done by including the *oneside* option in the `\documentclass`), only the pages on the right are printed and any blank pages (including those on the left side) are not included towards the page count. Numbering always finishes on the last printed page.

On two-sided theses, numbers are printed on the center of each footer. On one-sided theses, numbers are printed on the right side of each footer.

3.6.9 Page formatting

Pages are formatted as in this sample:

- **Margins:** 2cm on all sides, plus a 0.5cm gutter on the center of the book where the pages are bound (i.e, on the left side for printed odd pages and on the right for printed even pages).
- **Header:** 1.25cm from the top of the page. Contains the title of the thesis, on the right side for odd pages, on the left for even pages, if printed two-side. Appears only in the main matter, that is not in the cover pages or the front matter (abstract, inscription, table of contents, ...), and not in blank pages.
- **Footer:** 1.25cm from the bottom of the page. Contains the name(s) of the author(s)
  - on the right side for odd pages, on the left for even pages, if printed two-side.

  - on the left side of all pages if printed one-side.

  Appears only in the main matter, that is not in the cover pages or the front matter (abstract, inscription, table of contents, ...), and not in blank pages.

- **Page number:** included in the footer
  - at the center if printed two-side.
  - on the right if printed one-side.
- **Paragraph formatting:**
  - **Justification:** Justified, each non paragraph-breaking line covers the entire line width spaced as necessary.
  - **Paragraph spacing:** 0pt for the first, 6pt between all paragraphs.
  - **Line spacing:** 1 line.
- **Font:** Arial, 12pt, Normal or Regular style
- **Chapter numbering:** Arial, bold, 12pt or 14 pt
- **Chapter title:** Arial, bold, uppercase, 14pt., centered
- **Section and further division title:** Arial, bold, 12pt.
- **Figures:** Each figure must be uniquely numbered either by chapter or by the whole document. They must also include a caption below them, as in this sample.
- **Tables:** Each table must be uniquely numbered either by chapter or by the whole document. They must also include a caption above them, as in this sample.

3.7 Sample section

The section title will appear in the Table of Contents.

3.7.1 Sample Subsection

The subsection title will appear in the Table of Contents.

3.7.1.1 Sample subsubsection

The subsubsection title will appear in the Table of Contents.

Sample paragraph The paragraph title will not appear in the Table of Contents.

Sample subparagraph The subparagraph title will not appear in the Table of Contents.

4. A BRIEF INTRODUCTION TO LaTeX

LaTeX is a typesetting system. The authors compose a **.tex** file which is then compiled to produce a document, nowadays a PDF file. This chapter will give a brief, non-exhaustive introduction to the system.

4.1 Installing LaTeX

If you use macOS or Linux as your operating system, chances are LaTeX is already included. Windows does not include LaTeX out-of-the-box. If not, just install one of the following:

- TeX Live (https://www.tug.org/texlive/). Contains a comprehensive LaTeX installation with most packages included. Thus, it also requires a few GB worth of space in a typical installation. Supports most if not all packages used in this sample.
- MiKTeX (https://miktex.org/) is a more minimalist installation. Typical installations only include the essential packages, with the option of downloading more, either from the console or automatically when compiling. This document uses packages that are not included in the typical MiKTeX installation, but it will offer you to install them upon first compilation.

Both of these are cross-platform, they can install under Windows, macOS, or Linux.

4.2 Editing

TeX files can be modified by any plain text processor program, such as Notepad. Compilation to PDF is traditionally done via the terminal. Nonetheless, it is recommended to use a dedicated TeX IDE, a program designed to edit, compile and view the resulting PDF all in one program.

Some IDEs one could use to edit and compile TeX files are:

- **TeXworks** (https://tug.org/texworks/). Very basic functionality. This is bundled with both TeX Live and MiKTeX, so by installing any of those you don't have to install this program as well.
- **Texmaker** (https://www.xm1math.net/texmaker/). More advanced functionality. Includes document structure, symbol insertion, and other LaTeX commands that can be added to the document with the push of a button.
- **TeXstudio** (https://www.texstudio.org/). Forked from Texmaker. Functions almost the same as Texmaker.

- **Kile** (https://kile.sourceforge.io/). Built by KDE.

4.3 Compiling

Before we begin with the commands, I should mention that this project—either in English or Greek—uses the XeLaTeX engine to compile. This is because while LaTeX uses the ASCII character set to typeset, XeLaTeX uses Unicode, which includes a lot more characters than ASCII.

The compilation recipe will typically consist of:

1. Running xelatex
2. Running bibtex to compile the references
3. Running xelatex
4. Running xelatex

It appears that *xelatex* runs a total of 3 times, once before then twice after *bibtex*. This is to ensure all `\label`s have been scanned and paired with their `\ref`.

4.4 Formatting

4.4.1 Basics

Take at look at the following excerpt:

```
This is the first paragraph.
This is the next line, but is still part of the first paragraph.

This is the second paragraph. Notice the blank line between the two.
% This is a comment. Comments are not rendered in the final file.
% Comments begin with the percent symbol(%)
% and extend to the end of the line.
```

In LaTeX, it's rendered like this:

This is the first paragraph. This is the next line, but is still part of the first paragraph.

This is the second paragraph. Notice the blank line between the two.

As we can see, if we type something onto the next line it will remain part of that paragraph. But if we leave one line blank between texts, the next part of the text will appear on a new paragraph.

You might have also noticed that the lines starting with '%' did not print. That's because the % symbol works like the // in C-inspired languages: it begins a comment until the next line. "But wait", you might ask, "how did you type the '%' just now"? Simple: if you type `\%` in LaTeX you'll get a % character. Most commands begin with the backslash (\) character, which is also used in escaping other special characters, as we'll see in 4.4.2.

4.4.2 Escaping

Some characters like the % are special, and behave differently in LaTeX than in text documents. For example:

- The **%** starts comments.
- The **&** is used to separate columns in a row in tables.
- The **** either creates a new line without breaking the paragraph or starts a new row in a table environment.
- **{** and **}** to enclose into a group.
- **$** to write some math.
- **# _ ~ ^** as other reserved characters.

Most of these are escaped by adding the backslash character next to them (e.g. \&). But to print the backslash itself, only the `\textbackslash` command will print the '\'.

4.4.3 Formatting commands

Some LaTeX commands act like their word processor equivalents. For example:

This...	will print this
`\textbf{bold}`	**bold**
`\textit{italics}`	*italics*
`\underline{underline}`	underline
`\textbf{\textit{bold and italics}}`	***bold and italics***

Want to center something like the table above? Just type something like:

```
\begin{center}
    I'm a centered section!
    I start in the .tex file with \verb|\begin{center}|
    and end with \verb|\end{center}|.
\end{center}
```

and you'll get:

I'm a centered section! I start in the .tex file with `\begin{center}` and end with `\end{center}`.

Want to make a list? For unordered (bulleted) lists:

```
\begin{itemize}
    \item This is an item in an \textit{itemize} environment.
    \item Each item starts with \verb|\item|
          and continues until the next \verb|\item|
          or \verb|\end{itemize}|.
    \item You can also nest lists.
          \begin{itemize}
              \item Just open another \textit{itemize}
                    environment inside.
          \end{itemize}
\end{itemize}
```

will produce:

- This is an item.
- Each item starts with `\item` and continues until the next `\item` or `\end{itemize}`.
- You can also nest lists.
  - Just open another *itemize* environment inside.

If you want to make ordered lists, with numbers:

```
\begin{enumerate}
    \item This is an ordered item
          in an \textit{enumerate} environment.
    \item It behaves like in the \textit{itemize} environment.
          But instead of bullets that are the same for each item,
          you see sequences of numbers or letters.
    \item Once again, you may nest lists.
          \begin{enumerate}
              \item Just like that.
              \item But you can go deeper:
                    \begin{itemize}
                        \item You can even nest unordered lists.
                              \begin{enumerate}
                                  \item Just how deep can this
                                        rabbit hole go?
                              \end{enumerate}
                    \end{itemize}
          \end{enumerate}
    \item Phew, back at last!
\end{enumerate}
```

this is what you get:

1. This is an ordered item in an *enumerate* environment.
2. It behaves like in the *itemize* environment. But instead of bullets that are the same for each item, you see sequences of numbers or letters.
3. Once again, you may nest lists.
   (a) Just like that.
   (b) But you can go deeper:
   - You can even nest unordered lists.
     i. Just how deep can this rabbit hole go?
4. Phew, back at last!

To add footnotes, you can do something like this:

```
Hello there!\footnote{General Kenobi!}
```

to get:

Hello there![1]

Want to add URLs in your document? Simple! Just type:

```
\TeX{} Users Group homepage: \url{https://www.tug.org/}
```

to get:

TEX Users Group homepage: https://www.tug.org/

And speaking of links, if you wish to add a bibliographical reference as it's declared in the **references.bib** file, type:

```
The K-means algorithm\cite{bib:k_means_clustering_algorithm}
is a well-known clustering algorithm.
```

to render:

The K-means algorithm[1] is a well-known clustering algorithm.

Just pay attention to the label as it is given in the corresponding **references.bib** entry. A sample has been given in this project. Bibliographical entries don't have to begin with "bib:", but it helps in organizing labels.

Finally, if you want to link to another reference:

```
Go to Chapter \ref{figures_and_tables}.
```

will show up as:

Go to Chapter 5.

As you can see, this not only displays the number of the "FIGURES AND TABLES" chapter, but clicking on it navigates to the start of that chapter. This can be used to reference anything with the `\label{label_key}` command. Just tag something with `\label{key}`, then refer (link) to it with `\ref{key}`.

Tables and figures are analyzed in the next sample chapter. If you want to know more about advanced LATEX commands, there are a few tutorials online.

4.4.4 Adding math equations

LATEX also supports adding math formulae and equations. These expressions tend to use a different notation and font than the rest of the document.

There are two ways to add a math expression: *inline* and *display*.

[1] General Kenobi!

In inline mode, the math expression is written on the same paragraph as the text. For example, typing:

```
One of Albert Einstein's famous equations is \(E = mc^2\),
where \(E\) is energy,
\(m\) is mass,
\(c\) is the speed of light (approx. 300,000,000 m/s).
```

will yield:

One of Albert Einstein's famous equations is $E = mc^2$, where E is energy, m is mass, c is the speed of light (approx. 300,000,000 m/s).

In display mode, math expressions are rendered in their own blocks, usually centered. For instance, if one types:

```
Ohm's law is expressed as:
\[V = I R \quad \textrm{or} \quad
I = \frac{V}{R} \quad \textrm{or} \quad
R = \frac{V}{I} \]
where \(V\) is voltage, \(I\) is current, \(R\) is resistance.
```

this will output:

Ohm's law is expressed as:

$$V = IR \quad \text{or} \quad I = \frac{V}{R} \quad \text{or} \quad R = \frac{V}{I}$$

where V is voltage, I is current, R is resistance.

And if you want numbered equations:

```
The ideal gas law is defined as:
\begin{equation}
    p V = n R T
\end{equation}
where \(p\) is pressure of the gas,
\(V\) is the volume,
\(n\) the number of moles (not the animal kind),
\(R\) the gas constant,
\(T\) the temperature expressed in the Kelvin scale.
```

to display:

The ideal gas law is defined as:

$$pV = nRT \tag{4.1}$$

where p is pressure of the gas, V is the volume, n the number of moles (not the animal kind), R the gas constant, T the temperature expressed in the Kelvin scale.

5. FIGURES AND TABLES

5.1 Inserting tables

Below is a sample table with its caption. Make sure you put the caption **over** the table. For instance, this code in LaTeX:

```
\begin{table}
    % this is before the table
    \caption{A sample event log, sorted by timestamp}
    % this is to label and include it in our list of tables
    \label{tbl:event_log_sample}
    \begin{center} % center the table
    \begin{tabular}{c|c|c}
        \textbf{Case Id} & \textbf{Activity} & \textbf{Timestamp} \\
        \hline
        1 & Activity A & 2021-05-24T12:30:00Z \\
        1 & Activity B & 2021-05-24T12:30:26Z \\
        1 & Activity C & 2021-05-24T12:31:04Z \\
        2 & Activity A & 2021-05-24T13:19:39Z \\
        2 & Activity B & 2021-05-24T13:19:50Z \\
        3 & Activity A & 2021-05-24T13:19:59Z \\
        2 & Activity C & 2021-05-24T13:22:14Z \\
        3 & Activity B & 2021-05-24T13:24:24Z \\
        4 & Activity A & 2021-05-24T13:25:06Z \\
        3 & Activity C & 2021-05-24T13:25:27Z \\
        4 & Activity B & 2021-05-24T13:25:53Z \\
        4 & Activity C & 2021-05-24T13:27:48Z \\
        \hline
    \end{tabular}
    \end{center}
\end{table}
```

renders table 5.1:

Table 5.1: A sample event log, sorted by timestamp

Case Id	Activity	Timestamp
1	Activity A	2021-05-24T12:30:00Z
1	Activity B	2021-05-24T12:30:26Z
1	Activity C	2021-05-24T12:31:04Z
2	Activity A	2021-05-24T13:19:39Z
2	Activity B	2021-05-24T13:19:50Z
3	Activity A	2021-05-24T13:19:59Z
2	Activity C	2021-05-24T13:22:14Z
3	Activity B	2021-05-24T13:24:24Z
4	Activity A	2021-05-24T13:25:06Z
3	Activity C	2021-05-24T13:25:27Z
4	Activity B	2021-05-24T13:25:53Z
4	Activity C	2021-05-24T13:27:48Z

5.2 Inserting figures

Below is an example of inserting a figure with its caption. Make sure you put the caption **under** the figure. For example, this LaTeX code:

```
\begin{figure}
    % Define the figure file in the \includegraphics command
    % Must be relative to one of the paths
    % in the \graphicspath command at the top
    \includegraphics[width=\linewidth]{k-means_steps}
    % Place caption under the figure
    \caption{The steps of a k-means clustering algorithm.
        \cite{bib:k_means_clustering_algorithm}}
    % Label the caption to include in the list of figures
    \label{fig:k-means_steps}
\end{figure}
```

renders Figure 5.1:

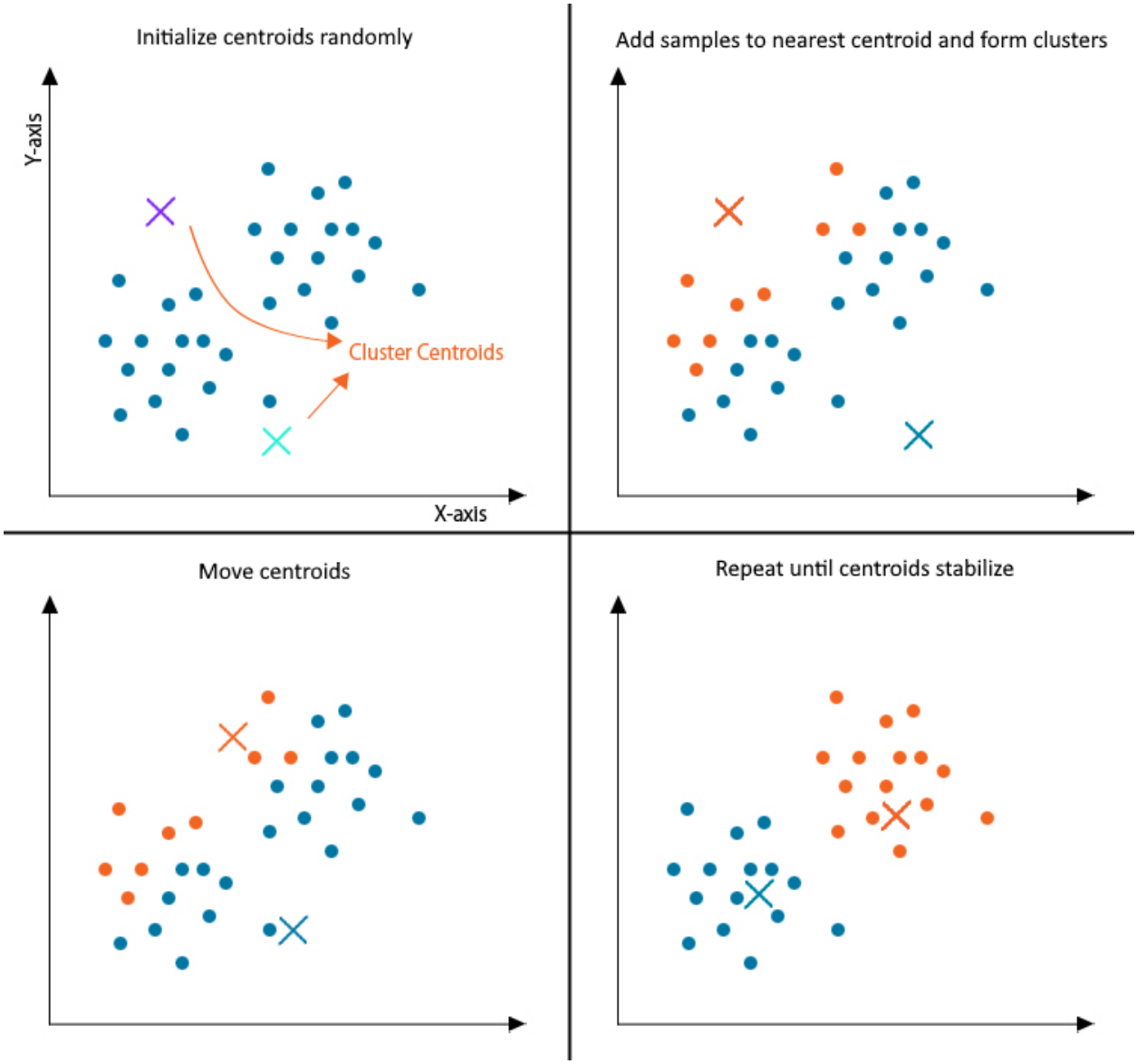

Figure 5.1: The steps of a k-means clustering algorithm. [1]

ABBREVIATIONS - ACRONYMS

AI	Artificial Intelligence
SPARQL	SPARQL Protocol and RDF Query Language
OWL	Web Ontology Language
OGC	Open Geospatial Consortium

APPENDIX A. FIRST APPENDIX

APPENDIX B. SECOND APPENDIX

REFERENCES

[1] K-Means Clustering Algorithm. EDUCBA. Accessed Feb. 08, 2022. [Online]. Available: https://www.educba.com/k-means-clustering-algorithm/